

Applications of the Bures-Wasserstein Distance in Linear Metric Learning

by

Davis Cooper

A thesis
submitted to the Victoria University of Wellington
in fulfilment of the
requirements for the degree of
Doctor of Philosophy
in Computer Science.

Victoria University of Wellington
2023

Abstract

Metric Learning has proved valuable in information retrieval and classification problems, with many algorithms using statistical formulations to their advantage. Optimal Transport, a powerful framework for computing distances between probability distributions, has seen significant uptake in Machine Learning and some applications in Metric Learning. The Bures-Wasserstein distance, a particular formulation of Optimal Transport, has seen limited application in Metric Learning despite being well suited to the field. In this thesis Linear Metric Learning algorithms incorporating the Bures-Wasserstein distance are developed, utilising its novel properties. The first set of algorithms we contribute use the Bures-Wasserstein distance for learning linear metrics using Cyclic Projections, Riemannian barycenters and gradient-based algorithms. The convergence properties and classification performance of these algorithms are then compared to benchmark Linear Metric Learning algorithms. Our algorithms achieve competitive classification performance on feature and image datasets and converge reliably.

The second set of contributions extend Linear Metric Learning with the Bures-Wasserstein distance to a low-rank context to address issues related to scalability. An extension of the previous Cyclic Projections algorithm and a Riemannian optimisation method are developed. Convergence and classification experiments analogous to the full-rank case are conducted to examine how the reduced rank affects both of these performance metrics. Both algorithms converge reliably under a variety of synthetic conditions. Reduction in rank generally decreased the training time for algorithms until effects caused by the size of the training data dominated. Reducing rank either left classification error unchanged or increased it. With respect to other low-rank Linear Metric Learning algorithms our algorithms again performed competitively with respect to classification error.

The last set of contributions consider Linear Metric Learning on Symmetric Positive Definite (SPD) matrices. Some datasets are expressed as or are suited to a SPD representation. As such, Metric Learning algorithms that respect the structure of these matrices have found success when adapted to this kind of data. Another extension of the Cyclic Projections algorithm is developed to use the Generalised Bures-Wasserstein distance on SPD matrices. Two other methods based on a ground metric formulation of the Bures-Wasserstein distance for learning linear metrics on SPD data are also introduced. One ground metric formulation uses an approximation (inspired by the Sinkhorn distance) to increase its scalability. Convergence experiments on synthetic problems adapted to SPD matrices are conducted on the ground metric algorithms to establish their convergence properties. Classification experiments are also conducted with respect to several types of SPD data. While the Cyclic Projections method is competitive, the ground metric algorithms have poor performance. Despite their poor classification performance, the approximation method does evaluate faster than the Generalised Bures-Wasserstein distance, potentially offering some scalability improvements.

Acknowledgments

I would first like to thank my supervisors Stephen Marsland and Marcus Fread for their support during my thesis. The guidance and support I have received from them has made this thesis possible. Being challenged to justify my work and methods has resulted in a significant amount of growth for me and has greatly improved the quality of my thinking. The administration staff in the ECS department have helped me through difficult situations on several occasions so I would like to thank them, particularly Patricia Stein and Diana Siwiak. I would also like to thank the technical staff in ECS for maintaining the computing resources that have been critical to my work, particularly Mark Davies. I would like to thank my examination committee for putting the time into reviewing my work and their valuable insights. I would also like to thank the numerous staff at Vic Books who have kept me highly-caffeinated throughout my time at Victoria. My family have been amazing through this process, keeping me sane and safe in too many instances to count. My mum and step-dad Dorianne and Peter, and my dad Geoff have given me so much love and support and I am eternally grateful to them for this. My friends Cam, Jordan, Tom, James and Jemima have allowed me to keep my perspective during my PhD, which has gotten me through some tough times. I also would like to thank the Victoria University of Wellington Tramping Club for allowing me to explore my other passion, the outdoors. The sense of community and friends I have gained from VUWTC will always be with me. Particular gratitude goes to: Michael, David, Hamish, Julia, Jackson, Lillian, Ivan and Florian. Finally, my wonderful girlfriend Alison, who completed her thesis along side me, has been my rock, my sunshine and the best adventure partner I could ask for.

Notation

- $\mathbb{N}$ - The set of natural numbers $\{1, 2, 3, \dots\}$.
- $[m]$ - The subset of natural numbers $\{1, 2, \dots, m\}$.
- $\mathbb{R}$ - Real numbers.
- $\mathbb{R}_+$ - Non-negative real numbers.
- $\mathbb{R}_{++}$ - Positive real numbers.
- $[0, 1] \subset \mathbb{R}$ - Closed unit interval between 0 and 1.
- $(0, 1) \subset \mathbb{R}$ - Open unit interval between 0 and 1.
- $\mathbb{R}^n$ - n dimensional real valued vectors.
- $\mathbb{R}_+^n$ - n dimensional real non-negative vectors.
- $\mathbb{R}_{++}^n$ - n dimensional real positive vectors.
- $\mathbb{R}^{n \times m}$ - n by m real valued matrices.
- Δ_n - n -dimensional simplex.
- S^n - Symmetric n by n matrices.
- S_+^n - n by n positive semidefinite (PSD) matrices.
- S_{++}^n - n by n symmetric positive definite (SPD) Matrices.
- $S_+(p, n)$ - n by n positive semidefinite matrices with rank p .
- $St(p, n)$ - n -dimensional Stiefel Manifold of rank p .
- $\mathcal{O}(n)$ - n dimensional orthogonal matrices.
- $\mathcal{X}, \mathcal{Y}$ - Generic sets.
- $\mathcal{X} \times \mathcal{Y}$ - Cartesian product of sets $\mathcal{X}$ and $\mathcal{Y}$.

- $\mathcal{M}$ - A Generic Manifold.
- $\mathcal{T}_X \mathcal{M}$ - Tangent space at $X \in \mathcal{M}$.
- $\mathcal{P}(\mathcal{X})$ - Space of probability distribution on $\mathcal{X}$.
- $X \in \mathcal{X}$ - Denotes membership of element X to a set $\mathcal{X}$.
- $\mathcal{X} \subseteq \mathcal{Y}$ - Indicates a subset relation between $\mathcal{X}$ and $\mathcal{Y}$.
- $\bar{\mathcal{X}}$ - Closure of $\mathcal{X}$.
- $X \succeq 0$ - Denotes X a member of S_+^n .
- $X \succ 0$ - Denotes X a member of S_{++}^n .
- $\langle x, y \rangle$ - Euclidean inner product between $x, y \in \mathbb{R}^n$.
- $\langle X, Y \rangle$ - Frobenius inner product between $X, Y \in \mathbb{R}^{m \times n}$.
- $\langle \eta_X, \zeta_X \rangle_x$ - Riemannian metric of tangent vectors $\eta_X, \zeta_X \in \mathcal{T}_X \mathcal{M}$ at $X \in \mathcal{M}$.
- $\sqrt[p]{x}$ - p -th root of real number x .
- $\|\cdot\|_1$ - l_1 -norm.
- $\|\cdot\|_2$ - Euclidean or l_2 norm.
- $\|\cdot\|_F$ - Frobenius norm.
- $\|\cdot\|_M$ - Generalised Mahalanobis distance with respect to $M \in S_+^n$.
- I_p - p by p identity matrix.
- A^T - Matrix transpose of matrix $A \in \mathbb{R}^{n \times m}$.
- A^{-1} - Matrix inverse of matrix $A \in \mathbb{R}^{n \times m}$.
- $A^{\frac{1}{2}}$ - Matrix square root of matrix $A \in \mathbb{R}^{n \times n}$.
- $\text{vec}(A)$ - Vectorised matrix $A \in \mathbb{R}^{n \times m}$.
- $\text{diag}(x)$ - Diagonal matrix in $\mathbb{R}^{n \times n}$ of $x \in \mathbb{R}^n$.
- $\text{diag}(X)$ - Vector of diagonal entries of matrix $X \in \mathbb{R}^{n \times n}$.
- $\text{trace}(A)$ - Trace of square matrix $A \in \mathbb{R}^{n \times n}$.
- $\log(A)$ - Matrix logarithm.
- $\log \det(A)$ - Log-determinant function.
- $\text{rank}(A)$ - The rank of matrix A .

- $\text{range}(A)$ - The linear subspace spanned by the columns of A .
- AB - Matrix multiple of matrices $A, B \in \mathbb{R}^{n \times m}$.
- A_{ij} - ij -th entry of matrix A .
- u, v - Generic probability distributions in $\mathcal{P}(\mathcal{X})$.
- $P_{\mathcal{X}}$ - Marginalisation operator onto $\mathcal{X}$ for probability distributions.
- $f : \mathcal{X} \rightarrow \mathcal{Y}$ - A function from set $\mathcal{X}$ to set $\mathcal{Y}$.
- $g \circ f$ - Composition of functions f and g .
- $\frac{df}{dX}$ - Euclidean derivative of function f with respect to variable X .
- ∇f - Euclidean gradient of function f .
- $\text{grad} f$ - Riemannian gradient of f .
- $\sum_{i=1}^n x_i$ - Sum of elements x_i i.e. $x_1 + x_2 + \dots + x_n$.
- $x \odot y$ - Entry-wise multiplication between vectors $x, y \in \mathbb{R}^n$.
- $x \oslash y$ - Entry-wise division between vectors $x, y \in \mathbb{R}^n$.
- $s.t.$ - short hand for 'Subject to'.

Table of Algorithms

Name	Acronym	Data & Metric Type	Optimisation Algorithm	Ref
Bures-Wasserstein Metric Learning	BWML	$\mathbb{R}^n / S_{++}^n$	Cyclic Projections	§3.2.1
Averaged - Barycentric Bures-Wasserstein Metric Learning	A-BWBML	$\mathbb{R}^n / S_{++}^n$	Barycentric	§3.2.2
Sequential - Barycentric Bures-Wasserstein Metric Learning	S-BWBML	$\mathbb{R}^n / S_{++}^n$	Barycentric	§3.2.2
Transport - Bures-Wasserstein Metric Learning	T-BWML	$\mathbb{R}^n / S_{++}^n$	PENLAB	§3.2.3
Transport - Bures-Wasserstein Metric Learning - Similarity	T-BWML-S	$\mathbb{R}^n / S_{++}^n$	L-BFGS	§3.2.3
Low-Rank - Bures-Wasserstein Metric Learning	LR-BWML	$\mathbb{R}^n / S_{++}^p, p < n$	Cyclic Projections	§4.2.1
Bures-Wasserstein - Logistic Discriminant Metric Learning	BW-LDML	$\mathbb{R}^n / S_+(p, n), p < n$	Riemannian BFGS	§4.2.2
Symmetric Positive Definite - Bures-Wasserstein Metric Learning	SPD-BWML	S_{++}^n / S_{++}^n	Cyclic Projections	§5.2.1
Ground Metric - Bures-Wasserstein Metric Learning	GM-BWML	S_{++}^n / S_{++}^n	SDPT3 & Riemannian BFGS	§5.2.2
Sinkhorn - Ground Metric - Bures-Wasserstein Metric Learning	S-GM-BWML	S_{++}^n / S_{++}^n	Alternating Projections & Riemannian BFGS	§5.2.2

x

Contents

1	Introduction	1
1.1	Motivation	5
1.2	Aims	6
1.3	Roadmap	7
2	Background and Literature Review	11
2.1	Metrics: Distances and Divergences	12
2.2	Learning: Convex Analysis and Optimisation	19
2.3	Optimal Transport	34
2.4	Literature Review	39
3	Bures-Wasserstein Metric Learning	57
3.1	Introduction	57
3.2	Learning a Metric with the Bures-Wasserstein Distance	59
3.2.1	Bures-Wasserstein Metric Learning	60
3.2.2	Bures-Wasserstein Barycentric Metric Learning	71
3.2.3	Transport Bures-Wasserstein Metric Learning	77
3.3	Implementation	82
3.4	Experiments	82
3.4.1	Synthetic Data	83
3.4.2	UCI Datasets	91
3.4.3	Image Datasets	95
3.5	Conclusion	96

4	Low-rank Bures-Wasserstein Metric Learning	99
4.1	Introduction	99
4.2	Methods	101
4.2.1	Low-Rank BWML	101
4.2.2	Bures-Wasserstein Logistic-Discriminant Metric Learning	105
4.3	Implementation	108
4.4	Experiments	108
4.4.1	Synthetic Data	109
4.4.2	UCI Datasets	124
4.4.3	Image Datasets	128
4.5	Conclusion	133
5	Symmetric Positive Definite Matrix Metric Learning	135
5.1	Introduction	135
5.2	Methods	138
5.2.1	SPD - Bures-Wasserstein Metric Learning	139
5.2.2	Ground Metric Bures-Wasserstein Metric Learning	142
5.3	Implementation	147
5.4	Experiments	148
5.4.1	Sinkhorn Bures-Wasserstein Convergence	149
5.4.2	GM-BWML and S-GM-BWML Convergence	153
5.4.3	k -NN Classification Tests	163
5.5	Conclusion	169
6	Conclusion	173
6.1	Future Work	176
6.2	Final Remark	178

Chapter 1

Introduction

The ability to quantify similarity between datapoints underlies many Machine Learning tasks, such as classification [1, 2], clustering [3, 4] and information retrieval [5, 6]. Unfortunately, there is no canonical way to quantify similarity on many of the datasets containing images, audio and text, making the above tasks difficult to do well. Metric Learning, a well-established sub-field of Machine Learning, seeks to answer this problem. By learning a distance function using data to quantify similarities, Metric Learning can capture features that appear intuitive, but are difficult to describe. One of the defining characteristics of Metric Learning is that it utilises the concept of similarity using relations between datapoints, where two datapoints are similar if they share a classification label. The complementary relation to similarity is dissimilarity, which relates datapoints that do not have the same classification label. A parameterised distance function is then optimised so that the distance between similar data is minimised and the distance between dissimilar data is maximised. The learnt distance function may then be used in algorithms such as k -nearest-neighbours classification [1] or k -means clustering [3]. These simple principles have spawned a large number of diverse algorithms suited to different tasks, datasets and disciplines [7, 8, 9].

Here we provide a brief overview of Metric Learning and Optimal Transport leaving a more detailed introduction for chapter two. The first true Metric Learning al-

gorithm, Mahalanobis Metric Learning for Clustering (MMC) [10], parameterised the Generalised Mahalanobis distance (see §2 - (2.1)) [11] as a learnable distance function. MMC is a Linear Metric Learning algorithm, as learning a Generalised Mahalanobis distance can be interpreted as learning a linear transformation on the data. In addition to this, MMC introduced and formalised similarity and dissimilarity relations on datasets. Many Linear Metric Learning algorithms have been formulated using the Generalised Mahalanobis distance, similarity, and dissimilarity relations. Significant contributions include: learning metrics using statistical regularisers [12, 13], relative distance constraints [14], non-linear extensions [15] and applications adapted for large-scale face re-identification [16, 17]. Information-theoretic methods in particular have had success within Linear Metric Learning [12, 13, 18, 16]. These methods model the similarity and dissimilarity relationships of the dataset as probability distributions and use the Kullback-Leibler divergence [19], or a related divergence, to train a Generalised Mahalanobis distance.

While a significant amount of research has been devoted to training the Generalised Mahalanobis distance for Metric Learning, this is certainly not the only learnable distance function considered in Metric Learning. For instance, Deep Metric Learning [8, 20] uses a neural network to transform input data, where the Euclidean distance between transformed datapoints serves as the distance function. While this is not strictly a distance function on the dataset, Deep Metric learning uses many of the principles of Metric Learning such as similarity and dissimilarity relations, and achieves superior performance on classification tasks. Metric Learning on Symmetric Positive Definite (SPD) matrix data is another extension of Metric Learning. For example, data such as textures [21], human actions [22] and medical images [23] benefit from the second-order information provided by a SPD representation. However, SPD matrices are a manifold and as such there are a variety of ways to define distances on them. While there are Metric Learning algorithms for SPD matrices [24, 25, 26], it is a less mature area than Metric Learning with the Generalised Mahalanobis distance.

Optimal Transport is another type of distance, which has seen significant attention from the Machine Learning community [27, 28]. Optimal Transport is one of the few frameworks that allows the study of probability distributions from a geometric viewpoint [29] and is a diverse topic that touches on many topics such as Control Theory [30], Economics [31] and Fluid Mechanics [32]. Optimal Transport has its origins in the work of Gaspard Monge in 1781 [33]. Monge studied how to most efficiently distribute resources from source to destination, such as moving metal ore from a mine to a factory where it can be processed. Intuitively, there is a cost associated with the transportation of material, proportional to its mass, from a source location to destination. Monge formalised this problem as a discrete optimisation problem that accounts for cost of transportation, allowing it to be studied in a rigorous way.

Contemporary Optimal Transport was introduced by Leonid Kantorovich [34], who provided a generalisation of the Monge problem. Kantorovich instead considered many-to-one transportation between source and destination, and transportation between probability distributions. This formulation, known as the Kantorovich-Rubinstein formulation, is far more flexible than the Monge problem and remains a primary formulation of contemporary Optimal Transport.

A common form of the Kantorovich-Rubinstein formulation is the Wasserstein distance, in which the transport cost is described by a distance function. The discovery of the Wasserstein distance is credited to several authors [35, 36, 37], including Kantorovich, though its namesake is Leonid Vaserstein [38]. Following Kantorovich's work, there has been a significant amount of research in establishing different properties of Optimal Transport and Wasserstein distances [39, 28]. This includes notable achievements in solubility of the particular Wasserstein distances [40], Riemannian geometry of Optimal Transport [29] and Gradient Flows in probability spaces [41, 42]. Optimal Transport, and particularly Wasserstein distances, have enjoyed significant uptake in Computer Vision and Machine

Learning. Landmark applications include the Earth Mover’s Distance for image retrieval [43] the Wasserstein GAN [44], 3D geometry processing [45] and classification algorithms [46].

Typically, Wasserstein distances are expressed as optimisation problems. However there exists a special case of the Wasserstein distance known as the Bures-Wasserstein distance, which has a closed-form expression. The Bures-Wasserstein distance, also known as the Frechét distance [47], is a Wasserstein distance between two zero-mean multivariate Gaussian distributions. The Bures-Wasserstein distance also has applications in Machine Learning including generative model evaluation [48], point embeddings [49], Bayesian inference [50] and domain adaptation [51].

This thesis considers the use of the Bures-Wasserstein distance for Metric Learning. While Optimal Transport has been used for Metric Learning applications [52, 53, 54], there has been very limited study of the Bures-Wasserstein distance for Metric Learning [55]. To the best of our knowledge, this thesis is the first in-depth study into the applications of the Bures-Wasserstein distance for Metric Learning. We investigate and present Linear Metric Learning methods and not Deep Metric Learning methods. We have consciously made this choice for several reasons. Firstly, it is difficult to perform analysis on the convergence properties of deep methods since they are typically highly non-convex. Many of the methods we examine are amenable to theoretical analyses, which will give a better basis for understanding how the Bures-Wasserstein distance influences the Metric Learning problem. Secondly, linear models are less data hungry than deep models [56] and may perform better when presented with limited data. Thirdly, recent critiques of the experimental methodology in the Deep Metric Learning literature [57, 58, 59, 60] indicate a lack of reliability in the results being published. By studying Linear Metric Learning methods we can have more confidence in the reliability of our results relative to the literature.

We proceed with our motivation for the application of the Bures-Wasserstein distance in Metric Learning, followed by a description of the aim(s) of this thesis, and a roadmap describing our contributions.

1.1 Motivation

The Bures-Wasserstein distance possesses many advantageous properties for Metric Learning:

- In Machine Learning, algorithms that use Optimal Transport instead of an information-theoretic divergence often find increases in performance and training reliability. As expressed earlier, some significant examples of this are the Earth Mover’s Distance [43], Word Mover’s Distance [61], Wasserstein GAN [44] and multi-label classification [46]. Algorithms that use Optimal Transport for Metric Learning have also demonstrated benefits to performance [62, 52, 63]. Given the prevalence of information-theoretic methods in Metric Learning, there are many options to apply this type of extension using the Bures-Wasserstein distance. Because of this, we believe the Bures-Wasserstein distance has great potential to improve the performance of Metric Learning algorithms.
- Statistical divergences between SPD matrices are often used to train the Generalised Mahalanobis distance for Metric Learning [12, 64, 65]. These distances have an affine-invariance property, meaning that they are invariant to affine transforms of their arguments. This is not a beneficial property in the context of Linear Metric Learning, since linear transformations are less distinguishable with respect to the divergence. In comparison, the Bures-Wasserstein distance is unitarily invariant i.e. it is invariant to rotations of the data and not more general linear transforms. This enables the Bures-Wasserstein distance to distinguish between a larger set of linear transformations relative to the statistical divergences, allowing for finer-grained comparisons between metrics.

- Almost all of the Metric Learning algorithms incorporating Optimal Transport do so using the Kantorovich-Rubenstein formulation. There exist algorithms specifically designed for evaluating discrete Wasserstein distances [66, 67] as well as fast entropic approximation methods [68]. However, solving these optimisation problems can be taxing on the overall learning algorithm. The Bures-Wasserstein distance on the other hand is a closed-form expression and is much less burdensome to evaluate. Additionally, the Bures-Wasserstein distance is convex with respect to one variable argument and one fixed argument, unlike many of the other formulations of Wasserstein distances. Thus, the Bures-Wasserstein distance is amenable to convex optimisation algorithms, increasing its efficiency versus more general Wasserstein distances.
- The Bures-Wasserstein distance also has an associated Riemannian geometry on the manifold of SPD matrices [69]. The benefits of this are two-fold. Firstly, the Bures-Wasserstein distance can be generalised to a parameterised distance on SPD matrices [55]. This means it may be used to learn a distance for SPD Metric Learning. There are very few parameterised distances on SPD matrices and so the Bures-Wasserstein distance presents a valuable tool for this type of Metric Learning. Secondly, the geometric nature of the Bures-Wasserstein distance provides a more natural treatment of the SPD manifold, in contrast to many Metric Learning algorithms that use Euclidean structures on SPD matrices. We believe that using a geometry that is natural to the SPD manifold during optimisation may have beneficial properties for Metric Learning.

1.2 Aims

The primary aim of this thesis is to investigate the use of the Bures-Wasserstein distance for Linear Metric Learning algorithms. There are several qualities that we wish to investigate with respect to our algorithms:

Classification Error. The primary application we study for our Metric Learning algorithms in this thesis is the learning of distances for use in k -nearest-neighbours (k -NN) classification [1]. Therefore, we wish to achieve reduced k -NN classification error when using Bures-Wasserstein Metric Learning algorithms.

Scalability. We aim to design algorithms that can scale to modern datasets, in both the dimension and number of examples in the dataset. For an algorithm to be relevant to contemporary Metric Learning it should be able to operate on modern datasets.

Convergence. We also aim to design algorithms that reliably converge to a local optimum in a feasible amount of time. Some algorithms in Machine Learning can be difficult and unreliable to train. As part of this thesis, we seek to understand how our algorithms behave during training and under what conditions they reach a solution.

Utilise Properties of Bures-Wasserstein. We aim to explore the different properties of the Bures-Wasserstein distance and apply them in some beneficial way to learning a metric. For instance, existing concepts associated with the Bures-Wasserstein distance such as Barycentre computation (§2 - Equation (2.15)) and its Riemannian geometry are potentially applicable to Metric Learning.

Applicability to Different Types of Data. Datasets may be recorded and expressed in a myriad of ways. As such one of our aims is to create Metric Learning algorithms that may be adapted to different types of data. Primarily we are interested in learning metrics on feature data, image data and SPD matrix data.

1.3 Roadmap

Following this introduction, the necessary background material on Metric Learning and Optimal Transport is reviewed in Chapter two. This material provides

the essential tools for the algorithms derived in later chapters. A survey of the literature relevant to metric learning with the Bures-Wasserstein distance follows, including a detailed description of the algorithms used for experimental comparison later in the thesis.

Chapter three forms the basis for Metric Learning with the Bures-Wasserstein distance in this thesis. Three classes of Metric Learning algorithm that use the Bures-Wasserstein distance to learn a Generalised Mahalanobis distance are derived and introduced. These algorithms differ in the way that they optimise for a metric. Specifically, they are projection-based, barycentre-based and gradient-based algorithms. Where the relevant theory is available, convergence proofs are provided. The results of empirical convergence experiments on our algorithms are then given to complement the theoretical analysis and comparisons are made to existing Metric Learning algorithms. Results on k -NN classification experiments on feature and image datasets are then provided.

Chapter four considers an extension of the work in the earlier chapter to learning low-rank metrics with the Bures-Wasserstein distance. The motivation for learning a low-rank metric is to increase scalability by reducing the dimensionality of the learning problem. One of the algorithms from the previous chapter is extended to learning in the low-rank setting. Another method, utilising a Riemannian geometry associated with a low-rank version of the Bures-Wasserstein distance, is also introduced. As in the previous chapter, the results of convergence and k -NN experiments for these algorithms are presented.

Chapter five considers Linear Metric Learning with respect to data comprised of SPD matrices. This type of data has become relevant in recent years and the Bures-Wasserstein distance provides a metric for learning in this paradigm. Another extension of an algorithm from the first contribution chapter is derived for learning in the SPD matrix setting. An algorithm for learning a generalisation of the Bures-Wasserstein distance is also introduced. The complexity associated with learning

this generalisation is quite high, so an approximation to it is also introduced. The combination of this approximation and the generalisation forms another learning algorithm. Convergence tests on the two algorithms using the generalisation of the Bures-Wasserstein distance are conducted. k -NN classification experiments on several SPD datasets are run on our algorithms, in comparison to some existing SPD Matrix Metric Learning methods.

The thesis concludes with a discussion of the contributions with respect to the aims, and considerations for future work.

Chapter 2

Background and Literature Review

The work in this thesis lies at the intersection of Optimal Transport and Linear Metric Learning. In this chapter we review the relevant elements of each of these fields. There are two main sections: first an introduction to the fundamental concepts of Metric Learning and Optimal Transport, and then a review of relevant Linear Metric Learning literature.

The necessary concepts for Metric Learning involve descriptions of different types of metric, how to construct learning problems using these metrics and how to solve them. The background we provide on Metric Learning follows this sequence of concepts. A brief overview of Optimal Transport and the Bures-Wasserstein distance is then given. The second section of this chapter is a review of the Metric Learning literature relevant to this thesis, which includes established algorithms that are used in later chapters. Specific focus is given to foundational Metric Learning, Information-Theoretic Metric Learning, Symmetric Positive Definite Matrix Metric Learning and Optimal Transport based Metric Learning algorithms. Note that a notation section is included in the front matter (pg v) for reference.

2.1 Metrics: Distances and Divergences

The metrics used in this study take the form of parameterised distances. Because we study Linear Metric Learning in particular, these parameters are described by linear transforms. In this section we introduce the Generalised Mahalanobis distance, which is the metric that features most heavily in the algorithms we discuss. Other types of distance used for Metric Learning are also introduced here. However these distances are much less prevalent than the Generalised Mahalanobis distance. For the sake of clarity their parameterised versions are introduced, with their respective algorithms, in the literature review. Some of these distances are also used to define Metric Learning algorithms that learn a Generalised Mahalanobis distance. Divergences, a related concept to distances, are also introduced here. In the context of Metric Learning, divergences are frequently used to define Metric Learning problems, but generally are not used as metrics themselves. We now provide definitions for both of these concepts:

Definition 2.1.1 *A distance on a set $\mathcal{X}$ is a function $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$ such that $\forall x, y \in \mathcal{X}$:*

1. *(Equivalence):* $d(x, y) = 0$ iff $x = y$
2. *(Symmetry):* $d(x, y) = d(y, x)$
3. *(Triangle Inequality):* $d(x, y) + d(y, z) \geq d(x, z)$

Definition 2.1.2 *A function $D : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$ that only adheres to the equivalence condition above is a divergence.*

For both distances and divergences, if two inputs $x, y \in \mathcal{X}$ are given we can use them to discern if x and y are equal, and the output of both functions is non-negative.

Vectors

A set of real values can be concatenated into a list forming a vector x where x_i is the i -th real value. The real vectors of length n are elements of the finite dimensional Euclidean space $\mathbb{R}^n$. A set of distances between vectors $x, y \in \mathbb{R}^n$ that adheres to the conditions of Definition 2.1.1 is:

$$d_p(x, y) = \sqrt[p]{\sum_{i=1}^n |x_i - y_i|^p},$$

where p is a positive integer. These are referred to as the l_p distances on $\mathbb{R}^n$. These distances are also written as $\|x - y\|_p$. The l_p norm $\|x\|_p = d_p(x, 0)$ describes the magnitude of x with respect to d_p . A common choice of l_p distance is $d_2(x, y) = \sqrt{\sum_{i=1}^n |x_i - y_i|^2}$ where $p = 2$, typically referred to as the Euclidean distance.

The Euclidean distance is a commonly-used distance since it is computationally cheap to use and admits many simple or closed-form expressions. It is the basis of the mean-squared error, which is used to train many Machine Learning models [70]. The l_2 and l_1 norms have seen a variety of applications as regularisers such as Lasso regression [71] and Elastic Net [72]. The l_1 norm was also the driving factor for the compressed sensing revolution [73].

A linear transformation $L \in \mathbb{R}^{p \times n}$ can be applied to each input before computing the norm i.e. for $x, y \in \mathbb{R}^n$, $d_2(Lx, Ly) = \|Lx - Ly\|_2$. If the linear transform is such that $L^T L = \Sigma^{-1}$, where Σ is the covariance matrix of a dataset, this is the Mahalanobis distance [11] with respect to the data. If L is a general linear transform then this is referred to as the Generalised Mahalanobis distance. The Generalised Mahalanobis distance is a parameterised distance, normally parameterised by $L^T L = M \in S_+^n$, a symmetric positive definite matrix. It is defined as:

$$\|x - y\|_M = \sqrt{(x - y)^T M (x - y)}. \quad (2.1)$$

In the case that M is rank-deficient, i.e. $\text{rank}(M) \leq n$, the Generalised Mahalanobis distance does not satisfy the first condition of the definition of a distance. In this case, the Generalised Mahalanobis distance may be interpreted as a distance on a linear subspace of $\mathbb{R}^n$, which has useful implications. In the literature review we will show how the Generalised Mahalanobis distance provides the basis for many Linear Metric Learning algorithms.

Many kinds of divergence can also be defined on Euclidean space. A predominant class of divergences are Bregman divergences [74], which are induced by strictly convex functions (see Definition 2.2.3) $\phi : \mathcal{X} \rightarrow \mathbb{R}$, defined as:

$$D_\phi(x, y) = \phi(x) - \phi(y) - \langle \nabla \phi(y), x - y \rangle, \quad (2.2)$$

where $\langle \cdot, \cdot \rangle$ is the Euclidean inner product. The l_2 norm may also be expressed as a Bregman divergence i.e. $D_\phi(x, y) = \|x - y\|_2^2$ if $\phi(x) = \|x\|_2^2$. Another prevalent Bregman divergence is the Kullback-Liebler (KL) divergence [75]:

$$D_{KL}(x, y) = \sum_{i=1}^n x_i \log \left(\frac{x_i}{y_i} \right) - x_i + y_i,$$

which is induced by the negative entropy function $\phi(x) = \sum_{i=1}^n x_i \log(x_i)$. This divergence is valid on the set of positive vectors i.e. $x \in \mathbb{R}_{++}^n$. If we consider the set of discrete probability distributions $\Delta_n = \{x \in \mathbb{R}_{++}^n \text{ s.t. } \sum_{i=1}^n x_i = 1\}$ the KL-divergence can be interpreted as a measure of difference between two probability distributions. Typical algorithms such as projection and clustering have seen extensions to Bregman divergences [76, 77] with significant success.

We now broaden these concepts of distance and divergence to distances and divergences on matrices.

Matrices

Distances and divergences that are analogous to vector distances may be defined for matrices. One method is to treat a matrix as a vector, by stacking its columns

and then computing the corresponding vector distance. This is the case for the Frobenius distance. The Frobenius distance between matrices $A, B \in \mathbb{R}^{m \times n}$ is:

$$\begin{aligned} d_F(A, B) &= \left(\sum_{i=1}^m \sum_{j=1}^n (A_{ij} - B_{ij})^2 \right)^{1/2} \\ &= \sqrt{\text{trace}((A - B)^T (A - B))}, \end{aligned}$$

also written as $\|A - B\|_F$. If the exponent of 2 and its reciprocal in the above expression are replaced with another positive integer we arrive at a family of distances that are analogous to l_p distances. In this way matrices have not been treated all that differently to vectors, as $\|A - B\|_F = \|\text{vec}(A) - \text{vec}(B)\|_2$.

Matrix Bregman divergences can also be induced by entry-wise functions, where familiar divergences may be defined. For example, the entry-wise Bregman matrix divergence between matrices with positive entries $A, B \in \mathbb{R}_{++}^{m \times n}$ is defined as:

$$D_{KL}(A||B) = \sum_{i=1}^n \sum_{j=1}^m A_{ij} \log \left(\frac{A_{ij}}{B_{ij}} \right) - A_{ij} + B_{ij},$$

which is equivalent to the KL-divergence between vectors $\text{vec}(A), \text{vec}(B)$. Both the Frobenius distance and KL-divergence are Bregman divergences with analogous definitions to equation (2.2) where the vector inner product is replaced with the matrix inner product ($\langle A, B \rangle = \text{trace}(A^T B)$).

While distances and divergences on matrices may be defined following the kinds of extensions shown above, other interesting divergences may be defined by restricting their definition to subsets of real matrices. An important subset of the real n by n matrices are symmetric matrices:

$$S^n = \{X \in \mathbb{R}^{n \times n} : S^T = S\}.$$

A subset of symmetric matrices are the positive semidefinite (PSD) matrices:

$$S_+^n = \{X \in S^n : y^T X y \geq 0, \forall y \in \mathbb{R}^n\}.$$

Similarly, the symmetric positive definite (SPD) matrices are:

$$S_{++}^n = \{X \in S^n : y^T X y > 0, \forall y \neq 0 \in \mathbb{R}^n\},$$

which is a subset of S_+^n where the inequality in the definition is strict.

The majority of Metric Learning algorithms presented in this thesis learn a Generalised Mahalanobis distance, and as such are presented as learning problems on subsets of S_+^n . Therefore we now focus on metric structures on S_+^n . Bregman divergences for positive semidefinite and symmetric positive definite matrices are defined similarly to previous Bregman matrix divergences using equation (2.2) [75]. A predominant example is the log-determinant divergence D_{ld} , induced by the function $\phi(A) = -\log \det(A)$:

$$D_{ld}(A||B) = \text{trace}(AB^{-1}) - \log \det(AB^{-1}) - n, \quad (2.3)$$

which is defined on the set of symmetric positive definite matrices. Another example is the von Neumann divergence, induced by $\phi(A) = \text{trace}(A \log(A) - A)$:

$$D_{vN}(A||B) = \text{trace}(A(\log(A) - \log(B)) - A + B).$$

For both of these divergences the inducing function ϕ is no longer defined as an entry-wise function. In this case, the inducing functions for these divergences are only valid for SPD matrices. It is worth noting that the set of positive semidefinite matrices has a one-to-one relationship with the set of n -dimensional zero-mean Gaussian distributions, characterised by their correlation matrix $\Sigma \in S_+^n$. In light of this, the log-determinant divergence is equivalent to the KL-divergence between two zero-mean, non-degenerate Gaussian distributions. Bregman divergences have also seen application in matrix nearness problems such as non-negative matrix approximation [78], covariance estimation [79] and image and texture classification [80].

Matrix Manifolds

While we have already defined some divergences on S_+^n and S_{++}^n , defining distances on these sets and related sets requires a different approach. The set of n by n SPD matrices is a subset of the real n by n matrices, so the Frobenius distance is a distance on this set. However, it does not incorporate the underlying geometry of SPD matrices [81]. In some cases, using the Frobenius norm for data analysis gives anomalous results, such as the swelling-effect [82, 23]. This is because SPD matrices are a matrix manifold. This motivates us to introduce distances defined on them. Since matrix manifolds are non-linear structures, the shortest path between matrices $A, B \in S_{++}^n$ is no longer the straight line $l(t) = (1 - t)A + tB$. We introduce the matrix manifolds relevant to our study and some of the distances that may be induced on them. Later, when we introduce the learning components of Metric Learning, the general structure of manifolds will be reviewed.

The distance between symmetric positive definite matrices at the core of this thesis is the Bures-Wasserstein distance [83]:

$$\begin{aligned} \mathcal{B}(A, B) &= \sqrt{\text{trace}(A + B - 2(A^{\frac{1}{2}}BA^{\frac{1}{2}})^{\frac{1}{2}})} \\ &= \min_{U \in \mathcal{O}(n)} \|A^{\frac{1}{2}} - B^{\frac{1}{2}}U^T\|_F, \end{aligned} \quad (2.4)$$

where $\mathcal{O}(n) = \{U \in \mathbb{R}^{n \times n} : U^T U = U U^T = I_n\}$ is the set of n by n orthogonal matrices. The relationship of the Bures-Wasserstein distance to Optimal Transport will be presented later in the Optimal Transport section (§2.3).

The Affine-Invariant (AI) distance [81, 23] is also a distance on S_{++}^n , expressed as:

$$d_{AI}(A, B) = \|\log(A^{-\frac{1}{2}}BA^{-\frac{1}{2}})\|_F. \quad (2.5)$$

The Affine-Invariant distance is named as such because it has the property that the distance is unchanged under matrix multiplication on both sides, i.e. $d_{AI}(PAP^T, PBP^T) =$

$d_{AI}(A, B)$ for an invertible matrix $P \in \mathbb{R}^{n \times n}$. Another distance on S_{++}^n is the Log-Euclidean distance [82], expressed as:

$$d_{LE}(A, B) = \|\log(A) - \log(B)\|_F. \quad (2.6)$$

The Log-Euclidean distance maps SPD matrices to the set of symmetric matrices using the matrix logarithm, where the Frobenius or Euclidean distance is then applied to compute the distance. The Log-Euclidean distance is generally cheaper to work with than the Affine-Invariant distance due to the linear nature of its construction. Both the log-Euclidean and AI distances were introduced in the context of Diffusion Tensor Imaging, but they have also been used for classification of other kinds of SPD data [22, 26].

The set of semidefinite matrices does not form a manifold. However, if this set is stratified into the subsets of fixed-rank positive semidefinite matrices, then each subset forms a matrix manifold, i.e.

$$S_+(p, n) = \{X \in S_+^n : \text{rank}(X) = p\}$$

is a manifold. $S_+(p, n)$ is distinct from S_{++}^n and has its own set of associated distances [84, 85, 86]. There is also a distance on these sets equivalent to the Bures-Wasserstein distance [87]. For $Y_0, Y_1 \in \mathbb{R}^{p \times n}, p < n$ such that $A = Y_0^T Y_0, B = Y_1^T Y_1 \in S_+(p, n)$ the Bures-Wasserstein distance between them is:

$$\mathcal{B}(Y_0, Y_1) = \min_{U \in \mathcal{O}(p)} \|Y_0 - UY_1\|_F,$$

This is essentially the same as expression (2.4), except that the orthogonal matrix U is p by p instead of n by n . Example applications of this distance include distance matrix completion [88], action recognition [89] and MEG/EEG regression [90].

The final matrix manifold we introduce is the Stiefel manifold [91]:

$$St(p, n) = \{V \in \mathbb{R}^{p \times n} : VV^T = I_p\},$$

where I_p is the $p \times p$ identity matrix. The Stiefel manifold is composed of p , n -dimensional vectors that are orthonormal to each other, which makes it ideal for applications to generalised eigenvalue problems [92]. A distance function for the Stiefel manifold is not provided here as it does not have a convenient closed-form solution, nor is a distance on the Stiefel manifold used in the algorithms we consider. In this study we primarily use the Stiefel manifold to cast dimensionality reduction tasks in an optimisation framework [93].

Now that we have described all of the necessary distances and divergences for our study we turn to a description of convex analysis and optimisation. These tools describe how we construct optimisation problems, how to solve them, and under what circumstances the algorithms are successful. Using this information we will be able to describe Metric Learning in the literature review and contribution chapters.

2.2 Learning: Convex Analysis and Optimisation

In Linear Metric Learning there are many ways to learn a metric and in this thesis we will express learning linear metrics as optimisation problems. An optimisation problem for Linear Metric Learning requires: a dataset $\mathcal{D}$ which we wish to learn a metric on, a set $\mathcal{X}$ of metrics and an objective function $f : \mathcal{X} \rightarrow \mathbb{R}$. In the algorithms we present $\mathcal{X}$ will be one of: the set of SPD matrices S_{++}^n , the set of positive semidefinite matrices S_+^n , the set of fixed-rank positive semidefinite matrices $S_+(p, n)$ or the Stiefel manifold $St(p, n)$. Constraints on the set of metrics will also be used frequently, which take the form of equality or inequality constraints with respect to linear maps on $\mathcal{X}$. Linear maps on $\mathcal{X}$ are analogous to linear maps $A : \mathbb{R}^n \rightarrow \mathbb{R}^m$ on real n -dimensional vectors, where $(Ax)_i = a_i^T x$ and a_i is the i -th row of A . For a linear map $\mathcal{A} : \mathcal{X} \rightarrow \mathbb{R}^m$, its i -th entry is defined as $(\mathcal{A}X)_i = \text{trace}(A_i^T X)$ for $X \in \mathcal{X}$, where $A_i \in S^n$ and the concatenation of these entries forms the output in $\mathbb{R}^m$. For equality constraints, each $(\mathcal{A}X)_i$ defines a hyperplane in $\mathcal{X}$ and correspondingly an inequality defines a half-space.

We will also use the transpose $\mathcal{A}^T : \mathbb{R}^n \rightarrow S_+^n$ of $\mathcal{A}$. This is defined as $\mathcal{A}^T$ as $\mathcal{A}^T x = \sum_{i=1}^m x_i A_i$, which satisfies the necessary property of a transpose:

$$\begin{aligned} \langle \mathcal{A}X, x \rangle &= \sum_{i=1}^m \text{trace}(A_i^T X) x_i \\ &= \text{trace} \left(\sum_{i=1}^m x_i A_i^T X \right) \\ &= \langle X, \sum_{i=1}^m x_i A_i \rangle \\ &= \langle X, \mathcal{A}^T x \rangle. \end{aligned}$$

Using the combination of, choice of metrics, objection function, and constraints we formulate the generic form of optimisation problem we encounter as:

$$\begin{aligned} \min_{X \in \mathcal{X}} f(X) \\ \text{s.t. } \mathcal{A}X &= b, \\ \mathcal{C}X &\leq d \end{aligned} \tag{2.7}$$

where $\mathcal{A} : \mathcal{X} \rightarrow \mathbb{R}^m$, $b \in \mathbb{R}^m$ describe m equality constraints and similarly $\mathcal{C} : \mathcal{X} \rightarrow \mathbb{R}^k$, $d \in \mathbb{R}^k$ describe k inequality constraints. To incorporate knowledge of the dataset into the optimisation problem, at least one of the objective or constraints will be defined with values x_i from the dataset $\mathcal{D}$. The role that the dataset plays in each optimisation problem will be made clear in the particular context.

Convex Analysis

Finding solutions to optimisation problems like (2.7) can be very challenging and even infeasible in some circumstances. Crafting problems that can be efficiently solved, that give good empirical performance and use available data appropriately is the real crux of Machine Learning.

A common way to ensure efficiency in solving optimisation problems is to formulate them so that they are smooth and convex. Smoothness allows us to use gradient-based solvers, while convexity tells us that problems such as (2.7) have a unique solution. We first define convexity and smoothness and then discuss their relevance in learning.

Definition 2.2.1 [94] *A set $C \subseteq \mathcal{X}$ is convex if*

$$tX + (1 - t)Y \in C, \forall X, Y \in C, t \in [0, 1].$$

The intuitive interpretation of convexity is that if a straight line between any two points in a set is constructed, then that line is entirely contained in the set. Positive definite matrices S_{++}^n are a convex set and thus this type of analysis is applicable for optimisation problems involving them. On the other hand, $S_+(p, n)$ is non-convex since the sum of two fixed-rank PSD matrices is not necessarily the same rank. Convexity is a valuable property in optimisation problems. Convexity can also be defined for functions:

Definition 2.2.2 [94] *A function $f : C \rightarrow \mathbb{R}$, for convex $C \subseteq \mathcal{X}$, is convex iff*

$$f(tX + (1 - t)Y) \leq tf(X) + (1 - t)f(Y), \forall X, Y \in U, t \in [0, 1].$$

One interpretation of this definition is that given a straight line between two points of a function, this line always lies above the function, i.e. straight lines are over-estimators. We can use this property to guarantee minimisers along descent directions. When the inequality is reversed in the above definition one obtains the definition for concavity of a function.

We now provide a definition of Lipschitz continuity:

Definition 2.2.3 [94] *A function $f : U \rightarrow V$ is Lipschitz continuous with $\beta \in \mathbb{R}_+$ if:*

$$\|f(x) - f(y)\| \leq \beta \|x - y\|, \forall x, y \in U,$$

where $U \subseteq \mathbb{R}^n, V \subseteq \mathbb{R}^m$ and U is a convex set.

For our purposes Lipschitz continuity is useful with respect to the gradient of a function. If $\nabla f(x)$ is Lipschitz continuous we say that f is β -smooth. β -smoothness allows algorithms to take gradient steps with the confidence that the size of the step will be bounded.

A function that is frequently used to analyse constrained optimisation functions is its Lagrangian function:

Definition 2.2.4 [95, §5.1.1] For problem (2.7) the Lagrangian $\mathcal{L} : \mathcal{X} \times \mathbb{R}^m \times \mathbb{R}_+^k$ is defined as:

$$\mathcal{L}(X, \lambda, \nu) = f(X) + \lambda^T(\mathcal{A}X - b) + \nu^T(\mathcal{C}X - d),$$

where $\lambda \in \mathbb{R}^m$ and $\nu \in \mathbb{R}_+^k$ are referred to as the dual variables and X is the primal variable. The problem

$$\max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}_+^k} \min_{X \in \mathcal{X}} \mathcal{L}(X, \lambda, \nu) \quad (2.8)$$

yields the same value for the primal variable as problem (2.7). In convex analysis the concept of duality [95, §5] is frequently used to transform constrained optimisation problems into unconstrained problems that are easier to solve. The dual problem to the primal problem is defined as:

$$\phi(\lambda, \nu) = \min_{X \in \mathcal{X}} \mathcal{L}(X, \lambda, \nu).$$

Then the solution to the dual problem can be found by:

$$\max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}_+^k} \phi(\lambda, \nu), \quad (2.9)$$

which is referred to as the dual solution. The primal solution and the dual solution are related by weak duality, which says that:

$$d^* \leq p^*,$$

where d^* and p^* are the optimal values of Problems (2.9) and (2.7) respectively. The dual problem (if it exists) is always convex and so a lower-bound on the primal optimisation problem may be easier. If the primal problem is also convex and Slater's condition (that the feasible set is non-empty) is satisfied, then strong duality holds, i.e. $d^* = p^*$ [95, §5.2.3].

Another tool used to find the optimal points (X^*, λ^*, ν^*) of problem (2.7) are the Karush-Kuhn-Tucker (KKT) optimality conditions:

$$\nabla_X f(X^*) + \mathcal{A}^T \lambda^* + \mathcal{C}^T \nu^* = 0 \quad (2.10a)$$

$$\mathcal{A}X^* = b \quad (2.10b)$$

$$\mathcal{C}X^* \leq d \quad (2.10c)$$

$$\nu_j^* (\mathcal{A}X^* - b)_j = 0, j = 1, \dots, k \quad (2.10d)$$

$$\nu_j^* \geq 0, j = 1, \dots, k \quad (2.10e)$$

These conditions can be used to check for optimality, and if they can be solved simultaneously then the solution (X^*, λ^*, ν^*) is optimal.

Problems of the form of (2.7) infrequently have closed-form or analytic solutions such as those offered by KKT conditions. Therefore iterative optimisation algorithms are needed to compute solutions to these problems. We now introduce several types of algorithm for solving these problems. For simplicity we start with algorithms that solve unconstrained problems, then build on these for the constrained case.

First and Second-Order Gradient Algorithms

For an unconstrained convex optimisation problem, optimality occurs when $\nabla f(X) = 0$, i.e. a stationary point is achieved (which is equivalent to the KKT conditions). A successful approach for solving this type of problem is to use gradients to iteratively compute the solution. Gradient methods rely on the observation that for a

function $f(X)$ at a position X the gradient $\nabla f(X)$ specifies the direction of greatest change in the value of f . By updating X according to the gradient we can successively minimise $f(X)$. This procedure is referred to as Gradient Descent [95, §9.3] and is a simple and often effective algorithm for unconstrained minimisation. More concretely, for an unconstrained problem (consider Problem (2.7) without its constraints) the Gradient Descent algorithm is described in Algorithm 1.

Algorithm 1: Gradient Descent

Input: X_0, ϵ
Start: $X_t \leftarrow X_0$
While $\|\nabla_X f(X_t)\| \geq \epsilon$
 $\Delta X = -\nabla_X f(X_t)$
 $\theta \leftarrow \text{linesearch}(X_t, \Delta X, f)$
 $X_t \leftarrow X_t + \theta \Delta X$
End While

The line search step in Algorithm 1 determines how far along the descent direction the next step will be. This is a one-dimensional optimisation problem and in some circumstances may be solved exactly for the minimum point along this line. However, in many cases this is not an option. This procedure is then approximated with a backtracking search, or in some cases a constant step-size is selected. A description of backtracking line search, a popular line search algorithm, is given in Algorithm 2.

A related algorithm to Gradient Descent is Newton's Method [95, §9.5], which is a second-order gradient method, meaning that it uses second-order derivative information. The iterates used for Newton's Method are $X_{t+1} = X_t - H_t^{-1} \nabla f(X_t)$ where $H_t = \nabla^2 f(X_t)$ is the Hessian operator of f at x_t . Newton's Method converges quadratically, whereas Gradient Descent converges linearly, so typically fewer iterations will be required.

Algorithm 2: Backtracking Line Search

Input: $X_t, \Delta X, f$
 Set: $\alpha \in (0, 0.5), \beta \in (0, 1)$
 Start: $\theta \leftarrow 1$
 While $f(X_t + \theta \Delta X) > f(X_t) + \alpha \theta \langle \nabla f(X_t), \Delta X \rangle$
 $\theta \leftarrow \beta \theta$
 Return θ

Algorithm 3: Newton's Method

Input: X_0, ϵ
 Start: $X_t \leftarrow X_0$,
 While $\nabla_X f(X_t)^T (\nabla_X^2 f(X_t))^{-1} \nabla_X f(X_t) \geq \epsilon$
 $\Delta X = -(\nabla_X^2 f(X_t))^{-1} \nabla_X f(X_t)$
 $\theta \leftarrow \text{linesearch}(X_t, \Delta X, f)$
 $x_t \leftarrow X_t + \theta \Delta X$
 End While

The main challenge in using second-order techniques is that computing the Hessian matrix can be computationally expensive. So while Newton's Method will require fewer iterations, the time per iteration may total more than if a first-order method were used. However, there are methods to try and circumvent this. Quasi-Newton Methods, which approximate the Hessian using first-order gradient updates, are a very successful family of optimisation algorithms [96]. To further reduce the cost of calculating second-order information, algorithms have been developed to reduce the expression of the approximate Hessian to a list of vectors, rather than a matrix. L-BFGS [97] is an example of this type of algorithm. Typically L-BFGS retains the quadratic convergence of a second-order algorithm, but at a much lower computational cost.

Constrained Gradient Methods

So far we have only considered unconstrained optimisation algorithms. However, problem (2.7) has constraints, so how are these addressed in an optimisation algorithm? A first-order method for constrained optimisation is Projected Gradient Descent [98] an extension of Gradient Descent. The essential difference in Projected Gradient Descent is that the update $X_t \rightarrow X_t + \theta \Delta X$ in Algorithm 1 is replaced with $X_t \rightarrow \mathcal{P}_C(X_t + \theta \Delta X)$ where $\mathcal{P}_C : \mathcal{X} \rightarrow C$ is a projection onto the constraint set $C \subseteq \mathcal{X}$. This projected gradient step also replaces the gradient step during the line search algorithm. Since Projected Gradient Descent is a simple extension of Gradient Descent it can enjoy the same fast updates on sets where the projection operation is relatively simple and low-cost.

Primal-Dual Interior-Point Methods (IPMs) are second-order methods for optimising constrained optimisation problems [95]. These algorithms are generally preferred over Projected Gradient Descent when the constraints become more complex or numerous. Primal-Dual Interior-Point methods differentiate a modified version in the KKT conditions with respect to the primal and dual variables to obtain a second-order gradient step. In this way, these methods apply Newton's Method to a modification of the equality constraints of the KKT conditions. Feasibility of the inequality constraint is maintained through step size selection. The following system defines the search direction on the primal and dual variables:

$$\begin{pmatrix} \nabla^2 f(X) & \mathcal{A}^T & \mathcal{C}^T \\ \mathcal{A} & 0 & 0 \\ -\text{diag}(\nu)\mathcal{C} & 0 & -\text{diag}(\mathcal{C}X - d) \end{pmatrix} \begin{pmatrix} \Delta X \\ \Delta \lambda \\ \Delta \nu \end{pmatrix} = \begin{pmatrix} \nabla f(X) + \mathcal{A}^T \lambda + \mathcal{C}^T \nu \\ \mathcal{A}X - b \\ -\text{diag}(\nu)(\mathcal{C}X - d) - \mathbb{1}/t \end{pmatrix}$$

During optimisation the value of t is progressively reduced so that the true KKT conditions can be adhered to as closely as desired.

First and second-order optimisation algorithms are very powerful tools that are applicable to a wide variety of problems. However, when the structure of a problem takes on a certain form, more specialised algorithms can be applied with in-

creased efficiency, among other benefits. We now examine such a type of algorithm for convex projection problems.

Projection Algorithms

A commonly-encountered problem in Machine Learning is to find the closest projection of a point onto a convex set [99]. Projections are defined with respect to a distance or a divergence, as they give a quantitative notion of closeness between objects. Projections are useful when we already have a parameter or statistic X_0 that does not have some desirable property and we want the element most similar to it that possesses this property. The projection problem can be formalised as:

$$\begin{aligned} \min_{X \in \mathcal{X}} D(X, X_0) \\ \text{s.t. } X \in \Phi, \end{aligned}$$

where $\Phi \subseteq \mathcal{X}$ is a constraint set, $X_0 \in \mathcal{X}$ is an initial point, and $D : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$ is a distance or divergence.

A straightforward case of this problem is:

$$\begin{aligned} \min_{X \in \mathcal{X}} D(X, X_0) \\ \text{s.t. } \mathcal{A}X = b, \end{aligned}$$

where $\mathcal{A} : \mathcal{X} \rightarrow \mathbb{R}^m$ is a linear operator. This problem may be solved using Alternating Projections [99], which decomposes the projection problem into a sequence of projections with respect to each hyperplane constraint $(\mathcal{A}X)_i = b_i$. These projections are normally closed-form, making the algorithm very efficient.

A more complex problem is:

$$\begin{aligned} \min_{X \in \mathcal{X}} D(X, X_0) \\ \text{s.t. } \mathcal{C}X \leq d, \end{aligned} \tag{2.11}$$

where the increased complexity is due to the inequality in the constraints. Alternating Projections will not necessarily compute the optimal solution in this case. Instead we use Cyclic Projections, an algorithm designed for this type of projection problem. Cyclic Projections coordinates the projection updates between the constraints, which is necessary for problem (2.11).

We will prove the convergence of Cyclic Projections for one of our algorithms in the next chapter, so we now provide the necessary theorems to ensure its convergence. For problem (2.11) to converge using Cyclic Projections its objective must be a Bregman function and zone consistent (which are defined shortly). In following Definitions and Theorem, the objective function $D(X, X_0)$ is relabelled to $f(X)$, where f is a Bregman function. Also the sequence of iterates with respect to the constraints need to be *almost cyclic*.

Definition 2.2.5 ([100] pg. 31) *A function $f : \mathcal{X} \rightarrow \mathbb{R}$, $\mathcal{X} \subseteq \mathbb{R}^{n \times m}$ is a Bregman function if there exists a non-empty, open, convex set C (referred to as the zone of f), such that $\bar{C} \subseteq \mathcal{X}$ (the closure of C) and the following hold:*

- (i) *$f(X)$ has continuous first partial derivatives at every $X \in C$.*
- (ii) *$f(X)$ is strictly convex on $\bar{C}$.*
- (iii) *$f(X)$ is continuous on $\bar{C}$.*
- (iv) *For every $\alpha \in \mathbb{R}$, the partial level sets $\{X \in C : D_f(X, Y) \leq \alpha\}$ and $\{X \in \bar{C} : D_f(X, Y) \leq \alpha\}$ are bounded for every $Y \in C$ and $X \in \bar{C}$ respectively.*
- (v) *If $Y_k \in C$, $\forall k \in \mathbb{N}$ and $\lim_{k \rightarrow \infty} Y_k = Y^*$, then $\lim_{k \rightarrow \infty} D_f(Y^*, Y_k) = 0$.*
- (vi) *If $Y_k \in C$, $X_k \in \bar{C}$, $\forall k \in \mathbb{N}$ and if $\lim_{k \rightarrow \infty} D_f(X_k, Y_k) = 0$ and $\lim_{k \rightarrow \infty} Y_k = Y^*$ and $\{X_k\}_{k \in \mathbb{N}}$ is bounded, then $\lim_{k \rightarrow \infty} X_k = Y^*$.*

The family of Bregman functions on $C \subseteq \mathbb{R}^n$ is denoted as $\mathcal{B}(C)$.

Definition 2.2.6 ([100] pg 35) (i) A function $f \in \mathcal{B}(C)$ with zone C is said to have the zone consistency property with respect to the hyperplane H if, for every $Y \in C$ we have $P_H(Y) \in C$. That is, the generalised projection $P_H(Y)$ of any point $Y \in C$ onto the hyperplane H remains in C .

(ii) $f \in \mathcal{B}(C)$ with zone C is strongly zone consistent with respect to the hyperplane H if it is zone consistent with respect to H and with respect to every other hyperplane H' that is parallel to H and lies between Y and H .

Definition 2.2.7 ([101]) Let $[m] = \{1, \dots, m\}$. A sequence of indices $\{i_k\}_{k=0}^\infty$ is called almost cyclic on $[m]$ if $i_k \in [m]$, $k \geq 0$ and there exists an integer J such that:

$$[m] \subseteq \{i_{k+1}, i_{k+2}, \dots, i_{k+J}\}, \forall k \geq 0.$$

The convergence of Cyclic Projections is guaranteed by the conditions in the following theorem:

Theorem 2.2.8 ([100] pg 137) Assume the following:

- (i) $f \in \mathcal{B}(C)$.
- (ii) f is strongly zone consistent with respect to each H_i .
- (iii) $\{i(v)\}_{v=0}^\infty$ is almost cyclic with constant of almost cyclicity C .
- (iv) $Z_0 = \{X \in C : \exists z \in \mathbb{R}_+^n \text{ such that } \nabla f(X) = -A^T z\} \neq \emptyset$.

Then, any sequence $\{X_k\}_{k \in \mathbb{N}}$ produced by Algorithm 4 converges to the point X^* , which is the solution of (2.11).

The algorithm for Cyclic Projections defined in Algorithm 4 allows projection problems to be decomposed into updates with respect to each constraint $\text{trace}(A_i X) \leq b_i$. Typically the updates have closed-form expressions, so the algorithm executes quickly.

Algorithm 4: Cyclic Projections

Input: $C_0 \in \mathcal{X}$, $f : \mathcal{X} \rightarrow \mathbb{R}_+$, $\{A_i \in S_+^n, b_i \in \mathbb{R}\}_{i=1}^N$, ϵ

Set $k \leftarrow 0$, $\lambda \leftarrow \mathbf{0}_N$

Do

For $i = 1:N$

Solve for $\hat{\lambda}_i$: $\nabla f(X^{(k+1)}) = \nabla f(X^{(k)}) + \hat{\lambda}_i A_i$, $\langle A_i, X^{(k+1)} \rangle = b_i$

$\alpha \leftarrow \min(\lambda_i, \hat{\lambda}_i)$

Update $X^{(k+1)}$: $\nabla f(X^{(k+1)}) = \nabla f(X^{(k)}) + \alpha A_i$

$\lambda_i \leftarrow \lambda_i - \alpha$

$k \leftarrow k + 1$

end

While $\|\lambda_k - \lambda_{k+N}\| \leq \epsilon$

end

Matrix Manifold Optimisation Algorithms

In our discussion of metrics in the previous section, the concepts of matrix manifolds and their associated distances were introduced. We now extend our presentation of optimisation algorithms to optimisation on matrix manifolds. These matrix manifold optimisation algorithms are essentially generalisations of the gradient-based optimisation algorithms that we have already encountered. The primary difference between Euclidean optimisation and manifold optimisation are that gradients are no longer elements of the manifold. So this requires a new way of combining a gradient with the current iterate. We will briefly expand on these points for context, however our treatment of these concepts is fairly qualitative. Our presentation of Riemannian optimisation algorithms in the following chapters will be similarly high-level, as Manopt, a manifold optimisation toolbox [102], provides most of the underlying mechanics. In this way, we avoid unnecessary detail in our presentation of these algorithms. A more in-depth treatment is available in reference materials on matrix optimisation such as [103, 104]. For our purposes, it is sufficient to think of manifold or geometric concepts as analogous to

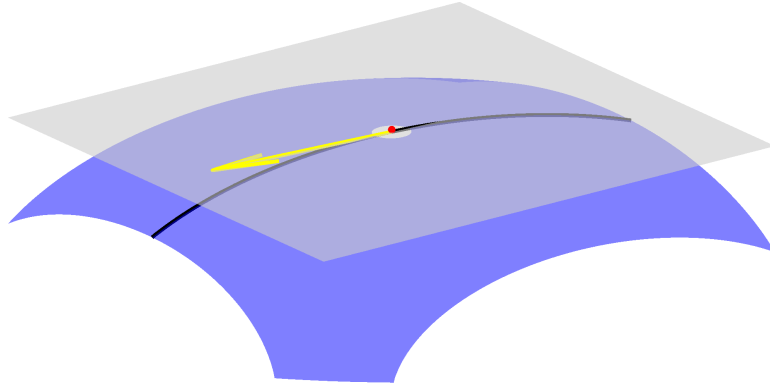


Figure 2.1: Tangent vector η_X (yellow) to curve $\gamma(t)$ (black) on tangent space $\mathcal{T}_X \mathcal{M}$ (grey) at $X \in \mathcal{M}$ (red) on manifold $\mathcal{M}$ (blue).

their Euclidean counterparts.

In a curved manifold, such as the blue surface in Figure 2.1, the tangent space $\mathcal{T}_X \mathcal{M}$ at a point $X \in \mathcal{M}$ is a vector space defined as the vectors tangent to curves passing through X . Tangent spaces, and mappings between them, provide the basis for differential operations on manifolds. In order to perform optimisation on a manifold it must be equipped with a Riemannian metric $g_X : \mathcal{T}_X \mathcal{M} \times \mathcal{T}_X \mathcal{M} \rightarrow \mathbb{R}_+$ as without a metric there is no canonical way to define distances, angles and gradients on a manifold. With a metric, the manifold is referred to as a Riemannian manifold. A Riemannian metric is essentially a generalisation of a Euclidean inner product to the tangent space of a manifold. Tangent spaces vary at different locations on a manifold, so the Riemannian metric also varies between these spaces. For a given manifold there are many possible choices of a Riemannian metric, as we have seen for S_{++}^n . Different choices of Riemannian metric result in different geometries associated to distance functions, such as the log-Euclidean or Affine-Invariant distances. The Riemannian gradient $\text{grad} f$ of a real-valued function $f : \mathcal{M} \rightarrow \mathbb{R}$ is a map from the manifold $\mathcal{M}$ to tangent space $\mathcal{T}_X \mathcal{M}$ analogous to a Euclidean gradient, but also accounts for the geometry induced by the metric.

While a gradient is necessary, it cannot be used alone to compute optimisation iterates on a Riemannian manifold. To produce iterations of an optimisation algorithm, a retraction function is also required. A retraction function R maps a point $X \in \mathcal{M}$ to another point Y in $\mathcal{M}$ according to the direction specified by a tangent vector $\eta_X \in \mathcal{T}_X \mathcal{M}$. A retraction function is well-behaved in the sense that a zero vector $0_X \in \mathcal{T}_X \mathcal{M}$ maps X to itself and its differential at the zero vector maps tangent vectors to themselves, i.e. the identity mapping. Now that both the Riemannian gradient and retractions have been introduced, the algorithms for Riemannian Gradient Descent can be defined.

Riemannian Gradient Descent [103], described in Algorithm 5, is fairly simple. For each iteration a gradient is computed and a line search with respect to the retraction is performed to find the next update until the gradient is sufficiently small. A backtracking line search algorithm is also included in Algorithm 6. Comparing

Algorithm 5: Riemannian Gradient Descent

Input: X_0, ϵ
 Start: $X_t \leftarrow X_0$
 While $\|\text{grad}f(X_t)\|_g \geq \epsilon$
 $\eta_t = -\text{grad}f(X_t)$
 $\theta \leftarrow \text{linesearch}(X_t, \eta_t, f, R)$
 $X_t \leftarrow R(X_t, \theta \Delta X)$
 End While

Algorithms 1 and 5, we see that they are very similar. In fact Riemannian Gradient Descent may be applied to a Euclidean space, which will result in steps equivalent to Gradient Descent.

Second-order Riemannian gradient optimisation algorithms may also be defined in a similar way to second-order Euclidean optimisation algorithms. However to define a second-order Riemannian optimisation method the Riemannian Hessian needs to be defined. The Riemannian Hessian at $X \in \mathcal{M}$ associated with f is a lin-

Algorithm 6: Backtracking Line Search with Retraction

Input: X_t, η, f, R
 Set: $\alpha \in (0, 0.5), \beta \in (0, 1)$
 Start: $\theta \leftarrow 1$
 While $f(R(X_t, \theta\eta)) > f(X_t) + \alpha\theta g_{X_t}(\text{grad}f(X_t), \eta)$
 $\theta \leftarrow \beta\theta$
 Return θ

ear function $\text{Hess}f_X : T_X\mathcal{M} \rightarrow T_X\mathcal{M}$ that measures the curvature of the function f . However the curvature associated with the Riemannian metric is also incorporated into the computation of the Riemannian Hessian. The search direction for Newton's Method in Riemannian space is then:

$$\eta_t = (\text{Hess}f(X_t))^{-1} \text{grad}f(X_t),$$

which is analogous to the search direction of Newton's Method (Algorithm 3). This search direction is then subject to a line search using the retraction. The solution to this line search is then the update for X_{t+1} . Riemannian Quasi-Newton Methods have also been developed and similarly attempt to lower the cost of computing the search direction by using different approximations [106]. Riemannian trust-regions [107] is another a prevalent algorithm for solving Riemannian optimisation problems. At the t -th iteration Riemannian trust regions solves the problem:

$$\begin{aligned}
 \min_{\eta \in T_{X_t}\mathcal{M}} & f(X_t) + g_{X_t}(\text{grad}f(X_t), \eta) + \frac{1}{2}g_{X_t}(\text{Hess}f(X_k)\eta, \eta) \\
 \text{s.t. } & g_{X_k}(\eta, \eta) \leq \Delta_t^2,
 \end{aligned}$$

where Δ_t is the trust region radius at iteration t . The solution η_k is then applied to X_k using a retraction R to find the next iterate.

Having introduced the metrics and optimisation tools relevant to our work, we now provide a brief introduction to Optimal Transport in the context of Machine Learning and introduce the Bures-Wasserstein distance more formally.

2.3 Optimal Transport

Optimal Transport at its core is a way of defining a distance between probability distributions with respect to an underlying cost function. There are two types of Optimal Transport distance that we discuss in this thesis, where the Bure-Wasserstein distance is one of them.

The first form of Optimal Transport introduced here is between discrete probability distributions $a \in \Delta^m, b \in \Delta^n$, potentially differing in the number of indexes. To formulate a distance between a and b a cost function must be specified between the respective entries of the histograms. This takes the form of a cost matrix $C \in \mathbb{R}^{m \times n}$ where each C_{ij} quantifies the cost of moving probability mass from a_i to b_j . To actually 'move' probability mass from a to b a plan or scheme to transport this mass is required. Such a plan that moves all of the probability mass from a to b is referred to as a coupling $P \in \mathbb{R}^{m \times n}$, which is itself a discrete probability distribution between the indexes of a and b . Taking the product of each C_{ij} and P_{ij} and summing over all of the indices gives an aggregate cost of transporting the mass of a to b .

If C is appropriately defined and the minimum cost transportation plan with respect to the cost between a and b is specified, then this describes an Optimal Transport distance between discrete probability distributions [27]. This can be formalised as the constrained optimisation problem:

$$OT_C(a, b) = \inf_{P \in \Pi(a, b)} \langle C, P \rangle, \quad (2.12)$$

where $\Pi(a, b) = \{P \in \mathbb{R}^m \times \mathbb{R}^n : P\mathbf{1} = a, P^T\mathbf{1} = b, P_{ij} \geq 0\}$ is the set of m by n coupling matrices that assigns mass from the i -th index of a to the the j -th index of b .

This type of formulation is well-suited to many of the kinds of data encountered in Machine Learning as each data point $x_i \in \mathbb{R}^n$ in a dataset can be thought of as an

index with a probability weight assigned to it. We can then measure the distance between the dataset composed of the x_i samples and another dataset composed of y_j samples, with respect to a cost function. like. One particular example that we will encounter in the literature review is the 2-Wasserstein distance, where each entry of the cost matrix is specified by $C_{ij} = \|x_i - y_j\|_2^2$. This specific distance will be written as $W_2(a, b)$ for clarity when necessary.

It is also fortunate that problem (2.12) is convex in P , so it can be solved using convex optimisation algorithms such as Interior-Point Methods. However, these can be slow for modestly sized Optimal Transport problems, and so more efficient algorithms have been developed specifically to solve this type of problem [67]. An alternative to solving this problem is to regularise it using an entropic term:

$$OT_C^\epsilon(a, b) = \inf_{P \in \Pi(a, b)} \langle C, P \rangle - \epsilon \sum_{ij} P_{ij} \log(P_{ij}) \quad (2.13)$$

The addition of the entropic regulariser acts as a barrier for negative entries in P meaning the non-negative constraint $P_{ij} \geq 0$ in $\Pi(a, b)$ will be implicitly enforced, reducing the complexity of the problem greatly. Additionally, the objective of problem (2.13) is equivalent to a KL-divergence, so it is a projection problem. The solution to problem (2.13) is referred to as the Sinkhorn distance [68]. This choice of name reflects the matrix scaling algorithm introduced by Sinkhorn [108], that is used to solve problems of this form. The Sinkhorn algorithm performs Alternating Projections with respect to the constraints $P\mathbf{1} = a$ and $P^T\mathbf{1} = b$. The rate of convergence increases with larger values of the regularisation parameter ϵ , at the cost of obtaining a less accurate approximation. The iterations for the Sinkhorn algorithm take the form:

$$u_{t+1} = \frac{a}{K v_t}, \quad v_{t+1} = \frac{b}{K^T u_{t+1}} \quad (2.14)$$

where $K = \exp(-C/\epsilon)$ and $\exp$ is computed entry-wise. Once sufficient optimality is reached in u and v , the solution to problem (2.13) is computed as: $P^* = u^* K v^*$. Similar to the above case for the 2-Wasserstein distance, the entropically regu-

larised version of the 2-Wasserstein distance will be written as $W_2^\epsilon(a, b)$ when necessary.

Since the introduction of the Sinkhorn distance, variations on the Sinkhorn algorithm have been introduced to increase its rate of convergence, such as the Greenkhorn algorithm [109, 110].

A recent algorithm for computing solutions to problem (2.13) that has shown

Algorithm 7: Accelerated Alternating Minimisation

Input: Dual Function ϕ
Set: $A_0 \leftarrow 0, a_0 \leftarrow 0$
 $\eta_0 \leftarrow 0, \zeta_0 \leftarrow 0, \lambda_0 \leftarrow 0$
While $\|\nabla\phi(\lambda)\| \geq \epsilon$
 $\beta_k \leftarrow \arg \min_{\beta \in [0,1]} \phi(\eta_k + \beta(\zeta - \eta))$
 $\lambda_k \leftarrow \beta_k \zeta_k + (1 - \beta_k) \eta_k$
 $w \leftarrow \arg \max\{\|\nabla_u \phi(\lambda_k)\|, \|\nabla_v \phi(\lambda_k)\|\}$
 $\eta_{k+1} \leftarrow \arg \min_{\eta \in \lambda_k(w)} \phi(\eta)$
 Solve $\phi(\lambda_k) - \frac{(a_{k+1})^2}{2(A_k + a_{k+1})\|\nabla\phi(\lambda_k)\|_2^2} = \phi(\eta_{k+1})$ for a_{k+1}
 $A_{k+1} = A_k + a_{k+1}$
 $\zeta_{k+1} \leftarrow \zeta_k - a_{k+1} \nabla\phi(\lambda_k)$
 $x_{k+1} \leftarrow \frac{a_{k+1}P(\lambda_k) + A_k x_k}{A_{k+1}}$
Return x_{k+1}, η_{k+1}

promise is Accelerated Alternating Minimisation [111]. This algorithm incorporates Nesterov acceleration and the alternating minimisation formulation. It achieves faster convergence than other contemporary algorithms. This algorithm (shown in Algorithm 7) uses the dual function ϕ to problem (2.13), which has dual variables u, v , one for each marginal constraint. The dualisation removes the constraints from the problem, so unconstrained alternating minimisation with respect to u and v may be performed. The advantage of this is that minimisers with re-

spect to each dual variable may be computed with a closed-form solution, as is the case for the Sinkhorn iterations above. The dual variable that is minimised at each iteration is selected based on which one has the larger gradient with respect to ϕ , as this is equivalent to magnitude of the violation of the coupling constraints. The closed-form minimiser is then computed with respect to this variable, holding the other variable constant. This set is expressed as $\lambda(w)$ for the selected index, and followed by updating the variables using momentum terms.

We can see that there has been a substantial amount of work that has gone into how to compute the Optimal Transport distance between discrete probability distributions which is primarily where its use lies. However, Optimal Transport distances can also be defined between continuous probability distributions, which has a substantial amount of theoretical contributions and exploration. The particular case of interest here is the Bures-Wasserstein distance.

Bures-Wasserstein Distance and Symmetric Positive Definite Manifolds

In general, computing Optimal Transport distances between continuous probability distributions is intractable. However, placing restrictions on the family of distributions being used may simplify the problem greatly. A prime example of this is the family of non-degenerate Gaussian distributions.

Given two non-degenerate Gaussian distributions $\mathcal{N}(\mu_0, \Sigma_0), \mathcal{N}(\mu_1, \Sigma_1) \in \mathcal{P}(\mathbb{R}^d)$ where $\mu_i \in \mathbb{R}^d$ and $\Sigma_i \in S_{++}^d$ are the mean and covariance of each Gaussian the 2-Wasserstein distance between them is [112]:

$$W_2(\mathcal{N}(\mu_0, \Sigma_0), \mathcal{N}(\mu_1, \Sigma_1))^2 = \|\mu_0 - \mu_1\|_2^2 + \text{trace}(\Sigma_0 + \Sigma_1 - 2(\Sigma_0^{1/2}\Sigma_1\Sigma_0^{1/2})^{1/2}),$$

which is a variation of the 2-Wasserstein distance $W_2(a, b)$ that we have already introduced. In the above expression the mean and covariance terms independently contribute to the distance, so we can consider this as a distance on the product space $\mathbb{R}^d \times S_{++}^d$. In fact, both of these terms independently define distances on their

respective space. If we consider the components of the product space independently we can see that the distance on $\mathbb{R}^d$ is just the Euclidean distance. Similarly, the distance on S_+^d is the Bures-Wasserstein distance, which is expressed as:

$$\mathcal{B}^2(\Sigma_0, \Sigma_1) = \text{trace}(\Sigma_0 + \Sigma_1 - 2(\Sigma_0^{1/2}\Sigma_1\Sigma_0^{1/2})^{1/2}).$$

A significant amount of the research into the Bures-Wasserstein distance has been concerned with studying its barycentres [113, 114, 115]. In Euclidean space using the l_p distance, the barycentre of a finite collection of points $\{x_i\}_{i=1}^N \subseteq \mathbb{R}^d$ with weights λ_i such that $\sum_{i=1}^N \lambda_i = 1$, is:

$$\min_{x \in \mathbb{R}^d} \sum_{i=1}^N \lambda_i \|x - x_i\|_p^p,$$

which is essentially a weighted average. A typical case for this for $p = 2$ is the standard average where the problem has the closed-form solution $x^* = \frac{1}{N} \sum_{i=1}^N x_i$. This can be efficiently computed for very large instances of N and d .

Similarly, we would like to be able to compute an average of a collection of zero-mean Gaussian distributions, parameterised by the set of covariance matrices $\{\Sigma_i\}_{i=1}^N$. This can be expressed using the Bures-Wasserstein distance:

$$\min_{\Sigma \in S_+^n} \sum_{i=1}^N \lambda_i \mathcal{B}^2(\Sigma, \Sigma_i), \quad (2.15)$$

where $\lambda \in \mathbb{R}^N$ is a weighting vector such that $\sum_{i=1}^N \lambda_i = 1$. The solution to this problem is referred to as a Bures-Wasserstein barycentre. In general there is no closed-form expression for this problem like there was for a Euclidean barycentre. However, the regularity and existence of solutions to this problem has been studied thoroughly in recent years as well as different methods for computing its solution [116, 113, 115, 114].

It is also possible to equip the set S_{++}^n with a Riemannian metric such that the resulting distance is the Bures-Wasserstein distance [69, 117]. This Bures-Wasserstein

geometry has seen some application in a selection of Machine Learning tasks [55] and can be used to find barycentres [115]. Similarly, $S_+(p, n)$ can be equipped with a Riemannian geometry that induces the Bures-Wasserstein distance between elements [118, 87].

We now have all of the necessary tools to proceed with a discussion of Metric Learning algorithms relevant to the Bures-Wasserstein distance.

2.4 Literature Review

Many tasks in Machine Learning rely on the ability to judge distances between datapoints. An example of this is the k -Nearest-Neighbours classifier (k -NN). To classify an unknown datapoint using k -NN with a labelled dataset, its k closest neighbours with respect to a distance function are computed. The majority class of the collected datapoints is then assigned to the unknown datapoint. It is obvious that the distance function used in this algorithm significantly affects the choice of classification. The goal of Metric Learning is to learn a distance that is suited to the target problem based on data, rather than using a generic distance. If a metric for a particular data type or a certain property is desired, then the Metric Learning problem can be designed to accommodate it. This flexibility has resulted in the diversity of Metric Learning algorithms in the literature. For a broader exposition of different Metric Learning algorithms, see surveys [7, 8, 9].

In this section the algorithms relevant to our study of Linear Metric Learning (LML) are reviewed. First, foundational Metric Learning algorithms are introduced to illustrate the structure of LML algorithms and define some essential concepts. This is followed by a brief tour through probabilistic LML algorithms, as they provide essential prototypes for later algorithms. Symmetric Positive Definite Metric Learning algorithms are introduced as they also feature in this thesis. Finally, a review of the current Optimal Transport-based Metric Learning algorithms is provided to motivate the algorithms introduced in later chapters.

By LML algorithms we mean Metric Learning algorithms parameterised by either a linear transformation $L \in \mathbb{R}^{p \times n}$ or a semidefinite matrix $M \in S_+^n$. These parameterisations are usually interchangeable using $M = L^T L$ and the Cholesky decomposition [119] for the reverse direction.

An elementary method in LML is Principal Component Analysis (PCA) [120]. For a zero-mean dataset $\{x_i \in \mathbb{R}^n\}_{i=1}^N$, PCA computes the $p \leq n$ directions of highest variance associated with the dataset, which are referred to as the principal components. However, PCA does not incorporate label information. The principal components are expressed as a $V \in St(p, n)$. The optimisation problem that computes the p principal components is:

$$\begin{aligned} \max_L \quad & \text{trace}(VCV^T) \\ \text{s.t.} \quad & V \in St(p, d), \end{aligned} \tag{2.16}$$

where $C = \frac{1}{N-1}XX^T$ is the empirical covariance matrix of X . Frequently in data analysis, a small number of linear factors contribute to a quantity of interest or classification of a datapoint, while noise dominates the remaining directions. PCA gives a linear method to disentangle the factors from noisy measurements, improving the quality of data. In the context of Metric Learning, the learnt classifier V can be applied to the Mahalanobis distance as $M = V^T V$. PCA is often used as a pre-processing step for other Metric Learning algorithms [121]. While the above formulation makes it apparent that PCA is amenable to Riemannian optimisation, this is a more contemporary viewpoint and linear algebra algorithms were first used to compute principal components. One of the main benefits of PCA is that very efficient algorithms for computing it on large-scale data exist. However, because PCA does not incorporate label information, its performance on classification problems is reduced.

Another classical method is Linear Discriminant Analysis (LDA) [122], which incorporates label information. Instead of parameterising the learning problem us-

ing the covariance matrix of X , LDA defines two similar covariance matrices: the between-class covariance C_b and the within-class covariance C_w , defined as:

$$C_b = \frac{1}{N_C} \sum_{i=1}^{N_C} \mu_i \mu_i^T$$

$$C_w = \frac{1}{N} \sum_{i=1}^{N_C} \sum_{x_j: y_j=i} (x_j - \mu_i)(x_j - \mu_i)^T$$

where μ_i is the average of each $x_j \in X$ with label $y_j = i$, and N_C is the number of labelled classes in the data. The optimisation problem for LDA is then:

$$\max_V \frac{\text{trace}(V C_b V^T)}{\text{trace}(V C_w V^T)}$$

$$s.t. V \in St(p, n),$$

where the linear discriminant V maximises the variability between classes while also minimising the within-class variability. We can see that LDA incorporates label information. Due to its analogous formulation, LDA has similar efficiency to PCA and can also be solved for large datasets.

Both PCA and LDA rely on covariance structures to define their optimisation problems. This is sensible in the context of dimensionality reduction. However, for Metric Learning we are primarily interested in targeting a distance function for k -NN classification. The linear functions learnt by PCA and LDA only remove aspects of datapoints, since they are elements of the Stiefel manifold $St(p, n)$, offering a limited set of metrics available to the Generalised Mahalanobis distance.

As an aside, it worth noting the relationship between PCA and LDA to other non-linear dimensionality reduction techniques, such as Locally Linear Embedding [123] and Isomap [124]. These methods optimise a variable $Z \in St(p, m)$ of m points embedded into $\mathbb{R}^p$, whereas PCA and LDA optimise the Linear transform V . However, with all of these algorithms they treat their optimisation variable as an element of the Stiefel manifold where the resulting optimisation problem is an

eigenvalue type problem. PCA and LDA have the advantage in terms of scalability as their optimisation variables only scale with the dimensionality of the data, not the number of datapoints, but LLE and Isomap can capture nonlinear structure in the data. We remain in the paradigm of estimating linear functions and Generalised Mahalanobis distances on the data for their scalability.

Mahalanobis Metric Learning for Clustering (MMC), introduced by Xing *et al.* [10], provides an extension for learning distances more suited to classification and clustering than PCA or LDA. In MMC, the SPD matrix $M \in S_+^n$ in the Generalised Mahalanobis distance (expression (2.1)) is trained so that the distances $\|x_i - x_j\|_M$ reflect the class structure of the data. Because of this, the set of metrics that MMC can learn is a superset of those available to PCA and LDA. To describe the class structure relative to the distance function, Xing *et al.* also introduced similarity and dissimilarity relations on datasets. For dataset $\{x_i \in \mathbb{R}^n\}_{i=1}^N$ and corresponding labels $\{y_i \in [m]\}_{i=1}^N$, similarity and dissimilarity relations between indices $i, j \in [N]$ are frequently expressed as:

$$\begin{aligned}\mathcal{S} &= \{(i, j) : y_i = y_j\} \\ \mathcal{D} &= \{(i, j) : y_i \neq y_j\},\end{aligned}$$

where $\mathcal{S}$ denote the similarities and $\mathcal{D}$ the dissimilarities. The essential definitions of $\mathcal{S}$ and $\mathcal{D}$ are presented here, but many methods exist to select tractable subsets of $\mathcal{S}$ and $\mathcal{D}$ on datasets. These sets are used generically throughout this thesis to highlight the commonality of similarity and dissimilarity relations in Metric Learning. With the combination of the Generalised Mahalanobis distance and similarity and dissimilarity, MMC is formulated as:

$$\begin{aligned}\max_{M \in S_+^n} \quad & \sum_{(i,j) \in \mathcal{D}} \|x_i - x_j\|_M^2 \\ \text{s.t.} \quad & \sum_{(i,j) \in \mathcal{S}} \|x_i - x_j\|_M \leq 1.\end{aligned}$$

This problem is convex in $M \in S_{++}^n$ since the objective and constraint are convex functions of M . This problem is solved by Projected Gradient Descent, with re-

spect to the similarity constraints and the PSD constraint.

It is interesting to note the similarities of the objective functions of MMC and PCA. We can express the objective of MMC as:

$$\begin{aligned}
& \sum_{(i,j) \in \mathcal{D}} \|x_i - x_j\|_M^2 \\
&= \sum_{(i,j) \in \mathcal{D}} (x_i - x_j)^T M (x_i - x_j) \\
&= \sum_{(i,j) \in \mathcal{D}} \text{trace}((x_i - x_j)(x_i - x_j)^T M) \\
&= \text{trace} \left(\left[\sum_{(i,j) \in \mathcal{D}} (x_i - x_j)(x_i - x_j)^T \right] M \right),
\end{aligned}$$

It is apparent that this objective and the objectives for PCA and LDA are of the same form, especially if we consider substituting $M = L^T L$. MMC extends the formulations of PCA and LDA, computing the SPD matrix in the objective with respect to similarity and dissimilarity relations instead of a covariance structure.

Relative similarities are an important extension to similarity and dissimilarity relations, and were introduced by Weinberger *et al.* in their algorithm Large Margin Nearest-Neighbours (LMNN) [14]. Relative similarities $\mathcal{R}$ are triples $(i, j, k) \in [N] \times [N] \times [N]$, rather than the pairs used for similarity and dissimilarity. A relative similarity $(i, j, k) \in \mathcal{R}$ indicates that the datapoints x_i and x_j are more similar, than x_i and x_k . The optimisation problem for LMNN is:

$$\begin{aligned}
& \min_{M \in S_+^n, \xi \in \mathbb{R}_+^m} (1 - \mu) \sum_{(i,j) \in \mathcal{S}} \|x_i - x_j\|_M^2 + \mu \sum_{(i,j,k) \in \mathcal{R}} \xi_{ijk} \\
& \text{s.t. } \|x_i - x_k\|_M^2 - \|x_i - x_j\|_M^2 \geq 1 - \xi_{ijk}, \forall (i, j, k) \in \mathcal{R},
\end{aligned} \tag{2.17}$$

where $\mu \in (0, 1)$ weights the similarity and relative similarity objective terms and the new variable ξ encodes the margin of distance between relative similarities in the constraint. Minimising ξ in the objective encourages the distance in the constraint to be larger than the margin value of 1, while also minimising the distance

between similar pairs. Weinberger *et al.* find solutions to this convex optimisation problem using Projected Gradient Decent similar to MMC. Utilising relative similarities, LMNN demonstrated superior k -NN classification results and is one of the benchmark algorithms in Metric Learning.

The Linear Metric Learning algorithms we have presented thus far simply optimise the distance desired with respect to similarity, dissimilarity or relative similarity. This is a proxy for the goals of Metric Learning, such as minimising classification error. In this current paradigm classification error is highly discontinuous and not amenable to optimisation. One way to circumvent this is to model the classification problem probabilistically. This has the benefit of directly optimising the metric for the classification. Furthermore, by estimating probabilities these problems can retain their convexity and shed the constraints present in MMC and LMNN. We now introduce several probabilistic Metric Learning algorithms to illustrate their properties.

Probabilistic Metric Learning

The first Metric Learning algorithm to be based on a stochastic formulation was Neighbourhood Component Analysis (NCA) [18]. In this algorithm Goldberger *et al.* directly model Metric Learning as a k -NN classification problem. To make this classifier differentiable, NCA uses a softmax classifier and treats the outputs as probabilities:

$$p_{ij}(L) = \frac{\exp(-\|Lx_i - Lx_j\|_2^2)}{\sum_{k \neq i} \exp(-\|Lx_i - Lx_k\|_2^2)},$$

for $L \in \mathbb{R}^{p \times n}$, $p \leq n$. By selecting $p < n$ this algorithm learns a metric $M \in S_+(p, n)$, both reducing the dimensionality of the problem and obtaining benefits associated with dimensionality reduction, like PCA and LDA. For similar x_i and x_j the classifier should return $p_{ij}(L) = 1$, and $p_{ij}(L) = 0$ for dissimilar pairs. The

authors choose to maximise the probability of correct classification:

$$\max_{L \in \mathbb{R}^{p \times n}} \sum_{(i,j) \in \mathcal{S}} p_{ij}(L).$$

The linear transformation variable $L \in \mathbb{R}^{p \times n}$ allows a low-rank metric, i.e., $L^T L \in S_+(p, n)$ to be selected, although this comes at the expense of making the optimisation problem non-convex. NCA has closed-form gradient updates, so unconstrained gradient solvers may be used to optimise it.

An extension to NCA is Maximally Collapsing Metric Learning (MCML) [13]. Here the authors instead use a PSD variable $M \in S_{++}^n$ as their training variable to ensure a convex formulation, but retain a similar softmax classifier:

$$p_{ij}(M) = \frac{\exp(-\|x_i - x_j\|_M^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|_M^2)}.$$

Instead of maximising the classifier directly as NCA does, MCML minimises the KL-divergence between the ideal classifier, i.e. the one that correctly classifies pairs as either similar or dissimilar, and the softmax classifier $p_{ij}(M)$. The ideal classifier is specified by:

$$p_{ij}^* = \begin{cases} 1 & \text{if } (i, j) \in \mathcal{S} \\ 0 & \text{if } (i, j) \in \mathcal{D} \end{cases}.$$

The KL-divergence between the ideal classifier p^* and the classifier $p(M)$, parameterised by M is then the objective:

$$\min_{M \in S_+^n} D_{KL}(p^* || p(M)).$$

This objective is optimised using a projected gradient method with respect to the set of PSD matrices. This formulation was found to have improved performance over NCA and other Metric Learning algorithms. One of the benefits of the KL-divergence objective is that it incorporates dissimilarity relations as well as the similarity relations. This improves the performance of MCML compared to NCA.

Another probabilistically-motivated Metric Learning algorithm is Logistic Discriminant Metric Learning (LDML) [16]. LDML is similar to MCML and NCA in that it directly models the probability p_{ij} of x_i and x_j being similar datapoints. In this case, the probability of points x_i, x_j being similar is modelled as:

$$p_{ij}(M) = (1 + \exp(1 - \|x_i - x_j\|_M^2))^{-1},$$

where $1 - p_{ij}(M)$ correspondingly models the probability of x_i and x_j being dissimilar. This probability function is the logistic classifier on the distance between x_i and x_j , which is a soft-threshold on the classification of similarity, where the ideal probability distribution p_{ij}^* being the same as above. Logistic Discriminant Metric Learning takes a simplified approach to the probability estimation (relative to NCA and MCML) as the KL-divergence of the ideal classifier is taken with respect to the binary distribution $[p_{ij}(M), 1 - p_{ij}(M)]$ relative to each (i, j) . The optimisation problem for LDML therefore takes the form of a sum of KL-divergences between binary probability distributions of the ideal classifier, and the one described by $p_{ij}(M)$:

$$\min_{M \in S_+^n} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} p_{ij}^* \log(p_{ij}(M)) + (1 - p_{ij}^*) \log(1 - p_{ij}(M)),$$

referred to as the logistic discriminant analysis of the classifier $p_{ij}(M)$. The pairwise definition of the objective reduces the complexity of the objective function relative to NCA and MCML. Logistic Discriminant Metric Learning is optimised similarly to MCML using Projected Gradient Descent.

Dimensionality-Reduction Metric Learning (DRML) [121] extends LDML to learning a metric on $S_+(p, n)$. The authors factorise the Generalised Mahalanobis distance parameter as $M = V\tilde{M}V^T \in S_+(p, n)$ such that $\tilde{M} \in S_{++}^p$, $V \in St(p, n)$. As a result this factorisation M is rank p . Many Metric Learning algorithms perform dimensionality reduction as a pre-processing step for scalability on large datasets. DRML uses $V \in St(p, n)$ to incorporate dimensionality reduction into the algo-

rithm itself, so this can reflect the requirements of the problem. The binary classifier associated with DRML is:

$$p_{ij}(V\tilde{M}V^T) = (1 + \exp(1 - \|x_i - x_j\|_{V\tilde{M}V^T}^2))^{-1},$$

The problem for DRML is then constructed similarly to LDML:

$$\begin{aligned} \min_{\tilde{M} \in S_{++}^p, V \in St(p,n)} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} & \left[p_{ij}^* \log(p_{ij}(V\tilde{M}V^T)) + (1 - p_{ij}^*) \log(1 - p_{ij}(V\tilde{M}V^T)) \right] \\ & + \gamma D_{ld}(\tilde{M} \| \tilde{M}_0), \end{aligned} \quad (2.18)$$

where D_{ld} (the log-determinant divergence) is included as a regulariser with regularisation parameter γ , and p_{ij}^* is the ideal classifier described for LDML. This algorithm simultaneously learns a dimensionality reduction parameter V and a Mahalanobis metric M , hence its name. The authors derive a Riemannian metric to quotient out duplicate solutions in the product $V\tilde{M}V^T$ and then perform Riemannian Conjugate Gradient Descent. By incorporating dimensionality reduction into the algorithm DRML improves its performance relative to LDML and other Metric Learning algorithms by making a more informed choice of dimensionality reduction.

Information-Theoretic Metric Learning (ITML), was introduced by Davis *et al.* [12]. It bucks the trend of unconstrained optimisation for probabilistic Metric Learning methods. The Metric Learning algorithms reviewed thus far require an expensive projection onto S_{++}^n at every step during the optimisation. In contrast, ITML enforces the PSD constraint implicitly via Bregman projections, leading to an efficient optimisation routine. Instead of directly using a probability distribution on similarity and dissimilarity pairs, ITML presents these relationships as

constraints on the metric M . The optimisation problem for ITML is then:

$$\begin{aligned}
& \min_{\substack{M \in S_{++}^n \\ \xi \in \mathbb{R}_+^{|\mathcal{S} \cup \mathcal{D}|}}} D_{ld}(M \| M_0) + \gamma D_{ld}(\text{diag}(\xi) \| \text{diag}(\xi_0)) \\
& \text{s.t. } \|x_i - x_j\|_M^2 \leq \xi_{ij}, \forall (i, j) \in \mathcal{S} \\
& \quad \|x_i - x_j\|_M^2 \geq \xi_{ij}, \forall (i, j) \in \mathcal{D},
\end{aligned} \tag{2.19}$$

where $M_0 \in S_{++}^n$ is a prior metric that has moderate classification performance and D_{ld} is the log-determinant divergence (equation (2.3)). The variable ξ acts as a variable margin on the constraints to guarantee feasibility of the projection problem. Correspondingly, a log-determinant divergence regulariser for ξ is included in the objective. On the face of it, the formulation of ITML is not probabilistic, but the log-determinant divergence is equivalent to the KL-divergence between Gaussian distributions. Each constraint can be essentially mapped to an analogous $p_{ij}(M, \xi)$ term, as M and ξ induce a stochastic nature into the constraints. Because the constraints in M and ξ for ITML are linear and the objective is a Bregman function, Cyclic Projections may be used to solve this problem. The Cyclic Projections updates for ITML are closed-form and do not require an additional projection onto the S_{++}^n . As a result, ITML runs significantly faster than many of the probabilistic Metric Learning algorithms and attains high classification performance. ITML has been also extended to learning a metric in $S_+(p, n)$ [125]. This algorithm is equivalent to ITML except that the initial point of the optimisation is $M_0 \in S_+(p, n)$. However, because ITML is a linear algorithm, only a linear subspace of $S_+(p, n)$ is available during optimisation.

SPD Metric Learning

The Metric Learning algorithms we have examined thus far are limited to learning metrics on Euclidean data $\{x_i \in \mathbb{R}^n\}_{i=1}^N$ using the Generalised Mahalanobis distance. Another type of data worth considering is SPD-valued data $X_i \in S_{++}^n$.

There are many kinds of data that may be represented by SPD matrices including: actions [80], faces [26], pedestrians [22] and textures [21]. There exist Metric Learning algorithms that have been developed specifically for these types of data and we refer to these as SPD Metric Learning algorithms. Since the set of SPD matrices is a Riemannian manifold, many of the techniques used in Metric Learning for Euclidean data are not directly applicable. However, there are methods to adapt some existing algorithms to SPD Metric Learning.

Covariance Discriminative Learning (CDL) [26] was one of the first SPD Metric Learning algorithms. The target application for CDL was Metric Learning on image-sets, which arise as SPD descriptors of videos, for example. An image-set is $X = [x_1, x_2, \dots, x_N]$, where $x_i \in \mathbb{R}^n$ is the vectorised i -th image. To convert this into a SPD matrix the authors construct the covariance matrix:

$$C = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T,$$

where $\bar{x}$ is the average of x_i . By using a covariance matrix C the second-order information of X may be included in this representation. Once a set of covariance matrices has been constructed from image-sets, CDL transforms these matrices using the matrix logarithm, mapping the covariance matrices to Euclidean space. Linear Discriminant Analysis is then performed on the linearly structured mappings to produce a linear discriminant for nearest neighbour classification. This process is equivalent to a Kernelised LDA algorithm [126] by way of the matrix logarithm. Similarly, Jayasumana *et al.* [127] perform PCA using the matrix logarithm mapping on SPD data. However, these methods ignore some of the structure of the SPD manifold and only learn a dimensionality reduction technique.

Symmetric Positive Definite Manifold Learning (SPDML), introduced by Harandi *et al.* [80, 128], also learns a dimensionality-reduction function on SPD matrices. This algorithm aims to learn a discriminant that is more faithful to the geometry of the data than CDL. The approach differs from CDL by learning a projection from

the target SPD manifold to a lower-dimensional SPD manifold, rather than learning a discriminative classifier on linearised data. The projection mapping used in SPDML is expressed as:

$$f_V(X) = VXV^T$$

where $V \in St(p, n)$ parameterises the projection. SPDML solves:

$$\min_{V \in St(p, n)} \sum_{(i, j) \in \mathcal{S}} d(f_V(X_i), f_V(X_j))^2 - \sum_{(i, j) \in \mathcal{D}} d(f_V(X_i), f_V(X_j))^2,$$

where d is a distance or divergence on SPD matrices such as the log-Euclidean distance (equation 2.6), Affine-Invariant distance (equation 2.5), Stein divergence or Jeffery divergence (two symmetrised versions of the log-determinant divergence). Harandi *et al.* found the benefits of the dimensionality reduction to be two-fold. First, the reduction in dimension typically reduces the k -NN error relative to nearest neighbour classification in the original space. Secondly, the reduction in dimensionality reduces the cost of training and testing relative to the distances and divergences. These methods generally outperform CDL on classification problems and a decrease in the complexity of SPDML compared to CDL is also noted.

Log-Euclidean Metric Learning (LEML) [25] is an adaptation of ITML for learning a distance on the set of SPD matrices. LEML builds on the approach of SPDML by learning a mapping on the tangent space associated with the log-Euclidean manifold. The log-Euclidean distance on SPD matrices $X_i, X_j \in S_{++}^n$ is augmented with a linear function $L \in \mathbb{R}^{p \times n}$:

$$\|L \log(X_i)L^T - L \log(X_j)L^T\|_F$$

From this distance a Mahalanobis-like distance function is derived:

$$d_{LE}^M(X_i, X_j) = \text{trace}((\log(X_i) - \log(X_j))M(\log(X_i) - \log(X_j))),$$

where $M = (L^T L)^2 \in S_{++}^n$. Using this distance, the authors construct the problem:

$$\begin{aligned} \min_{M \in S_{++}^n, \xi \in \mathbb{R}_+^n} & D_{ld}(M \| M_0) + \gamma D_{ld}(\text{diag}(\xi) \| \text{diag}(\xi_0)) \\ \text{s.t. } & d_{LE}^M(X_i, X_j) \leq \xi_{ij}, \forall (i, j) \in \mathcal{S} \\ & d_{LE}^M(X_i, X_j) \geq \xi_{ij}, \forall (i, j) \in \mathcal{D}, \end{aligned} \quad (2.20)$$

which is ITML with the modified log-Euclidean distance substituted for the Generalised Mahalanobis distance. Other algorithms [129, 130] consider similar Metric Learning formulations, but LEML distinguishes itself through its parameterisation of the metric. Where previous algorithms vectorised the training data and learned a metric in a large number of dimensions, the metric M is the same size as the input SPD matrices, making for a much simpler problem. The use of Cyclic Projections with the closed-form updates of ITML, modified to accommodate the difference in the constraints, results in an algorithm significantly faster than SPDML, and has improved classification error.

An alternative approach for performing classification on SPD manifolds is to use SPD Dictionary Learning [131]. This approach learns a set of SPD atoms $B_i \in S_{++}^n$ that can be used to map from a test point $X_i \in S_{++}^n$ to a classification vector. Information Divergence Dictionary Learning (IDDL) [132, 133] is a fairly recent dictionary learning algorithm that uses an information-theoretic formulation. The optimisation problem takes the form:

$$\begin{aligned} \min_{\substack{B_i \in S_{++}^n, i=1, \dots, m, \\ W \in \mathbb{R}^{C \times m}, \alpha > 0, \beta > 0}} & \frac{1}{2} \sum_{i=1}^N \|h_i - W v_i\|_2^2 + \gamma \|W\|_F^2 \\ \text{s.t. } & v_i^k = D_{ld}^{(\alpha, \beta)}(X_i \| B_k), \end{aligned} \quad (2.21)$$

where $D_{ld}^{(\alpha, \beta)}(X_i \| B_k)$ is the $\alpha\beta$ log-determinant divergence and h_i encodes label information for X_i . For different parameters of α and β the $\alpha\beta$ log-determinant divergence is equivalent to several different divergences and in the limit of α and β to zero, $D_{ld}^{(\alpha, \beta)}$ yields the Affine-Invariant distance. Information Divergence Dictionary Learning learns a linear classifier $W \in \mathbb{R}^{C \times m}$ on the output of divergences

$v_i \in \mathbb{R}^m$ with respect to the true labels h_i . This type of algorithm learns significantly more parameters than a typical Metric Learning algorithm, which gives it a higher level of discriminatory power, albeit at the cost of increased dimensionality and complexity.

Optimal Transport in Metric Learning

While the probabilistic methods used in Linear Metric Learning are predominantly based on information-theoretic formulations, there are some alternatives based on Optimal Transport. However, the optimisation formulation of Optimal Transport typically means Metric Learning applications are more difficult to optimise and methods to mitigate this are required. Here we consider some relevant algorithms in Metric Learning that use Optimal Transport.

Ground Metric Learning (GML) [62] is seen as a supervised extension of the Earth Movers' Distance [43] that uses the W_1 distance for image retrieval. Instead of the W_1 distance, GML optimises the ground distance matrix C in expression (2.12) with respect to similarity and dissimilarity sets on histogram data $\{h_i \in \Delta^n\}_{i=1}^N$. Cuturi *et al.* believed they could improve the performance for classification on histogram features by incorporating label data into this type of metric. The ground metric distance between histograms h_i, h_j with respect to the ground metric M is:

$$G_{ij}(M) = \min_{P \in \Pi(h_i, h_j)} \langle M, P \rangle.$$

The authors of GML also constrained M to a set of metric matrices $\mathcal{M}_+$ that are typical of the ground metrics encountered when using Wasserstein distances. The problem that GML solves is:

$$\min_{M \in \mathcal{M}_+} \sum_{(i,j) \in \mathcal{S}} G_{ij}(M) - \sum_{(i,j) \in \mathcal{D}} G_{ij}(M). \quad (2.22)$$

This is optimised using a projected subgradient algorithm, with additional care being taken during initialisation due to the problem's non-convexity. While GML

performs well in experiments presented against ITML and LMNN, solving linear programs in M for each $(i, j) \in \mathcal{S} \cup \mathcal{D}$ is computationally expensive. This, as well as its non-convexity, makes GML more difficult to train than many Metric Learning algorithms.

An Optimal Transport method motivated by Linear Discriminant Analysis is Wasserstein Discriminant Analysis (WDA) [52]. This dimensionality reduction algorithm learns a projection $V \in St(p, n)$ that minimises within-class Wasserstein distances and maximises between-class Wasserstein distances, analogous to LDA. Wasserstein Discriminant Analysis acts on collections of points modelled by discrete distributions of a particular class c with probability mass $a^c \in \Delta^{N_c}$ located at $x_i^c \in \mathbb{R}^n$, where c denotes membership to a class c with N_c members. Applying the projection operator V^T to each of these distributions maps the location of the index x_i^c to $V^T x_i^c$. WDA finds the solution to:

$$\max_{V \in St(p, n)} \frac{\sum_{c \neq c'} W_{V,2}^\epsilon(a^c, a^{c'})}{\sum_c W_{V,2}^\epsilon(a^c, a^c)}, \quad (2.23)$$

where $W_{V,2}^\epsilon(a^c, a^{c'})$ is the entropic 2-Wasserstein distance between distributions a^c and $a^{c'}$ with the quadratic cost matrix $C_{ij} = \|V^T x_i^c - V^T x_j^{c'}\|_2^2$. The choice of the entropic 2-Wasserstein distance allows for increased efficiency in training, via use of the Sinkhorn algorithm to evaluate and differentiate the objective. The choice of regularisation parameter ϵ also allows for a trade-off between learning the projection relative to global features and local features. In comparison with other dimensionality reduction algorithms and LMNN, WDA performs well on k -NN classification tasks and converges quickly. While WDA improves on the scalability of GML by using the Sinkhorn distance, the linear projection it learns is not as expressive as a general metric, mirroring the shortcomings of LDA.

Supervised Word Mover's Distance (S-WMD) [63] is an extension of NCA and the Word Movers Distance [61] that trains a linear metric on the entropic 2-Wasserstein distance. Similar to WDA, S-WMD learns a linear function on the locations $x_k^i \in$

$\mathbb{R}^n$ of probability mass $a^i \in \Delta^m$, except in this case the linear transform is a general linear transform instead of a projection. The Supervised Word Movers Distance also learns a weighting vector $w \in \mathbb{R}_+^n$ on the weights of the Dirac-delta distributions. The weighting vector acting on a^i is:

$$w(a^i) = \frac{w \odot a^i}{w^T a^i}.$$

The model for the probability of similarity of distributions a^i and a^j , with locations x_k^i and y_l^j respectively, is then formulated as:

$$p_{ij}(L, w) = \frac{\exp(-W_{L,2}^\epsilon(w(a^i), w(a^j)))}{\sum_{k \neq i} \exp(-W_{L,2}^\epsilon(w(a^i), w(a^j)))},$$

where the authors choose to reduce the complexity of evaluating the problem by using the Sinkhorn distance and the cost matrix associated with $W_{L,2}$ is $C_{kl} = \|L^T x_k^i - L^T y_l^j\|_2^2$, similar to WDA. Using this, the problem that S-WMD solves is:

$$\min_{L \in \mathbb{R}^{p \times n}, w \in \mathbb{R}_+^n} - \sum_{(i,j) \in \mathcal{S}} \log(p_{ij}(L, w)),$$

which is the sum of log-likelihood terms associated with the similarity relation, analogous to NCA. S-WMD is specifically applied to learning distances on documents described by bag-of-words and frequency-inverse document frequency representations [134], which are amenable to Dirac-delta representations. The k -NN classification performance of S-WMD was evaluated against ITML, LMNN and NCA, where S-WMD frequently had the best classification performance on the datasets presented.

Han *et al.* briefly considered a Metric Learning algorithm, which we refer to as Generalised Bures-Wasserstein Metric Learning (GBWML), for learning on SPD matrices [135]. In this work the authors define the Generalised Bures-Wasserstein (GWB) distance:

$$\mathcal{GB}_M(A, B) = \sqrt{\text{trace}(MA + MB - 2(A^{\frac{1}{2}}MBMA^{\frac{1}{2}})^{\frac{1}{2}})}.$$

They then define a probability function on similarity of SPD matrices using the GBW distance:

$$p_{ij}(M) = (1 + \exp(1 - \mathcal{B}_M^2(X_i, X_j)))^{-1},$$

with respect to the variable M . This probability is used to define a logistic discriminant objective, as in LDML. The authors demonstrate convergence properties of this objective with their geometry relative to the other SPD matrix geometries, and indicate that convergence is generally faster with the generalised Bures-Wasserstein distance.

We have seen some of the more significant contributions of Optimal Transport to Metric Learning. Aside from GBWML none of the Optimal Transport methods we have introduced use the Bures-Wasserstein distance. This now motivates us to introduce our first chapter on Metric Learning with the Bures-Wasserstein distance.

Chapter 3

Bures-Wasserstein Metric Learning

3.1 Introduction

The Bures-Wasserstein distance provides the ability to compare symmetric positive definite matrices, such as those that parameterise the Generalised Mahalanobis distance. As seen in the literature review, information-theoretic methods have been a successful approach in Metric Learning. A prominent Linear Metric Learning algorithm that we have already introduced as Problem (2.19) is ITML [12]. In this chapter ITML is used as inspiration for Linear Metric Learning algorithms that utilise the Bures-Wasserstein distance. One motivation for using the Bures-Wasserstein distance instead of an information-theoretic divergence is the Talagrand Inequality [136] which specifies that:

$$W_2(\mu, \nu) \leq 2D_{KL}(\mu||\nu)$$

for two distributions μ and ν . This indicates that when minimising the KL-divergence the 2-Wasserstein distance will also be implicitly minimised. ITML minimises the log-determinant divergence (Equation (2.3)) between PSD matrices, which is equivalent to the KL-divergence between corresponding zero-mean gaussians, where the Bures-Wasserstein distance is a lower bound for this. This suggests that minimising the Bures-Wasserstein distance instead may result in useful metrics. Additionally, using the Wasserstein distance offers some beneficial proper-

ties. The Bures-Wasserstein distance is convex [83], which allows us to use the efficient convex optimisation algorithms introduced in the literature review. As we have also seen, the Bures-Wasserstein distance can also be induced by a Riemannian metric on S_{++}^n [69]. We may use this to develop algorithms that incorporate the manifold structure of the SPD matrices.

This chapter introduces three classes of Linear Metric Learning Algorithms that use the Bures-Wasserstein distance. The first, and most similar to ITML is Bures-Wasserstein Metric Learning (BWML) presented in section 3.2.1, which replaces the log-determinant objective term in ITML with the Bures-Wasserstein distance, but retains the other aspects of the algorithm. The next class of algorithm, referred to as Bures-Wasserstein Barycentric Metric Learning (BWBML) presented in section 3.2.2, phrases the Metric Learning problem as a barycentre problem. The third class of algorithm, Transport Bures-Wasserstein Metric Learning (T-BWML) presented in section 3.2.3, learns a factorisation of the desired metric using gradient-based methods, in contrast to the projection-based algorithms.

Each of these classes of algorithm uses a different framework to solve for a metric and we wish to see what their respective advantages and disadvantages are. The convergence and timing performance of these algorithms is first studied on synthetic data. Each algorithm is found to converge reliably, and some characteristics of convergence are examined in section 3.4.1. These tests are made in comparison with ITML in order to understand how the Bures-Wasserstein distance influences the performance of each algorithm. We then examine the k -nearest-neighbours classification (k -NN) performance of the algorithms for Metric Learning on UCI datasets in section 3.4.2, and k -NN classification on MNIST [137] and Fashion MNIST [138] in section 3.4.3. Another prominent Metric Learning algorithm LMNN [14] (Problem (2.17)) is also included in these tests as it has been benchmark algorithm in Metric Learning and provides a solid anchor to the literature. The error rates of our algorithms are found to be equal to or better than ITML on the UCI datasets, however on MNIST and Fashion MNIST performance

is more mixed. The error rates of our algorithms are comparable with benchmark algorithms and converge fairly quickly, although they do not substantially improve on the benchmark performance. This may be a limitation of learning in a linear paradigm, however it is interesting to see how the choices we make in algorithm design affect performance in specific ways.

3.2 Learning a Metric with the Bures-Wasserstein Distance

Here we show how to use the Bures-Wasserstein distance to compute metrics for classification tasks. The three classes of Metric Learning algorithm presented here all derive from the same basic formulation. That is, they find the closest metric in a feasible set with respect to the Bures-Wasserstein distance from an initial metric M_0 . As a reminder the Bures-Wasserstein distance is:

$$\mathcal{B}^2(M, M_0) = \text{trace}(M + M_0 - 2(M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}})^{\frac{1}{2}}),$$

and its gradient is:

$$\nabla_M \mathcal{B}^2(M, M_0) = I - M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}}.$$

A specific case of the Bures-Wasserstein distance we will encounter is the distance between diagonal PSD matrices, i.e.

$$\begin{aligned} \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0)) &= \text{trace}(\text{diag}(\xi) + \text{diag}(\xi_0) - 2(\text{diag}(\xi_0)^{\frac{1}{2}} \text{diag}(\xi) \text{diag}(\xi_0)^{\frac{1}{2}})^{\frac{1}{2}}) \\ &= \sum_{i=1}^m \xi_i + \xi_{0,i} - 2\sqrt{\xi_i \xi_{0,i}} \\ &= \|\sqrt{\xi} - \sqrt{\xi_0}\|_2^2, \end{aligned}$$

where $\xi, \xi_0 \in \mathbb{R}_+^m$ and its gradient is:

$$\nabla_{\xi} [\mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0))] = 1 - \sqrt{\xi_0}/\sqrt{\xi}.$$

Feasible metrics are described as the intersection of a set of similarity and dissimilarity constraints as in ITML. These constraints take the form of hyperplanes in the positive semidefinite manifold of matrices. The three classes of algorithms respectively rely on: Cyclic Projections, barycentric calculations and second-order gradients. We begin with the analysis and description of BWML, as a Cyclic Projections algorithm.

3.2.1 Bures-Wasserstein Metric Learning

Following the formulation of ITML using similarity and dissimilarity constraints, and using the Bures-Wasserstein distance as the objective value, a new Metric Learning problem can be formulated as:

$$\begin{aligned}
& \min_{M \in \mathbb{S}_{++}^n} \mathcal{B}^2(M, M_0) \\
& \text{s.t. } (x_i - x_j)^T M (x_i - x_j) \leq u, (i, j) \in \mathcal{S} \\
& \quad (x_i - x_j)^T M (x_i - x_j) \geq l, (i, j) \in \mathcal{D},
\end{aligned} \tag{3.1}$$

where M_0 is an initial metric and l and u are margin terms. These terms are generally selected heuristically and are subject to some level of tuning. In practice, parameter values of l and u are not guaranteed to provide a non-empty set of solutions to Problem (3.1). To address this, the problem can be modified to include variable margins $\xi \in \mathbb{R}_+^m$ that replace the static values u and l , where $m = |\mathcal{S}| + |\mathcal{D}|$ is the total number of constraints. The addition of the variable ξ ensures that the constraint set is non-empty. Introducing ξ requires an extra regularisation term and a natural choice for this is the Bures-Wasserstein distance between $\text{diag}(\xi)$ and $\text{diag}(\xi_0)$ (where ξ_0 is specified by l and u). The resulting problem is:

$$\begin{aligned}
& \min_{M \in \mathbb{S}_{++}^n, \xi \in \mathbb{R}_{++}^m} \mathcal{B}^2(M, M_0) + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0)) \\
& \text{s.t. } (x_i - x_j)^T M (x_i - x_j) \leq \xi_{ij}, (i, j) \in \mathcal{S} \\
& \quad (x_i - x_j)^T M (x_i - x_j) \geq \xi_{ij}, (i, j) \in \mathcal{D},
\end{aligned} \tag{3.2}$$

where a regularisation parameter γ has been introduced, that specifies how strictly the problem should be regularised with respect to the initial margins. This parameter γ is generally tuned to optimise the performance of the metric. The objective of problem (3.2) is convex, since the Bures-Wasserstein distance is convex [83]. The constraints are also linear with respect to the optimisation variables, therefore Problem (3.2) is a convex optimisation problem.

Similar to ITML we choose to solve Problem (3.2) using the method of Cyclic Projections [100]. To make Problem (3.2) conform to the formulation required for Cyclic Projections (§2 - Algorithm 4) we make some substitutions. Firstly, we use matrix variable $\Xi \in S_+^m$ instead of $\text{diag}(\xi) \in S_+^m$. In practice, as we will show, Ξ always updates to a diagonal matrix so it can be mapped to ξ via $\xi = \text{diag}(\Xi)$ to reduce computational complexity. For the constraints, this requires the substitution $\xi_{ij} = \langle E_{ij}, \Xi \rangle$ where $E_{ij} \in S_+^m$ is 1 at the ij -th position along the diagonal and zeros elsewhere. Another substitution is $(x_i - x_j)^T M (x_i - x_j) = \langle X_{ij}, M \rangle$ where $X_{ij} = (x_i - x_j)(x_i - x_j)^T \in S_+^n$. This reformulates Problem (3.2) as:

$$\begin{aligned} \min_{M \in S_{++}^n, \Xi \in S_+^m} \quad & \mathcal{B}^2(M, M_0) + \gamma \mathcal{B}^2(\Xi, \Xi_0) \\ \text{s.t.} \quad & \langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle \leq 0, (i, j) \in \mathcal{S} \\ & \langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle \geq 0, (i, j) \in \mathcal{D}. \end{aligned} \tag{3.3}$$

This makes clear that the constraints are hyperplane constraints in two SPD variables. Next, we show that Problem (3.2) satisfies the conditions (i)-(iv) of Theorem 2.2.8 to ensure convergence of the algorithm.

The first condition is that $g(M, \Xi) = \mathcal{B}(M, M_0) + \gamma \mathcal{B}(\Xi, \Xi_0)$, the objective function of (3.2), is a Bregman function.

Proposition 3.2.1 *$f(M) = \mathcal{B}^2(M, M_0)$ is a Bregman function.*

Proof: We show that f satisfies conditions (i)-(vi) in Definition 2.2.5. The zone of f is the set of positive definite matrices S_{++}^n , where its closure is the set of positive semidefinite matrices is S_+^n .

(i) The gradient of f is $\nabla f(M) = I - M_0^{\frac{1}{2}}(M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}})^{-\frac{1}{2}}M_0^{\frac{1}{2}}$, which is continuous on every $X \in S_{++}^n$ as the inverse, matrix square root and subtraction operations are all continuous functions and so their composition is too.

(ii) Here variables $M, N, M_0 \in S_{++}^n$ are used,

$$\begin{aligned}
f(\theta M + (1 - \theta)N) &= \mathcal{B}^2(\theta M + (1 - \theta)N, M_0) \\
&= \text{trace}(\theta M + (1 - \theta)N + M_0 - (M_0^{\frac{1}{2}}(\theta M + (1 - \theta)N)M_0^{\frac{1}{2}})^{\frac{1}{2}}) \\
&= \theta \text{trace}(M + M_0) + (1 - \theta) \text{trace}(N + M_0) \\
&\quad - 2\text{trace}((\theta M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}} + (1 - \theta)M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{\frac{1}{2}}) \\
&< \theta \text{trace}(M + M_0 - 2(M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}})^{\frac{1}{2}}) \\
&\quad + (1 - \theta) \text{trace}(N + M_0 - 2(M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{\frac{1}{2}}) \\
&= \theta f(M) + (1 - \theta)f(N),
\end{aligned}$$

where the inequality is due to the strong concavity of the map $M \mapsto \text{trace}(M^{\frac{1}{2}})$.

(iii) The trace function, addition/subtraction and matrix square root operations are continuous on S_{++}^n . f is a composition of these operators so it is continuous.

(iv) It is required that $D_f(M, N) \leq \alpha$ with fixed M or N is bounded. We first derive D_f :

$$\begin{aligned}
D_f(M, N) &= f(M) - f(N) - \langle \nabla f(N), M - N \rangle \\
&= \text{trace}(M + M_0 - 2(M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}})^{\frac{1}{2}}) - \text{trace}(N + M_0 - 2(M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{\frac{1}{2}}) \\
&\quad - \langle I - M_0^{\frac{1}{2}}(M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{-\frac{1}{2}}M_0^{\frac{1}{2}}, M - N \rangle \\
&= \text{trace}(M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}}(M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{-\frac{1}{2}} - 2(M_0^{\frac{1}{2}}MM_0^{\frac{1}{2}})^{\frac{1}{2}} + (M_0^{\frac{1}{2}}NM_0^{\frac{1}{2}})^{\frac{1}{2}}).
\end{aligned}$$

For brevity we define $\tilde{M} = M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}}$, $\tilde{N} = M_0^{\frac{1}{2}} N M_0^{\frac{1}{2}}$, so

$$\begin{aligned} D_f(M, N) &= \text{trace}(\tilde{M} \tilde{N}^{-\frac{1}{2}} - 2\tilde{M}^{\frac{1}{2}} + \tilde{N}^{\frac{1}{2}}) \\ &= \text{trace}(\tilde{N}^{-\frac{1}{4}} \tilde{M} \tilde{N}^{-\frac{1}{4}} - 2\tilde{M}^{\frac{1}{2}} + \tilde{N}^{\frac{1}{2}}) \\ &= \text{trace}((\tilde{M}^{\frac{1}{2}} \tilde{N}^{-\frac{1}{4}} - \tilde{N}^{\frac{1}{4}})^T (\tilde{M}^{\frac{1}{2}} \tilde{N}^{-\frac{1}{4}} - \tilde{N}^{\frac{1}{4}})) \\ &= \|\tilde{M}^{\frac{1}{2}} \tilde{N}^{-\frac{1}{4}} - \tilde{N}^{\frac{1}{4}}\|_F^2. \end{aligned}$$

It is clear that $\|\tilde{M}^{\frac{1}{2}} \tilde{N}^{-\frac{1}{4}} - \tilde{N}^{\frac{1}{4}}\|_F^2 \leq \alpha$ is bounded with respect to one variable if the other is fixed, hence $L_1^f(N, \alpha)$ and $L_2^f(M, \alpha)$ are bounded.

(v) Let $N_i \in S_{++}^n$ for all $i \in \mathbb{N}$ and $\lim_{i \rightarrow \infty} N_i = N^*$ then:

$$\begin{aligned} &\lim_{i \rightarrow \infty} D_f(N^*, N_i) \\ &= \lim_{i \rightarrow \infty} \|\tilde{N}^{*\frac{1}{2}} \tilde{N}_i^{-\frac{1}{4}} - \tilde{N}_i^{\frac{1}{4}}\|_F^2 \\ &= \|\tilde{N}^{*\frac{1}{2}} \tilde{N}^{*\frac{-1}{4}} - \tilde{N}^{*\frac{1}{4}}\|_F^2 \\ &= 0. \end{aligned}$$

(vi) Let $N_i \in S_{++}^n$ and $M_i \in S_+^n, \forall i \in \mathbb{N}$. Assume $\lim_{i \rightarrow \infty} N_i = N^*$ and $\lim_{i \rightarrow \infty} D_f(M_i, N_i) = 0$ and $\{M_i\}_{i \in \mathbb{N}}$ is bounded.

$$\begin{aligned} &\lim_{i \rightarrow \infty} D_f(M_i, N_i) \\ &= \lim_{i \rightarrow \infty} D_f(M_i, N^*) \\ &= 0, \end{aligned}$$

as D_f is continuous in N and M_i is convergent and $D_f(M, N) = 0$ implies $\tilde{M}^{\frac{1}{2}} \tilde{N}^{-\frac{1}{4}} - \tilde{N}^{\frac{1}{4}} = 0$. Therefore $\lim_{i \rightarrow \infty} M_i = N^*$.

■

Corollary 3.2.1.1 $g(M, \Xi) = f(M) + \gamma f(\Xi)$ is a Bregman function.

Condition (ii) is that $f(M, \xi)$ is strongly zone consistent with respect to each H_i .

Lemma 3.2.2 $g(M, \xi)$ is strongly zone consistent with respect to each halfspace $H_{ij} = \{(M, \Xi) : \langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle = 0\}$

Proof: The generalised projection of $(N, \Sigma) \in S_+^n \times S_+^m$ with respect to g is:

$$\begin{aligned} \min_{M \in S_+^n, \Xi \in S_+^m} D_f(M, N) + \gamma D_f(\Xi, \Sigma) \\ \text{s.t. } \langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle = 0. \end{aligned}$$

All that is required is to show that the solutions M^* and Ξ^* to the above problem are members of S_{++}^n and S_{++}^m respectively. The gradient of the objective is

$$\begin{aligned} \nabla_M D_f(M, N) &= M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} N M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}} - M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}} \\ \nabla_{\Xi} \gamma D_f(\Xi, \Xi_0) &= \gamma (\Xi_0^{\frac{1}{2}} (\Xi_0^{\frac{1}{2}} \Sigma \Xi_0^{\frac{1}{2}})^{-\frac{1}{2}} \Xi_0^{\frac{1}{2}} - \Xi_0^{\frac{1}{2}} (\Xi_0^{\frac{1}{2}} \Xi \Xi_0^{\frac{1}{2}})^{-\frac{1}{2}} \Xi_0^{\frac{1}{2}}). \end{aligned}$$

The gradient with respect to the variables will go to infinity as their eigenvalues shrink to zero. Therefore the solutions of the above problem, and projections onto parallel hyperplanes, cannot have zero eigenvalues. ■

Elements (i, j) of $\mathcal{S} \cup \mathcal{D}$ are sampled sequentially in our implementation, which satisfies condition (iii) of Theorem 2.2.8.

Finally, we show that condition (iv) of Theorem 2.2.8 is true for Problem (3.3).

Lemma 3.2.3 For Problem (3.3), the set:

$$Z_0 = \{(M, \xi) \in S_{++}^n \times \mathbb{R}_{++} : \exists z \in \mathbb{R}_+^m \text{ such that } \nabla g(M, \Xi) = -\mathcal{A}^T z\},$$

is non-empty.

Proof: The first task is to establish an expression for the linear operator $\mathcal{A}^T : \mathbb{R} \rightarrow S_{++}^n \times S_{++}^m$. The expression of the linear operator $\mathcal{A} : S_+^n \times S_+^m \rightarrow \mathbb{R}^m$ with respect to the ij^{th} constraint defined in Problem (3.2) is:

$$\mathcal{A}(M, \Xi)_{ij} = \langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle.$$

Then:

$$\begin{aligned} & \langle \mathcal{A}(M, \Xi), z \rangle \\ &= \sum_{ij \in [m]} (\langle X_{ij}, M \rangle - \langle E_{ij}, \Xi \rangle) z_{ij} \\ &= \langle \sum_{ij \in [m]} z_{ij} X_{ij}, M \rangle - \langle \sum_{ij \in [m]} z_{ij} E_{ij}, \Xi \rangle \end{aligned}$$

and so

$$\begin{aligned} \mathcal{A}^T z &= \sum_{ij \in [m]} z_{ij} \text{diag}(X_{ij}, -E_{ij}) \\ &= \text{diag}\left(\sum_{ij \in [m]} z_{ij} X_{ij}, -\sum_{ij \in [m]} z_{ij} E_{ij}\right). \end{aligned}$$

Therefore, by substituting the relevant gradients and the above linear operator, the equation $\nabla g(M, \Xi) = -\mathcal{A}^T z$ is expressible as:

$$\text{diag}(I - M_0^{\frac{1}{2}}(M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}}, I - \Xi_0^{\frac{1}{2}}(\Xi_0^{\frac{1}{2}} \Xi \Xi_0^{\frac{1}{2}})^{-\frac{1}{2}} \Xi_0^{\frac{1}{2}}) = -\text{diag}\left(\sum_{ij \in [m]} z_{ij} X_{ij}, -\sum_{ij \in [m]} z_{ij} E_{ij}\right).$$

The two components of the diagonal operator can be solved simultaneously, which results in the equations:

$$\begin{aligned} M &= \left(\sum_{ij \in [m]} z_{ij} X_{ij} - I \right)^{-1} M_0 \left(\sum_{ij \in [m]} z_{ij} X_{ij} - I \right)^{-1} \\ \Xi &= \left(I + \sum_{ij \in [m]} z_{ij} E_{ij} \right)^{-1} \Xi_0 \left(I + \sum_{ij \in [m]} z_{ij} E_{ij} \right)^{-1}. \end{aligned}$$

The only restriction, which is that $M \in S_{++}^n$ and $\Xi \in S_{++}^m$ is in the expression for M . $z \in \mathbb{R}_+^m$ must be large enough that $\sum_{ij \in [m]} z_{ij} X_{ij} - I \in S_{++}^n$. Since $X_{ij} \in S_+^n$ if z is selected large enough then the constraint is adhered to, so $Z_0 \neq \emptyset$. Similarly, if z is strictly positive then $\Xi \in S_{++}^n$. ■

With these theoretical properties of BWML established we may proceed with the derivation of the algorithm. Applying Cyclic Projections to Problem (3.3), we derive the updates for BWML. An iteration of BWML with respect to the hyperplane H_{ij} constraint is performed in 3 steps: a projection parameter $\alpha \in \mathbb{R}$ that projects M_t and Ξ_t onto H_{ij} is computed, this parameter then updates the correction vector λ_{ij} and may itself be corrected to $\hat{\alpha}$, and then the corrected projection parameter $\hat{\alpha}$ is used to compute M_{t+1} and Ξ_{t+1} .

We begin by deriving the update equations for M_{t+1} and Ξ_{t+1} , then we derive the equation for the projection parameter α . The updates for BWML are defined by the equation:

$$\nabla_{(M_{t+1}, \Xi_{t+1})}[g(M_{t+1}, \Xi_{t+1})] = \nabla_{(M_t, \Xi_t)}[g(M_t, \Xi_t)] + \delta_{ij} \alpha \text{diag}(X_{ij}, -E_{ij}).$$

where the above value of δ_{ij} is 1 if $(i, j) \in S$ or -1 if $(i, j) \in D$, which accounts for the difference between these two types of constraint, and α is a projection parameter. By substituting $g(M, \Xi) = f(M) + \gamma f(\Xi)$ we can split this equation into two independent equations of M and Ξ respectively:

$$\nabla_{M_{t+1}} f(M_{t+1}) = \nabla_{M_t} f(M_t) + \delta_{ij} \alpha X_{ij} \quad (3.4)$$

$$\gamma \nabla_{\Xi_{t+1}} f(\Xi_{t+1}) = \gamma \nabla_{\Xi_t} f(\Xi_t) - \delta_{ij} \alpha E_{ij}. \quad (3.5)$$

Starting with equation (3.4), we substitute the gradient expression for f :

$$\begin{aligned} I - M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} M_{t+1} M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}} &= I - M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}} + \delta_{ij} \alpha X_{ij} \\ \iff (M_0^{\frac{1}{2}} M_{t+1} M_0^{\frac{1}{2}})^{-\frac{1}{2}} &= (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{-\frac{1}{2}} - \delta_{ij} \alpha M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} \\ \iff (M_0^{\frac{1}{2}} M_{t+1} M_0^{\frac{1}{2}})^{\frac{1}{2}} &= \left((M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{-\frac{1}{2}} - \delta_{ij} \alpha M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} \right)^{-1}. \end{aligned}$$

3.2. LEARNING A METRIC WITH THE BURES-WASSERSTEIN DISTANCE 67

To simplify the right-hand side we may apply the Sherman-Morrison inverse identity [139] since $X_{ij} = (x_i - x_j)(x_i - x_j)^T$ is a rank one matrix. This gives the expression:

$$\begin{aligned} (M_0^{\frac{1}{2}} M_{t+1} M_0^{\frac{1}{2}})^{\frac{1}{2}} &= (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{\frac{1}{2}} \\ &+ \delta_{ij} \frac{\alpha (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{\frac{1}{2}} M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{\frac{1}{2}}}{(1 - \delta_{ij} \alpha (x_i - x_j)^T M_0^{-\frac{1}{2}} (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{\frac{1}{2}} M_0^{-\frac{1}{2}} (x_i - x_j))}. \end{aligned} \quad (3.6)$$

It would be very costly to solve explicitly for M_{t+1} using Equation (3.6) at each iteration, since it requires computing matrix square roots. We can circumvent this problem by solving in terms of the matrix square root directly. We label this variable as $Z_t = (M_0^{\frac{1}{2}} M_t M_0^{\frac{1}{2}})^{\frac{1}{2}}$, where it is clear that this transformation is invertible. Making this substitution in Equation (3.6) results in the update equation:

$$Z_{t+1} = Z_t + \delta_{ij} \frac{\alpha Z_t M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} Z_t}{1 - \delta_{ij} \alpha (x_i - x_j)^T M_0^{-\frac{1}{2}} Z_t M_0^{-\frac{1}{2}} (x_i - x_j)}. \quad (3.7)$$

This expression is simpler to calculate and does not require any repeated matrix square root terms. The $M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}}$ and $M_0^{-\frac{1}{2}} (x_i - x_j)$ terms can also be precomputed as they do not change during execution.

To find the update expression for Ξ_{t+1} starting from expression (3.5), we use analogous steps from the derivation for (3.6) to find:

$$\begin{aligned} (\Xi_0^{\frac{1}{2}} \Xi_{t+1} \Xi_0^{\frac{1}{2}})^{\frac{1}{2}} &= (\Xi_0^{\frac{1}{2}} \Xi_t \Xi_0^{\frac{1}{2}})^{\frac{1}{2}} \\ &- \delta_{ij} \frac{\alpha (\Xi_0^{\frac{1}{2}} \Xi_t \Xi_0^{\frac{1}{2}})^{\frac{1}{2}} \Xi_0^{-\frac{1}{2}} E_{ij} \Xi_0^{-\frac{1}{2}} (\Xi_0^{\frac{1}{2}} \Xi_t \Xi_0^{\frac{1}{2}})^{\frac{1}{2}}}{(\gamma + \delta_{ij} \alpha e_{ij}^T \Xi_0^{-\frac{1}{2}} (\Xi_0^{\frac{1}{2}} \Xi_t \Xi_0^{\frac{1}{2}})^{\frac{1}{2}} \Xi_0^{-\frac{1}{2}} e_{ij})}. \end{aligned} \quad (3.8)$$

To simplify Equation (3.8) we notice that Ξ_0 is diagonal and the term containing E_{ij} will only update the ij^{th} index along the diagonal, meaning Ξ_{t+1} will also be diagonal. The diagonality of Ξ_0 and Ξ_t means that matrix multiplication is commutative and the matrix square root can be distributed on its arguments, i.e.,

$(\Xi_0^{\frac{1}{2}} \Xi_t \Xi_0^{\frac{1}{2}})^{\frac{1}{2}} = \Xi_0^{\frac{1}{2}} \Xi_t^{\frac{1}{2}}$. Applying this to Equation (3.8) results in:

$$\begin{aligned} \Xi_0^{\frac{1}{2}} \Xi_{t+1}^{\frac{1}{2}} &= \Xi_0^{\frac{1}{2}} \Xi_t^{\frac{1}{2}} - \delta_{ij} \frac{\alpha \Xi_0^{\frac{1}{2}} \Xi_t^{\frac{1}{2}} \Xi_0^{-\frac{1}{2}} E_{ij} \Xi_0^{-\frac{1}{2}} \Xi_t^{\frac{1}{2}} \Xi_0^{\frac{1}{2}}}{(1 + \delta_{ij} \alpha e_{ij}^T \Xi_0^{-\frac{1}{2}} \Xi_t^{\frac{1}{2}} \Xi_0^{-\frac{1}{2}} e_{ij})} \\ \iff \Xi_0^{\frac{1}{2}} \Xi_{t+1}^{\frac{1}{2}} &= \Xi_0^{\frac{1}{2}} \Xi_t^{\frac{1}{2}} - \delta_{ij} \frac{\alpha E_{ij} \Xi_t}{(\gamma + \delta_{ij} \alpha e_{ij}^T \Xi_0^{-\frac{1}{2}} \Xi_t^{\frac{1}{2}} e_{ij})}. \end{aligned}$$

Notice that the $\alpha E_{ij} \Xi_t$ term will only have one entry - the ij^{th} diagonal term of Ξ_t , and the same is true for the division term. Therefore the update to Ξ will only occur at a single index, which we label ξ_{ij} . We express the update equation in terms of ξ_{ij} :

$$\xi_{0,ij}^{\frac{1}{2}} \xi_{t+1,ij}^{\frac{1}{2}} = \xi_{0,ij}^{\frac{1}{2}} \xi_{t,ij}^{\frac{1}{2}} - \frac{\delta_{ij} \alpha \xi_{t,ij}}{\gamma + \delta_{ij} \alpha \xi_{0,ij}^{\frac{1}{2}} \xi_{t,ij}^{\frac{1}{2}}}.$$

By solving this equation for $\xi_{t+1,ij}$ we have:

$$\xi_{t+1,ij} = \frac{\xi_{0,ij} \xi_{t,ij}}{(\xi_{0,ij}^{\frac{1}{2}} + \delta_{ij} \gamma^{-1} \xi_{t,ij}^{\frac{1}{2}} \alpha)^2}. \quad (3.9)$$

It is now apparent that only the diagonal terms of Ξ_t are needed in the steps for BWML. Accordingly, we express the margin variable as $\xi \in \mathbb{R}^m$ with the relationship $\xi_t = \text{diag}(\Xi_t)$.

The next task is to solve for projection parameter α such that M_{t+1} and Ξ_{t+1} are projected onto H_{ij} by substituting their expressions into the hyperplane constraint equation:

$$\begin{aligned} \langle X_{ij}, M_{t+1} \rangle - \langle E_{ij}, \Xi_{t+1} \rangle &= 0 \\ \iff \langle X_{ij}, M_0^{-\frac{1}{2}} Z_{t+1}^2 M_0^{-\frac{1}{2}} \rangle - \langle E_{ij}, \Xi_{t+1} \rangle &= 0 \\ \iff \langle X_{ij}, M_0^{-\frac{1}{2}} \left(Z_t + \delta_{ij} \frac{\alpha Z_t M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} Z_t}{1 - \delta_{ij} \alpha (x_i - x_j)^T M_0^{-\frac{1}{2}} Z_t M_0^{-\frac{1}{2}} (x_i - x_j)} \right)^2 M_0^{-\frac{1}{2}} \rangle \\ &\quad - \frac{\xi_{0,ij} \xi_{t,ij}}{(\xi_{0,ij}^{\frac{1}{2}} + \delta_{ij} \gamma^{-1} \xi_{t,ij}^{\frac{1}{2}} \alpha)^2} = 0. \end{aligned}$$

Solving this for α results in the quadratic equation:

$$(\xi_{0,ij}^{-1} \gamma^{-2} - q^2) \alpha^2 + \delta_{ij} 2(p \xi_{0,ij}^{-\frac{1}{2}} \xi_{t,ij}^{-\frac{1}{2}} \gamma^{-1} + q) \alpha + (p \xi_{t,ij}^{-1} - 1) = 0 \quad (3.10)$$

$$s.t. \quad -\delta_{ij} \gamma \xi_{0,ij}^{\frac{1}{2}} \xi_{t,ij}^{-\frac{1}{2}} \leq \delta_{ij} \alpha \leq \delta_{ij} \frac{1}{q}, \quad (3.11)$$

where $p = (x_i - x_j)^T M_0^{-\frac{1}{2}} Z_t^2 M_0^{-\frac{1}{2}} (x_i - x_j)$ and $q = (x_i - x_j)^T M_0^{-\frac{1}{2}} Z_t M_0^{-\frac{1}{2}} (x_i - x_j)$. Note that Equation (3.10) has associated bound constraints (3.11), as Equations (3.6) and (3.9) are not valid outside this range due to their division terms. Equation (3.10) is quadratic in α , so it can be solved for both potential solutions of α , with the invalid solution being excluded by the bound constraints (3.11). Once α has been computed the ij^{th} value of the correction vector $\lambda \in \mathbb{R}^m$ is compared to α to get the corrected projection parameter $\hat{\alpha}$:

$$\hat{\alpha} = \min\{\lambda_{ij}, \alpha\} \quad (3.12)$$

$$\lambda_{ij} = \lambda_{ij} - \alpha. \quad (3.13)$$

The updated values of M_{t+1} and ξ_{t+1} are calculated using the corrected projection parameter applied in Equations (3.4) and (3.5).

These updates are computed over each constraint in sequence until convergence is reached. Convergence is measured by monitoring the correction variable. If the successive difference in l_2 -norm after performing updates with respect to each constraint is sufficiently small then the algorithm has reached a desired optimality. These steps are summarised in Algorithm 8.

The convergence properties of Algorithm 8 on synthetic data is evaluated in section 3.4.1 as well as its performance on UCI and image datasets in sections 3.4.2 and 3.4.3.

Having presented an algorithm based on using the Bures-Wasserstein distance as a Bregman divergence, we now consider how to use the Bures-Wasserstein geometry of SPD matrices related to the Bures-Wasserstein distance to compute a Generalised Mahalanobis distance.

Algorithm 8: Bures-Wasserstein Metric Learning

Input: Dataset: $\{x_i \in \mathbb{R}^n\}_{i=1}^N$

Similarity and Dissimilarity indexes: $\mathcal{S}, \mathcal{D} \subseteq [N] \times [N]$

Initial Matrix: $M_0 \in \mathbb{R}^{n \times n}$, Initial Margins: $\xi_0 \in \mathbb{R}^{|\mathcal{S}|+|\mathcal{D}|}$, Regularisation

Parameter: $\gamma \in \mathbb{R}_+$, Convergence tolerance: $\epsilon \in \mathbb{R}_+$

1. $Z_0 \leftarrow M_0^{\frac{1}{2}}, \xi \leftarrow \xi_0, \lambda_0 \leftarrow 0$

2. **do**

(a) Sample $(i, j) \in \mathcal{S} \cup \mathcal{D}$ uniformly at random

(b) $\delta \leftarrow 1$ if $(i, j) \in \mathcal{S}$, -1 otherwise (to decide on the $\pm$ choices)

(c) $p \leftarrow (x_i - x_j)^T Z_t^2 (x_i - x_j)$

(d) $q \leftarrow (x_i - x_j)^T Z_t (x_i - x_j)$

(e) Solve Equation (3.10) for α

(f) $\alpha \leftarrow \min(\alpha, \lambda_{ij})$

(g) $\lambda_{t+1,ij} \leftarrow \lambda_{t,ij} - \alpha$

(h) set Z_{t+1}, ξ_{t+1} using Equations (3.6) and (3.9) respectively

(i) $t \leftarrow t + 1$

while $\|\lambda_t - \lambda_{t+1}\| \geq \epsilon$

3.2.2 Bures-Wasserstein Barycentric Metric Learning

As an alternative to Cyclic Projections, we may take advantage of the geometry of the Bures-Wasserstein manifold to solve for our desired metric. This type of approach has seen extensive research in the context of $Cat(\kappa)$ and Hadamard spaces [140, 141]. To this end the Metric Learning problem is expressed in a slightly different manner. We will refer to a similarity constraint set $\mathcal{S}_{ij} = \{M \in S_{++}^n : (x_i - x_j)^T M (x_i - x_j) \leq u\}$ and for dissimilarity $\mathcal{D}_{ij} = \{M \in S_{++}^n : (x_i - x_j)^T M (x_i - x_j) \geq l\}$. These sets are generically referred to as $\mathcal{H}_{ij}$ since they are half-spaces, distinct from H_{ij} , which is a hyperplane. Again, for brevity we write the number of constraints as $m = |\mathcal{S}| + |\mathcal{D}|$. The subproblem that we can solve efficiently is:

$$\min_{M \in \mathcal{H}_{ij}} \mathcal{B}^2(M, M_0), \quad (3.14)$$

which is referred to as $\mathcal{B}^2(M_0, \mathcal{H}_{ij})$. This function can be interpreted as the distance from M_0 to the closest element of $\mathcal{H}_{ij}$.

We define the function:

$$f(M) = \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} \mathcal{B}^2(M, \mathcal{H}_{ij}).$$

The minimum value of f is zero and is attained when $M \in \bigcap_{(i,j) \in \mathcal{S} \cup \mathcal{D}} \mathcal{H}_{ij}$ (assuming $\bigcap_{(i,j) \in \mathcal{S} \cup \mathcal{D}} \mathcal{H}_{ij} \neq \emptyset$) as f is the sum of distances to each $\mathcal{H}_{ij}$ and if M is in their intersection then the distance to each set is zero. Therefore, feasible metrics are minimisers of this function, so by solving:

$$\min_{M \in S_{++}^n} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} \mathcal{B}^2(M, \mathcal{H}_{ij}), \quad (3.15)$$

we can find a desirable metric. The inner problem of (3.15) is subproblem (3.14) and the sum can then be seen as a barycentre of these projections, as in the Bures-Wasserstein barycentre in problem (2.15). Therefore each subproblem can be solved with respect to each $\mathcal{H}_{ij}$ and then the barycentre of these projections is calculated

for the next iterate. We refer to this approach as Bures-Wasserstein Barycentric Metric Learning (BWBML). It is interesting to note that while Problem (3.15) is non-convex because the projection will certainly depend on the initial choice of M . However, this non-convexity reflects that an initial M_0 must be selected for projection, which makes the nature of this choice more transparent.

To compute the barycentre we choose to use the barycentre algorithms presented by *Chewi et al.* [115] as these algorithms admit simple update equations with respect to the transport matrix of each subproblem. *Chewi et al.* present Gradient Descent and Stochastic Gradient Descent versions of their algorithm, which we use to implement two different version of BWBML. To differentiate between the two versions of this algorithm we say the stochastic version is sequential and the non-stochastic version is averaged i.e. Sequential-BWBML (S-BWBML) and Averaged-BWBML (A-BWBML).

To make these algorithms applicable to Metric Learning, variable margins need to be included in the optimisation. The problem $\mathcal{B}^2(M, \mathcal{H}_{ij})$ becomes:

$$\min_{(M, \xi) \in \mathcal{H}_{ij}} \mathcal{B}^2(M, M_0) + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0)) \quad (3.16)$$

which is expressed as $\mathcal{B}^2(M_0, \xi_0, \mathcal{H}_{ij}, \gamma)$, where the half-spaces now include the variable margins. The corresponding Metric Learning problem is then:

$$\min_{M \in S_{++}^n, \xi \in \mathbb{R}_{++}^m} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} \mathcal{B}^2(M, \xi, \mathcal{H}_{ij}, \gamma).$$

If $(M_t, \xi_t) \in \mathcal{H}_{ij}$ then the solution is simply $M = M_t$ and $\xi = \xi_t$. Otherwise, the ij -th subproblem is solved using its relevant KKT-conditions (equations (2.10a)-(2.10e)). To do this we first define the Lagrangian:

$$L_{ij}(M, \xi, \lambda) = \mathcal{B}^2(M, M_t) + \gamma \mathcal{B}^2(\text{diag}(\xi_t), \text{diag}(\xi)) + \delta_{ij} \lambda_{ij} (x_{ij}^T M_t x_{ij} - \xi_{t,ij}),$$

where $\delta_{ij} = 1$ if $(i, j) \in \mathcal{S}$ or -1 otherwise and $x_{ij} = x_i - x_j$ is written for brevity. Then solving $\nabla_M L_{ij} = 0$ for M :

$$\begin{aligned}
 \nabla_M L_{ij} &= 0 \\
 \iff I - M_t^{\frac{1}{2}} (M_t^{\frac{1}{2}} M M_t^{\frac{1}{2}})^{-\frac{1}{2}} M_t^{\frac{1}{2}} + \delta_{ij} \lambda_{ij} x_{ij} x_{ij}^T &= 0 \\
 \iff (M_t^{\frac{1}{2}} M M_t^{\frac{1}{2}})^{-\frac{1}{2}} &= M_t^{-\frac{1}{2}} (I - \delta_{ij} \lambda_{ij} x_{ij} x_{ij}^T) M_t^{-\frac{1}{2}} \\
 \iff (M_t^{\frac{1}{2}} M M_t^{\frac{1}{2}})^{\frac{1}{2}} &= M_t^{\frac{1}{2}} \left(I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}} \right) M_t^{\frac{1}{2}} \\
 \iff M &= \left(I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}} \right) M_t \left(I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}} \right),
 \end{aligned}$$

where the Sherman-Morrison identity was applied at the fourth step. It is apparent that the update to M_t takes the form of a transport matrix T_{ij} being multiplied on both sides.

$\nabla_\xi L_{ij} = 0$ has two cases of interest. The first is $\frac{\partial}{\partial \xi_{kl}} L_{ij} = 0, kl \neq ij$. In this case there is no update to ξ as the derivative with respect to the constraint is zero. For $kl = ij$ then we find the update for index ξ_{ij} .

$$\begin{aligned}
 \frac{\partial}{\partial \xi_{ij}} L_{ij} &= 0 \\
 \iff 1 - \sqrt{\xi_{t,ij}} / \sqrt{\xi_{ij}} - \delta_{ij} \lambda_{ij} &= 0 \\
 \iff \xi_{ij} &= \xi_{t,ij} / (1 - \delta_{ij} \gamma^{-1} \lambda_{ij})^2.
 \end{aligned}$$

Substituting these expressions for M and ξ_{ij} into constraint $x_{ij}^T M x_{ij} - \xi_{ij} = 0$ and solving for λ_{ij} gives us the expression:

$$\lambda_{ij} = \delta_{ij} \left(\frac{1}{\sqrt{\xi_{t,ij}}} - \frac{1}{\sqrt{x_{ij}^T M_t x_{ij}}} \right) \left(\frac{x_{ij}^T x_{ij}}{\sqrt{x_{ij}^T M_t x_{ij}}} - \frac{1}{\sqrt{\xi_{t,ij}}} \right)^{-1}, \quad (3.17)$$

which is the required projection parameter to solve subproblem (3.16).

We now describe the process for computing the barycentres of the projections using the solutions of subproblems (3.16). For the averaged method, the transport matrices T_{ij} are averaged as $T = \frac{1}{m} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} T_{ij}$ and then this averaged transport matrix is applied to the current iterate M_t . It is fortunate that the expression of the updates for each subproblem naturally admits the transport matrix as this avoids the more costly barycentre computation. The update equation takes the form:

$$M_{t+1} = \left(\frac{1}{m} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} x_{ij}^T x_{ij}} \right) M_t \left(\frac{1}{m} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} x_{ij}^T x_{ij}} \right),$$

where δ is the vectorised form of δ_{ij} .

The update to ξ is simply the individual updates applied to each index. We can re-write this in a vectorised form as:

$$\xi_{t+1} = \xi_t \oslash (1 - \gamma^{-1} \delta \odot \lambda)^2.$$

To determine convergence we compare the l_2 norm between the next iterate and the current iterate for both variables and see if it is sufficiently small. This is summarised in Algorithm 9.

In the case of the sequential method the transport matrices are applied in sequence like Cyclic Projections. However, the midpoint of transport matrix T_{ij} and the identity matrix with respect to an interpolation parameter η is computed and then immediately used to transport M_t . The transport matrices T_{ij} of each subproblem are computed as before. The ij -th update for the sequential method is:

$$M_{t+1} = \left((1 - \eta)I - \eta \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}} \right) M_t \left((1 - \eta)I - \eta \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}} \right)$$

Similarly, for ξ the midpoint between the updated ξ_{ij} and the current $\xi_{t,ij}$ is the new value $\xi_{t+1,ij}$, that is:

$$\xi_{t+1,ij} = \xi_{t,ij} / ((1 - \eta) - \eta \delta_{ij} \gamma^{-1} \lambda_{ij})^2.$$

Algorithm 9: Averaged Bures-Wasserstein Barycentric Metric Learning

Input: Dataset: $\{x_i \in \mathbb{R}^n\}_{i=1}^N$

Similarity and Dissimilarity indexes: $\mathcal{S}, \mathcal{D} \subseteq [N] \times [N]$

Initial Matrix: $M_0 \in \mathbb{R}^{n \times n}$, Initial Margins: $\xi_0 \in \mathbb{R}^{|\mathcal{S}|+|\mathcal{D}|}$, Regularization

Parameter: $\gamma \in \mathbb{R}_+$, Convergence term: $\epsilon \in \mathbb{R}_+$

1. $M \leftarrow M_0, \xi \leftarrow \xi_0, \lambda_0 \leftarrow 0$

2. **do**

(a) $M' \leftarrow M, \xi' \leftarrow \xi$

(b) $T \leftarrow 0_{n \times n}$

(c) **for** $(i, j) \in \mathcal{S} \cup \mathcal{D}$

i. Solve Equation (3.17) for λ_{ij}

ii. $T \leftarrow T + I - \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}}$

(d) $M \leftarrow \frac{1}{m^2} T M T$

(e) $\xi \leftarrow \xi_t \oslash (1 - \gamma^{-1} \delta \odot \lambda)^2$

while $\mathcal{B}(M, M') + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi')) \geq \epsilon$

Convergence is determined similarly to A-BWBML except that the iterates are compared between sweeps of each constraint projection being sequentially performed. This full algorithm is summarised in Algorithm 10.

It does appear that A-BWBML and S-BWBML are variations of Averaged Pro-

Algorithm 10: Sequential Bures-Wasserstein Barycentric Metric Learning

Input: Dataset: $\{x_i \in \mathbb{R}^n\}_{i=1}^N$

Similarity and Dissimilarity indexes: $\mathcal{S}, \mathcal{D} \subseteq [N] \times [N]$

Initial Matrix: $M_0 \in \mathbb{R}^{n \times n}$, Initial Margins: $\xi_0 \in \mathbb{R}^{|\mathcal{S}|+|\mathcal{D}|}$, Regularization

Parameter: $\gamma \in \mathbb{R}_+$, Step-size: $\eta \in [0, 1]$, Convergence term: $\epsilon \in \mathbb{R}_+$

1. $M \leftarrow M_0, \xi \leftarrow \xi_0, \lambda_0 \leftarrow 0$

2. **do**

(a) $M' \leftarrow M, \xi' \leftarrow \xi$

(b) **for** $(i, j) \in \mathcal{S} \cup \mathcal{D}$

i. Solve Equation (3.17) for λ_{ij}

ii. $T \leftarrow (1 - \eta)I - \eta \delta_{ij} \frac{\lambda_{ij} x_{ij} x_{ij}^T}{1 + \delta_{ij} \lambda_{ij} x_{ij}^T x_{ij}}$

iii. $M \leftarrow TMT$

iv. $\xi_{ij} \leftarrow \xi_{ij} / ((1 - \eta) - \eta \delta_{ij} \gamma^{-1} \lambda_{ij})^2$

while $\mathcal{B}(M, M') + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi')) \geq \epsilon$

jections and Alternating Projections [99] in Bures-Wasserstein space. There are analogous convergence results for algorithms similar to ours, but in different matrix manifolds [142], to our knowledge there are no convergence results in the case of the Bures-Wasserstein manifold. There appears to be some preliminary work in this field on proximal point algorithms in Wasserstein space [143]. However there is a significant gap between that work and proving convergence of algorithms like BWBML. We do not provide such a result as it is outside of the scope of our

work, which focuses primarily on practical design of algorithms, not theoretical aspects of optimisation. Additionally, since we know that BWML converges, we can compare outputs of these algorithms to the solutions generated by BWBML, which is considered in section 3.4.1. The classification performance of A-BWBML and S-BWBML are also evaluated on UCI and image datasets in sections 3.4.2 and 3.4.3.

We now turn away from projection-based algorithms in favour of gradient-based methods. We develop an alternative formulation of the Metric Learning problem for this context and discuss some algorithms for solving it.

3.2.3 Transport Bures-Wasserstein Metric Learning

Another approach to solving the Bures-Wasserstein Metric Learning problem is to use gradient-based methods. Similar approaches have been applied to semidefinite least-squares problems [144, 145, 146]. The gradient of the Bures-Wasserstein distance contains the inverse of a matrix square root, which is not amenable to an efficient gradient method. However we can reformulate the objective by estimating the transport matrix directly, rather than implicitly, by finding the optimal SPD matrix. To do this, the substitution $M = TM_0T$ where $T \in S_{++}^n$ is made, which results in the Bures-Wasserstein distance expressed as:

$$\begin{aligned} \mathcal{B}^2(M_0, TM_0T) &= \text{trace}(M_0 + TM_0T - 2(M_0^{\frac{1}{2}}TMTM_0^{\frac{1}{2}})^{\frac{1}{2}}) \\ &= \text{trace}(M_0 + TM_0T - 2M_0^{\frac{1}{2}}TM_0^{\frac{1}{2}}) \\ &= \|M_0^{\frac{1}{2}}(T - I)\|_F^2. \end{aligned}$$

This expression retains its convexity and has a simple-to-compute derivative that is more amenable to gradient based methods. Using this we express the Metric

Learning problem as:

$$\begin{aligned} \min_{T \in S_{++}^n} & \|M_0^{\frac{1}{2}}(T - I)\|_F^2 \\ \text{s.t.} & (x_i - x_j)^T T M_0 T (x_i - x_j) \leq u, (i, j) \in \mathcal{S} \\ & (x_i - x_j)^T T M_0 T (x_i - x_j) \geq l, (i, j) \in \mathcal{D}, \end{aligned}$$

where other terms are defined as in problem (3.2). To simplify the notation, $(x_i - x_j)^T T M_0 T (x_i - x_j)$ is expressed as $\text{trace}(T M_0 T X_{ij})$ where $X_{ij} = (x_i - x_j)(x_i - x_j)^T$. For the similarity constraints, a linear function $X_{\mathcal{S}} : S_+^n \rightarrow \mathbb{R}_+^{|\mathcal{S}|}$ is defined such that $(X_{\mathcal{S}} M)_{ij} = \text{trace}(X_{ij} M)$. Similarly, $X_{\mathcal{D}}$ is defined in terms of the dissimilarity constraints. The Lagrangian of this expression is:

$$\begin{aligned} L(T, \lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}) = & \|M_0^{\frac{1}{2}}(T - I)\|_F^2 \\ & + \lambda_{\mathcal{S}}^T (\text{trace}(X_{\mathcal{S}} T M_0 T) - l) + \lambda_{\mathcal{D}}^T (\text{trace}(u - X_{\mathcal{D}} T M_0 T)), \end{aligned}$$

with $\lambda_{\mathcal{S}} \in \mathbb{R}_+^{|\mathcal{S}|}$ and $\lambda_{\mathcal{D}} \in \mathbb{R}_+^{|\mathcal{D}|}$ being the similarity and dissimilarity dual variables respectively.

Solutions to this problem can be found by solving:

$$\max_{\lambda_{\mathcal{S}} \in \mathbb{R}_+^{|\mathcal{S}|}, \lambda_{\mathcal{D}} \in \mathbb{R}_+^{|\mathcal{D}|}} \min_{T \in S_{++}^n} L(T, \lambda_{\mathcal{S}}, \lambda_{\mathcal{D}})$$

It is costly to maintain both the positive semidefinite condition on T and the dual variables $\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}$ during optimisation. It is preferable to use only the dual variables during optimisation. Using weak duality we find the dual function (as in expression (2.9)), which is denoted as $\phi(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}})$. To find ϕ , we differentiate L with respect to T and set equal to zero which leads to:

$$-T(I + X_{\mathcal{S}}^T \lambda_{\mathcal{S}} - X_{\mathcal{D}}^T \lambda_{\mathcal{D}})M_0^{-1} - M_0^{-1}(I + X_{\mathcal{S}}^T \lambda_{\mathcal{S}} - X_{\mathcal{D}}^T \lambda_{\mathcal{D}})T = 2M_0^{-1}.$$

This is a Lyapunov equation, the solution of which is referred to as $T(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}})$.

One condition for the solution of the Lyapunov equation is that $I + X_{\mathcal{S}}^T \lambda_{\mathcal{S}} - X_{\mathcal{D}}^T \lambda_{\mathcal{D}} \succeq$

0, which we assume for now. However, this condition will be explicitly enforced during optimisation. We can guarantee that there will be solutions to the Lyapunov equation:

Lemma 3.2.4 *The Lyapunov equation*

$$-T(I + X_S^T \lambda_S - X_D^T \lambda_D)M_0^{-1} - M_0^{-1}(I + X_S^T \lambda_S - X_D^T \lambda_D)T = 2M_0^{-1}$$

possesses a solution $T(\lambda) \in S_{++}^n$ for $\lambda \in \mathbb{R}_+^n$

Proof: For the equation to have a solution, the matrix $M_0^{-1}(I + X_S^T \lambda_S - X_D^T \lambda_D)$ has eigenvalues such that $\lambda_i + \lambda_j \neq 0$ [147]. As assumed, $M_0^{-1}, I + X_S^T \lambda_S - X_D^T \lambda_D \succeq 0$, then by Corollary 7.6.2 b) of Horn and Johnson [147] their product has positive eigenvalues as required. ■

Therefore $\phi(\lambda_S, \lambda_D) = L(T(\lambda_S, \lambda_D), \lambda_S, \lambda_D)$. Using this we can solve:

$$\begin{aligned} & \max_{\lambda_S, \lambda_D} \phi(\lambda_S, \lambda_D) \\ & \text{s.t. } I + X_S^T \lambda_S - X_D^T \lambda_D \succeq 0 \\ & \lambda_S, \lambda_D \geq 0 \end{aligned}$$

to find our desired metric.

As with the previous methods we add variable margins to this problem and the margin regulariser uses the Bures-Wasserstein distance. This is expressed as:

$$\begin{aligned} & \min_{T \in S_{++}^n, \xi \in \mathbb{R}_{++}^m} \|M_0^{\frac{1}{2}}(T - I)\|_F^2 + \gamma \mathcal{B}^2(\text{diag}(\xi_0), \text{diag}(\xi)) \\ & \text{s.t. } (x_i - x_j)^T T M_0 T (x_i - x_j) \leq \xi_{ij}, (i, j) \in \mathcal{S} \\ & (x_i - x_j)^T T M_0 T (x_i - x_j) \geq \xi_{ij}, (i, j) \in \mathcal{D}. \end{aligned}$$

By constructing a Lagrangian and solving for $\phi(\lambda_S, \lambda_D)$, T has the same Lyapunov equation expression as before and ξ has the explicit equation:

$$\begin{aligned} \xi_S(\lambda_S, \gamma) &= \gamma^2 \xi_0 \oslash (\gamma + \lambda_S)^2 \\ \xi_D(\lambda_D, \gamma) &= \gamma^2 \xi_0 \oslash (\gamma - \lambda_D)^2, \end{aligned}$$

with the associated constraint $\gamma - \lambda_{\mathcal{D}} \geq 0$. $\xi(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}, \gamma)$ is the concatenation of $\xi_{\mathcal{S}}$ and $\xi_{\mathcal{D}}$. Substituting these into L as before gives:

$$\phi(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}) = L(T(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}), \xi(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}, \gamma), \lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}).$$

Using this we can solve the problem:

$$\begin{aligned} \max_{\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}} & \phi(\lambda_{\mathcal{S}}, \lambda_{\mathcal{D}}) \\ \text{s.t.} & I + X_{\mathcal{S}}^T \lambda_{\mathcal{S}} - X_{\mathcal{D}}^T \lambda_{\mathcal{D}} \succeq 0 \\ & \gamma - \lambda_{\mathcal{D}} \geq 0 \\ & \lambda_{\mathcal{S}}, \lambda_{\mathcal{D}} \geq 0. \end{aligned} \tag{3.18}$$

This problem is convex by weak duality, but the primal problem is not convex due to the dissimilarity constraints, so there may be a duality gap associated with the solution. If we omit the dissimilarity constraints from the optimisation this will give strong duality and also makes the semidefinite constraint trivially satisfied. This means the problem becomes:

$$\begin{aligned} \max_{\lambda_{\mathcal{S}}} & \phi(\lambda_{\mathcal{S}}) \\ \text{s.t.} & \lambda_{\mathcal{S}} \geq 0 \end{aligned} \tag{3.19}$$

At this point we choose two different algorithms to solve Problems (3.18) and (3.19). We are interested in seeing how the choice of reducing complexity in the optimisation problem, but also reducing the expressivity of the constraints, will effect the performance of the algorithms and metrics.

To solve Problem (3.19) we use L-BFGS-B [148]. This algorithm uses a Quasi-Newton approximation to the Hessian to obtain super-linear convergence at a lower complexity. L-BFGS-B requires the gradient of the objective $g(\lambda_{\mathcal{S}})$, which is:

$$X_{\mathcal{S}} T(\lambda_{\mathcal{S}}) M_0 T(\lambda_{\mathcal{S}}) - \xi_{\mathcal{S}}(\lambda_{\mathcal{S}})$$

To solve Problem (3.18) we use PENLAB [149] an open-source solver for solving non-linear semidefinite programming problems. In addition to requiring a gradient, PENLAB uses the Hessian of the objective during optimisation. The Hessian can be calculated by:

$$\begin{aligned} & \frac{\partial^2}{\partial_{ij}\partial_{lk}}[g(\lambda_S, \lambda_D)] \\ &= \frac{\partial}{\partial_{lk}}[\text{trace}(X_{ij}T(\lambda)M_0T(\lambda)) + \delta_{ij} \odot \xi_{ij}(\lambda_{ij})] \\ &= \text{trace}(X_{ij}(T(\lambda)M_0\left[\frac{\partial}{\partial_{lk}}T(\lambda)\right] + \left[\frac{\partial}{\partial_{lk}}T(\lambda)\right]M_0T(\lambda))) + \delta_{ij} \odot \left[\frac{\partial}{\partial_{lk}}\xi_{ij}(\lambda_{ij})\right] \end{aligned}$$

where the derivative of $T(\lambda)$ is the solution to the Lyapunov equation:

$$\begin{aligned} & \left[\frac{\partial}{\partial_{lk}}T(\lambda)\right](I + X_S^T\lambda_S - X_D^T\lambda_D)M_0^{-1} - M_0^{-1}(I + X_S^T\lambda_S - X_D^T\lambda_D)\left[\frac{\partial}{\partial_{lk}}T(\lambda)\right] \\ &= T(\lambda)X_{lk}M_0^{-1} + M_0^{-1}X_{lk}T(\lambda) \end{aligned}$$

and the derivative of the margin term is:

$$\left[\frac{\partial}{\partial_{lk}}\xi_{ij}(\lambda_{ij})\right] = \begin{cases} \delta_{lk}\gamma^{-1}\xi_{0,lk}/(1 + \gamma^{-1}\delta_{lk}\xi_{lk})^3, & \text{if } ij = lk \\ 0 & \text{otherwise} \end{cases}$$

Here ξ_{ij} indexes the margin associated with $(i, j) \in \mathcal{S} \cup \mathcal{D}$ and $\delta_{ij} = 1$ if $(i, j) \in \mathcal{S}$ or -1 otherwise, similar to the previous algorithms.

We denote Problem (3.18) being solved by PENLAB as T-BWML. Problem (3.19) being solved by L-BFGS-B is then denoted as T-BWML-S to indicate that in this case only similarity constraints are used.

Similarly to the case for BWBML, there are no prior convergence theorems for PENLAB or L-BFGS-B upon which to build for T-BWML or T-BWML-S. For L-BFGS, there exist global convergence results [97] that only require that the optimisation problem is convex, which is true for T-BWML-S. We empirically examine convergence in our algorithms in section 3.4.1 to see if they converge and how

well they do so, as well as evaluating their classification performance on UCI and image datasets in sections 3.4.2 and 3.4.3.

3.3 Implementation

For the experiments that follow, BWML, A-BWBML and BWBML-S were implemented in MATLAB using standard libraries. Implementations are available at <https://gitlab.ecs.vuw.ac.nz/cooperdavi1/thesis-implementations>. Our implementations of T-BWML and T-BWML-S are also available at the same url. However T-BWML uses the PENLAB solver <https://web.mat.bham.ac.uk/kocvara/penlab/> and similarly T-BWML-S uses a L-BFGS-B <https://github.com/stephenbeckr/L-BFGS-B-C> solver.

The implementation of ITML we used is available at <https://www.cs.utexas.edu/users/pjain/itml/download/itml-1.2.tar.gz> and for LMNN we used the implementation available at <https://www.cs.cornell.edu/~kilian/code/lmnn/lmnn.html>.

To perform experiments, our MATLAB programs were executed in MATLAB 2022a under a grid computing environment <http://star.mit.edu/cluster/docs/0.93.3/guides/sge.html> where each machine was either an Intel Core i7 6700 @ 3.4 GHz or an Intel Core i7 9700 @ 3.0 GHz.

3.4 Experiments

Results of the five algorithms described in the methods section on several kinds of Metric Learning problems are now presented.

The first set of experiments conducted are on synthetic data. We treat this problem solely as a convex feasibility problem, i.e., finding the closest PSD matrix in

a feasible set. Instead of using the Generalised Mahalanobis distance of two features $\|x_i - x_j\|_M$ as the constraint, we use random vectors $\|a_i\|_M$. Each $a_i \in \mathbb{R}^n$ is sampled from $\mathcal{N}(0, I)$ and initialised with $\xi_0 = \mathbb{1}_m$ and $M_0 = I$. Constraints in this test are treated as similarity constraints, as a test with a mixture of similarity and dissimilarity constraints would be much more difficult to interpret. The intention with this set of experiments is to gain an understanding of the convergence and timing behaviour of each of the proposed algorithms. By generating the problems ourselves we can select the parameters of the problem data to test how each algorithm performs under different conditions. The parameters of interest are: the number of constraints m in each problem, the dimensionality n of the problem and the regularisation parameter γ . ITML [12] is included in these experiments as it uses Cyclic Projections, so it provides a relevant comparison for convergence.

We then learn metrics for k -Nearest Neighbour classification on several datasets. The performance metrics of interest here are the classification error and fit time for each algorithm. $k = 4$ is used for all experiments as it is a typical value in Metric Learning experiments, and we also provide some empirical evidence for this choice. The datasets are separated into small-scale datasets obtained from the UCI dataset repository [150] and image datasets MNIST and Fashion-MNIST. Again, ITML is included in these experiments, as well as LMNN [14].

For the classification tasks the initial margin values provided to ITML and our algorithms are selected in the same way, i.e., the 5th and 95th percentile distance are selected from a random subset of the dataset presented. Similar to the synthetic tests our algorithms and ITML require an initial metric, so they are provided with $M_0 = I$, the Euclidean metric.

3.4.1 Synthetic Data

For these tests, the algorithms BWML, A-BWBML, S-BWBML, T-BWML-S, T-BWML and ITML project an initial matrix $M_0 = I$ onto the set of ellipses inside a randomly generated polytope. As previously stated, hyperplane constraints are gen-

erated using a unit normal distribution with initial margins ξ_0 being equal to one. For S-BWML the smoothing parameter η was set equal to 0.5 as a representative value. The effects of dimensionality of the data, number of half-plane constraints and the regularisation parameter γ are of interest in these tests, hence we independently vary each one of these parameters. For each combination of dimensionality, number of constraints and γ presented the tests were run with 100 different realisations of the stochastic parameters a_i . We present convergence figures and timing performance figures for these tests. The convergence for each algorithm is quantified as the suboptimality of the objective function at the current iterate: $|f(M_i, \xi_i) - f(M^*, \xi^*)|$ where M^*, ξ^* is the solution calculated by the algorithm for that particular problem. For each series in the convergence figures, the median suboptimality during optimisation with respect to the solution is shown and the 25% lower quantile and 75% upper quantile with respect to the median are presented. We do this so that the differences shown between the algorithms in convergence are not just statistical aberrations. In the timing figures the average time to compute the solution with a single standard deviation in error bars are shown. Outliers for the timing information are shown as an 'x'. The outliers are detected using the MATLAB `rmoutliers` function. Similarly, the error bars give us confidence in the conclusions we draw. In some cases the error subtracted from the mean is a negative quantity which cannot be represented in a logarithmic scale. In this case the lower error bar is omitted. Both convergence and timing results are presented on logarithmic scales as it displays the trends the most clearly.

When selecting the dimensionalities and number of constraints for these tests we had an interest in making the complexity of the problems large enough to capture useful performance characteristics of the algorithms while avoiding tests taking an unnecessary amount of time to run. We decided that dimensionalities and number of constraints between 10 and 100 were appropriate. The values of the regularisation parameter γ are between 10^{-3} and 10^3 as this provides a large enough range to understand the algorithm's performance at a variety of magnitudes.

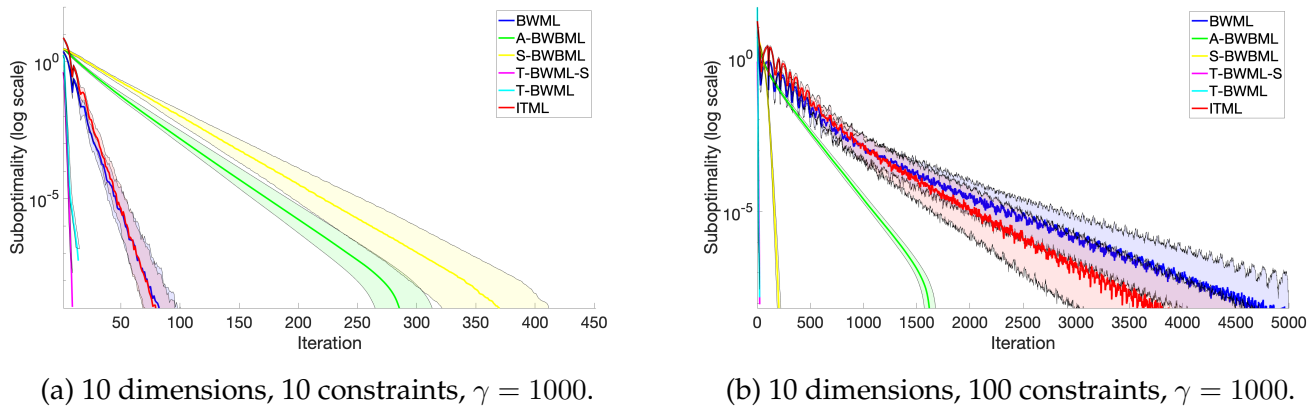


Figure 3.1: Effect of number of constraints on convergence for synthetic problems.

Effect of Number of Constraints

In Figure 3.1 the median suboptimality with quantile bounds of each algorithm during execution is shown for tests on the number of constraints. Generally speaking, as the number of constraints increases the rate of convergence for all algorithms decreases. This is not surprising, as the number of constraints corresponds to an increase in problem complexity. However, the change in the constraints affects some of the algorithms more than others. In both cases the T-BWML algorithms converge in a small number of iterations due to their second-order optimisation methods. For the smaller number of constraints BWML and ITML converge faster than the barycentric methods, while for a larger number of constraints the barycentric methods converge in fewer iterations. For the larger number of constraints, in A-BWBML the number of iterations to convergence increases 4 fold, whereas for BWML and ITML it increases by two orders of magnitude. More noticeably for S-BWBML, the number of iterations seems to be about the same in both tests. BWML and ITML use Cyclic Projections whereas A-BWBML and S-BWBML use similar projections combined with barycentric averages. The averaging steps act as a smoothing step between the iterations, meaning that the sequential iterations interfere with each other less. With a larger number of constraints, the effect of smoothing becomes more apparent. Additionally, this effect

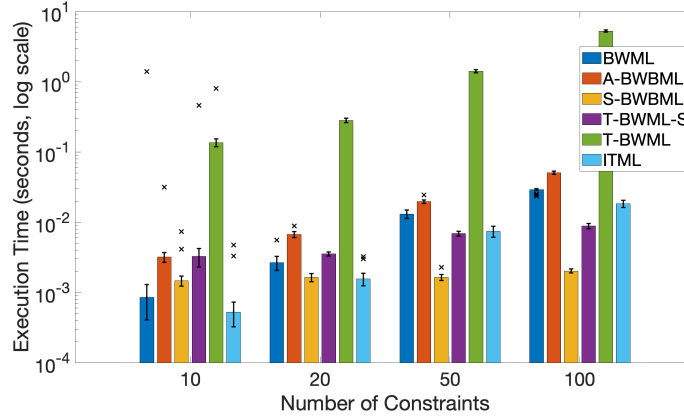


Figure 3.2: Effect of number of constraints on timing for synthetic problems in 10 dimensions and $\gamma = 1000$.

is apparent in the presence of oscillations in suboptimality of BWML and ITML during optimisation, but not A-BWBML and S-BWBML.

Figure 3.2 indicates that S-BWBML scales the best with respect to the number of constraints, where its execution time remains largely unchanged. Similarly, the execution time of T-BWML-S increases marginally with number of constraints. BWML, ITML and A-BWBML all increase in time required with respect to constraints, but A-BWBML seems slightly less sensitive to this, as the difference between these algorithms decreases with more constraints. T-BWML is by far the slowest algorithm and does not possess any beneficial scaling with size of constraint.

Effect of Dimensionality

In order to understand the effect of dimensionality on convergence and timing performance of these algorithms, we wish to test problems that are analogous to each other, but have different dimensions. To do this, we set the number of dimensions to be equal to the number of constraints, and vary both. Due to the method used to sample the constraints they are unlikely to be collinear, which means the

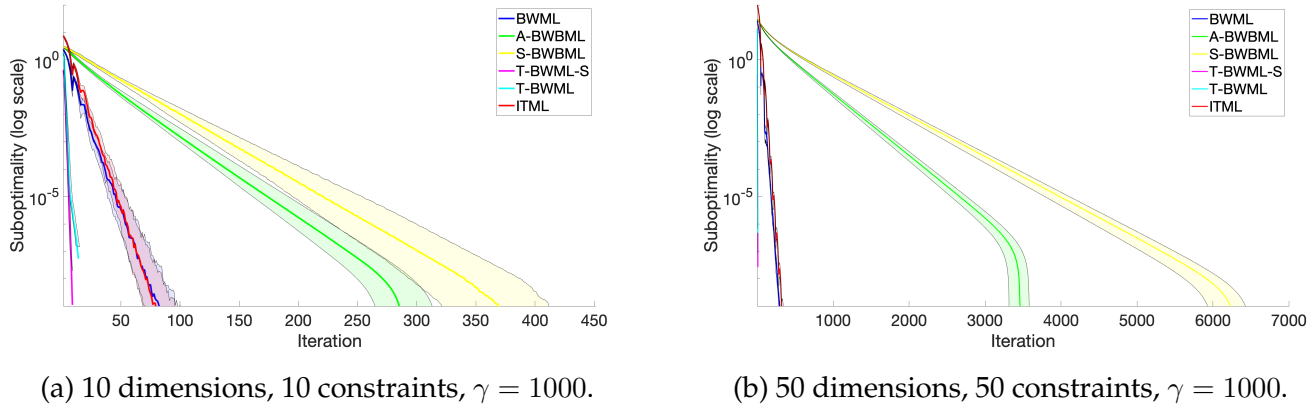


Figure 3.3: Effect of dimensionality on convergence for synthetic problems.

reported number of constraints should be the active number of constraints.

Figure 3.3 shows the affect of increased dimensionality on the convergence for the algorithms. The T-BWML algorithms again converge in a small number of iterations compared to the other algorithms, owing to their second-order gradient methods. The increased dimensionality affects the number of iterations of the barycentric methods more than the other methods. BWML and ITML appear to take less than 100 iterations in 10 dimensions and somewhere on the order of 200 iterations in 50 dimensions. However, both the BWBML algorithms increase by at least an order of magnitude in iterations between the two dimensionalities. This difference in the increase of convergence time may be related to the difference in the solution that the algorithms respectively compute. BWML, T-BWML-S and ITML compute the 'closest' ellipsoid to the initial ellipsoid with respect to their objective. In contrast the barycentric methods do not have such a guarentee, and simply calculate an ellipsoid that is inside the polytope by starting at the initial matrix. Therefore the barycentric methods are initially further from their solution compared to the other methods and are computing a kind of oblique projection. Owing to the curse of dimensionality, this effect will become more prevalent at higher dimensions and may increase the number of iterations and reduce the rate

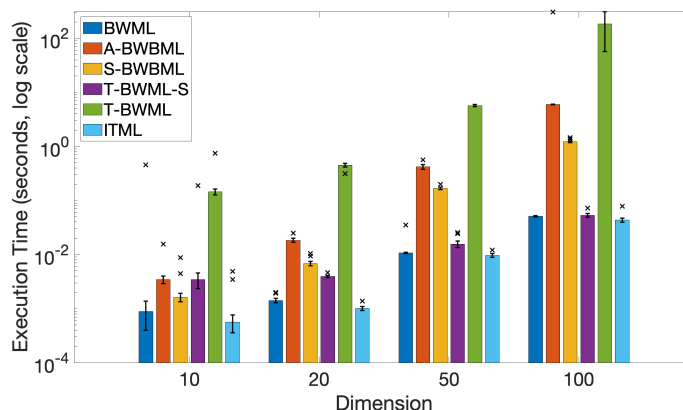


Figure 3.4: Effect of dimensionality on timing for synthetic problems.

of convergence.

In Figure 3.4 similar effects are apparent. The increase in dimensionality affects the execution time in the barycentric methods more than it does the others, which appear to rise faster than BWML or ITML. It is interesting to see that as the dimensionality increases the margin between BWML and ITML seems to decrease. T-BWML-S seems to be less sensitive to the increase in dimensionality, having essentially the same mean execution time in 100 dimensions as BWML and ITML. T-BWML seems to scale similarly to the barycentric methods, but has a much higher initial execution time.

Effect of Regularisation Parameter

We now consider the effect of the regularisation parameter γ on the convergence and timing of these algorithms. When $\gamma \rightarrow 0$ only the margin parameters are optimised and, conversely, when $\gamma \rightarrow \infty$ only the metric is optimised. The constraints in the margin terms are linearly independent, which in principle means that the problem can be solved in a coordinate-based manner.

In Figure 3.5 the convergence in 20 dimensions with 20 constraints is shown for

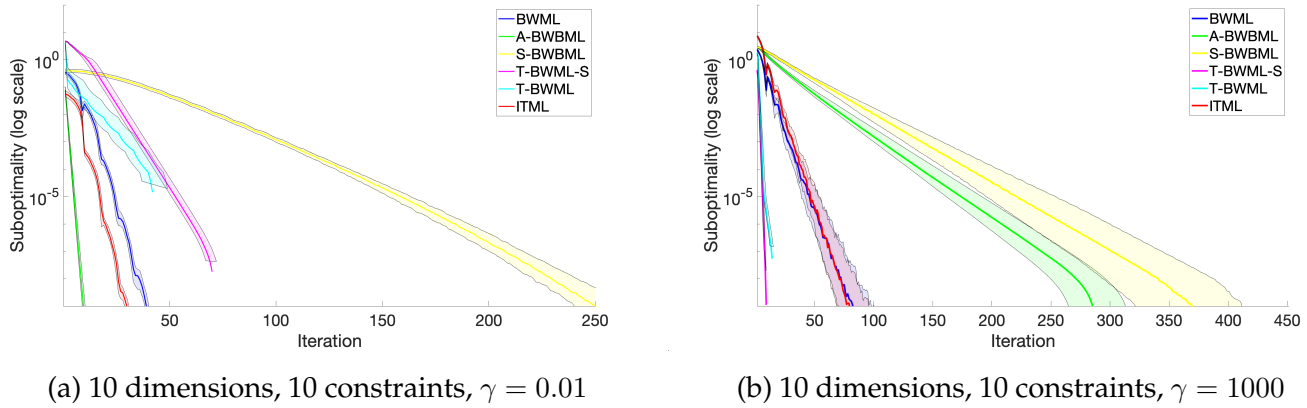


Figure 3.5: Effect of regularisation on convergence for synthetic problems.

two values of γ in each algorithm. As expected, when the value of the regularisation parameter is reduced the algorithms converge faster. For each algorithm they appear to converge in at least half the iterations with a smaller regularisation parameter. It is also worth noting that A-BWBML appears to utilise this advantage much more than the other algorithms. A-BWBML appears to converge in less than an eighth of the iterations in comparison to its series with a larger value of γ . S-BWBML does take fewer iterations, but the difference is much less marked. BWML and ITML appear to take less than half the iterations for the smaller value of γ . The T-BWML algorithms experiences optimisation issues with small values of γ because the solutions tend to take on small values, small enough that they become difficult to manage numerically. For $\gamma = 0.01$, A-BWBML, BWML and ITML all converge in fewer iterations than the T-BWML algorithms, which is quite different to previous tests. In Figure 3.6 the previously noted effects are also present. A-BWBML has the fastest execution time comparatively for small γ , but the second largest execution time for large γ . A-BWBML may be fastest on small γ as each ξ variable is independent and the alterations to M are small, so the averaging step may simultaneously solve each margin problem. BWML and ITML are much less sensitive than the other algorithms to γ . Similarly the execution times of S-BWBML do not vary significantly with γ , in fact its fastest execution time is with

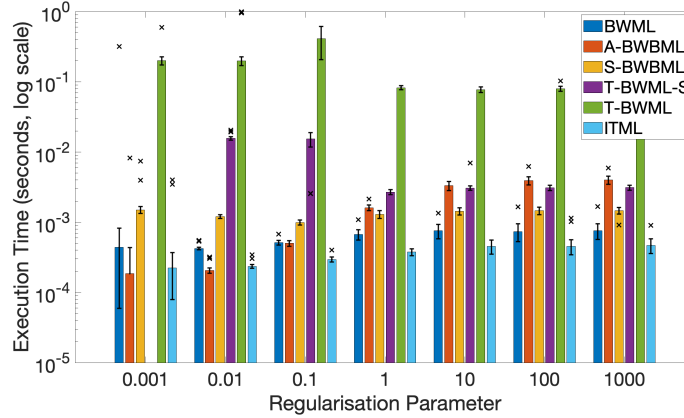


Figure 3.6: Effect of regularisation parameter on timing for synthetic problems.

$\gamma = 0.1$ which is in the middle of the range. The choice of step size for S-BWML may be more important in this case. The smoothing induced by the step-size may be more useful when the matrix variable is more prevalent, whereas the margin variables, which do not interact with each other, will not benefit from smoothing. The numerical issues encountered by T-BWML-S are also visible in these tests. For small γ the algorithm does not converge ($\gamma = 0.001$ has been omitted) and as γ becomes larger the execution times become more consistent. T-BWML seems to have more consistent execution times for larger γ , and more variability in the smaller values, it is unclear why this is.

To summarise the synthetic data results: we have seen that under several different kinds of experiment each algorithm appears to converge reliably and in reasonable time. T-BWML and T-BWML-S converge in very few iterations. However, T-BWML takes much longer in execution where as T-BWML-S keeps pace with the other algorithms. Neither of these algorithms seem especially sensitive to choices of dimensionality or constraints, but T-BWML-S takes longer to converge for smaller values of the regularisation parameter. BWML and ITML seem to handle larger dimensionalities better than the barycentric methods and conversely the barycentric metrics handle increased constraints better. Respectively,

this may be due to the barycentric methods not converging to the closest solution and the cyclic projection algorithms applying their updates without a smoothing step.

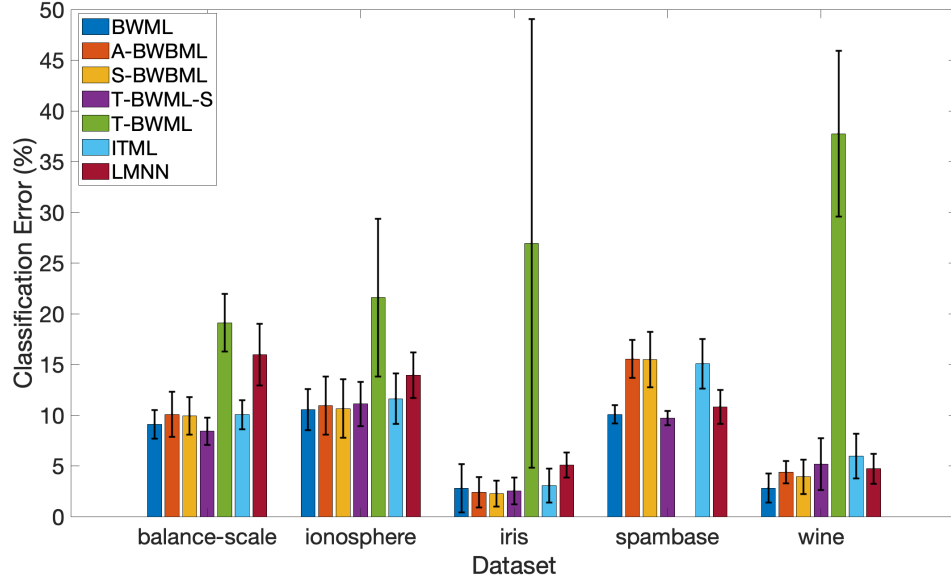
Having an understanding of our algorithms on a simple dataset, we consider our algorithms on some Metric Learning problems of modest size.

3.4.2 UCI Datasets

In the following tests each of the algorithms introduced in this chapter, ITML and LMNN are trained on similarity and dissimilarity constraints of 5 UCI datasets: Balance-scale, Ionosphere, Iris, Spambase and Wine. They vary in dimensionality, dataset size and underlying problem. The size of these problems is also small enough that the tests can be repeated to get an understanding of the statistical behaviour of the algorithms' performance. $40c(c - 1)$ pairs of points were randomly selected as the similarity and dissimilarity points for the constraints, where c is the number of classes in the dataset. These points were then identified as similar if they are of the same class or dissimilar if they are different classes.

k -fold cross validation was performed with 2 folds and 5 repeats on each algorithm-dataset pair. This resulted in a total of 10 repeats on each dataset. The k -NN classification error and time to fit each metric for each test were averaged and the standard deviation calculated. Manual tuning was performed on the γ parameter for each algorithm to find metrics with good performance. Each algorithm had the same convergence parameter and number of maximum iterations. We do not present convergence results here, instead focusing on classification error and execution time.

In Figure 3.7 the mean error for each algorithm on the datasets is presented, with the error bars indicating one standard deviation from the mean result. For Balance-scale, Ionosphere and Iris our algorithms and ITML appear to have sim-

Figure 3.7: UCI k -NN classification performance

ilar performance for classification error, with the exception of T-BWML. LMNN has a higher mean classification error on these datasets. However on Spambase, BWML, T-BWML-S and LMNN seem to have improved performance over A-BWML, S-BWML and ITML. Because T-BWML would not converge on Spambase problems its result is omitted. On Wine, our algorithms seem to have better performance than ITML or LMNN as well. Generally speaking, BWML or T-BWML-S have the smallest classification error for each dataset. This is interesting, as T-BWML-S only uses similarity constraints and not dissimilarity constraints, suggesting that for some datasets similarity is all that is necessary for finding an adequate metric. T-BWML performs significantly worse than the other algorithms. This may be because the duality gap incurred in its formulation is large enough that the problem it solves for are much less reflective of the original problem.

In Figure 3.8 the mean time to fit each metric is presented and a standard de-

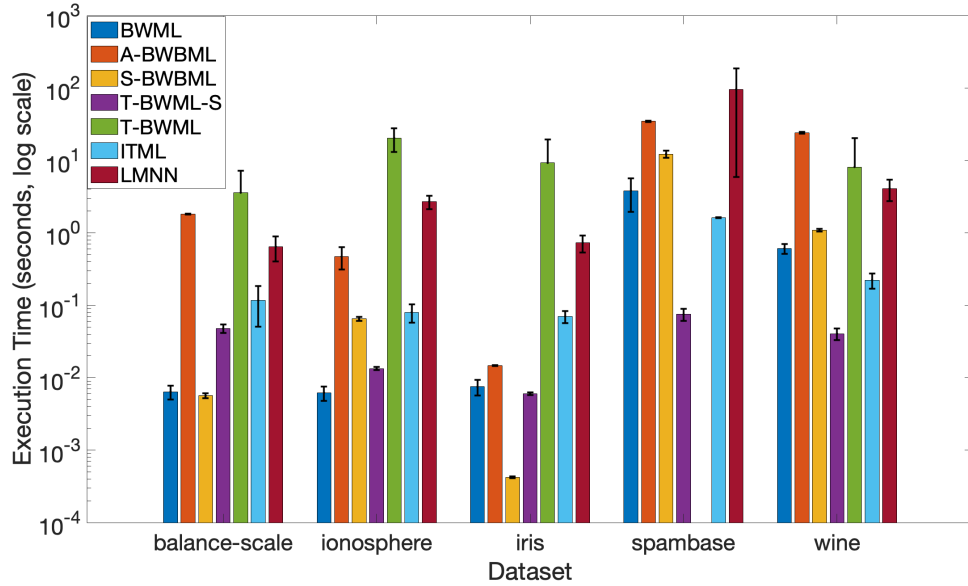


Figure 3.8: UCI timing performance

viation around the mean is shown. Note that the times are presented on a logarithmic scale. In some cases one standard deviation away from the mean is a negative quantity, as before we omit these error bars. For timing performance BWML and T-BWML-S seem to attain the best results. For Balance-scale, Ionosphere and Iris BWML, S-BWBML and T-BWML-S are the fastest algorithms then followed closely by ITML. However on Spambase and Wine these algorithms (except for T-BWML-S) seem to take significantly longer than ITML. This may be happening because the Spambase and Wine classifiers seem to prefer solutions with residual eigenvalues in some dimensions, i.e., low-rank solutions. The Bures-Wasserstein distance can become arbitrarily large for matrices converging on low-rank subspaces, which may increase the distance to the solution from the initial point. The same is true of the log-determinant divergence used in ITML, but our algorithms suffer more in this case. We believe that this occurs because as eigenvalues vanish, the gradient for the Bures-Wasserstein algorithms vanishes faster than ITML. T-BWML-S uses dual variables and can bypass this limitation so it does not suffer

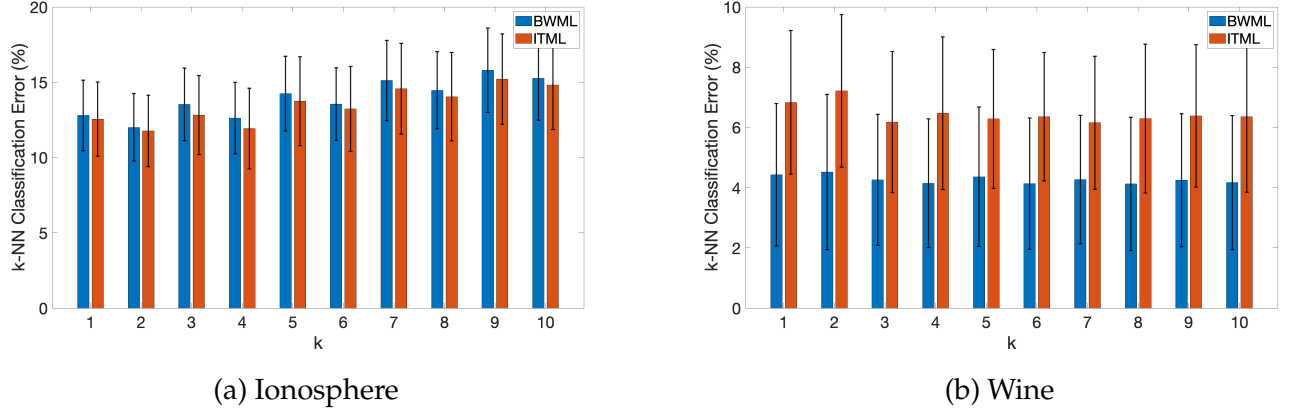


Figure 3.9: Variable choice of k for k -NN classification.

the same way.

We wish to validate our choice of $k = 4$ for these classification tasks. To examine this we evaluate the k -NN classification error with respect to values of k from 1 to 10 for BWML and ITML on Wine and Ionosphere. Each value of k is repeated 100 times to account for variability in the choice of the similarity and dissimilarity pairs. BWML and ITML are selected as our benchmark algorithms for these tests as BWML appears to be our best algorithm and ITML would be the best comparative algorithm. The results of these tests are presented in Figure 3.9.

For Ionosphere, as k increases the classification error tends to increase for both BWML and ITML. On the other hand as k increases for Wine the classification error decreases. With respect to these datasets it appears that $k = 4$ gives a good compromise between the affect of k on classification error present in each of these datasets.

3.4.3 Image Datasets

The datasets used for the image classification experiments were MNIST [137] and Fashion-MNIST [138]. Instead of using $40c(c - 1)$ pairs of constraints, as in the UCI-datasets, we increased the number of constraints to $10000c(c - 1)$ pairs, which is 90000 for both datasets. Because of the increased dimensionality of the datasets we perform dimensionality reduction using PCA (problem (2.16)) on the input features, as is standard for many Metric Learning algorithms. For both datasets a reduction from 784 to 100 dimensions was used. Also, due to the high cost of running some of the algorithms on large-scale data, cross-validation or repeats were not performed during testing. We also chose not to include T-BWML in these tests since its performance on the UCI tests was so poor and the training time on these larger datasets was excessive compared to the other algorithms. Manual parameter tuning of γ was performed, but again without repeats or cross-validation. However, from a small number of repeats it appeared that the test classification error did not vary significantly. For both datasets, the training and test datasets were used for their respective designated tasks. Similar to the UCI tests we measure the classification error of each metric using k -NN ($k = 4$) and the associated training time.

The results for the image classification tests are presented in Table 3.1. Most of these algorithms have similar on classification errors on MNIST. The largest difference is in the training time. ITML is the fastest algorithm, followed closely by BWML. S-BWBML and T-BWML-S are the next fastest algorithms, followed by LMNN. A-BWBML is by far the slowest algorithm on MNIST, as the averaging operations with the large number of constraints become very time consuming. T-BWML-S does not have as good performance on MNIST, maybe in this case the dissimilarity constraints are relevant.

On Fashion-MNIST there is a larger spread in the classification error. LMNN has the best classification error on this dataset, but is the slowest algorithm by

Table 3.1: Image dataset k -NN classification performance

Algorithm	MNIST		Fashion MNIST	
	Error (%)	Fit Time (s)	Error (%)	Training Time (s)
BWML	3.0	7.42	14.4	73.86
A-BWBML	2.8	28756.12	13.7	13.80
S-BWBML	2.8	41.42	14.4	238.99
T-BWML-S	3.9	92.47	13.6	114.86
ITML	3.1	4.75	13.8	4.75
LMNN	2.8	3309.74	13.2	2337.72

an order of magnitude. A-BWBML performs better comparatively, with a good classification error and much faster training time. T-BWML-S also has marginally better classification error, but a much larger training time. ITML also has a good classification error and the best training time. Surprisingly BWML has a poor classification error and its training time is much larger than ITML. S-BWBML also has a poor training time and classification error on this dataset. For this dataset information-theoretic formulation may be better than the optimal transport-based one.

3.5 Conclusion

In this chapter we have presented the problem of learning a Generalised Mahalanobis distance for Metric Learning using the Bures-Wasserstein distance as an objective function. This approach was motivated by the success of Information-Theoretic Metric Learning and the comparative benefits of Optimal Transport.

Several algorithms leveraging different properties of the Bures-Wasserstein distance were presented to solve this problem. BWML uses Cyclic Projections in a similar manner to ITML. The BWBML algorithms decompose the projection problem similarly to ITML, but use a barycentric method to combine these projections.

The T-BWML algorithms solve the same problem by factorising the problem in a unique way and then use constrained gradient methods to find a solution. On synthetic problems each algorithm converged reliably and had reasonable timing performance. In some cases we were able to discern how the design of the algorithm influenced its performance relative to others.

While BWML and ITML exhibited fairly fast execution time, the Cyclic Projections algorithm resulted in counter-productive interference and oscillations in convergence during training. In contrast, A-BWBML and S-BWBML converge without oscillation due to the averaging operations they incorporate. This comes at the expense of not having a guarantee of converging to the closest solution, increasing the time A-BWBML and S-BWBML take in higher dimensions. Overall, ITML was the best algorithm on the synthetic problems as it generally converged the fastest in time due to its low-complexity updates, however the margin between BWML and ITML was small.

The tests of our algorithms on UCI data showed they had advantageous performance in several cases in both speed and classification error. The difference between the synthetic data and UCI data could indicate that the information-theoretic formulation is better suited to the synthetic problems than the UCI data.

On image data, the performance in classification error is less clear-cut. Our algorithms present a range of executions times, and differences in classification error were at times marginal. Perhaps in higher dimensions the Generalised Mahalanobis distance is less useful and a more expressive metric is required. It is surprising that T-BWML-S often performed competitively without using dissimilarity constraints. It would be useful to find theoretical relationships between the solutions of BWBML algorithms and BWML. BWML finds the optimal metric with respect to the Bures-Wasserstein distance whereas BWBML algorithms do not have such a guarantee. This may inform whether or not it is possible to find the optimal metric using a modified BWBML algorithm. Investigating how

to improve the performance of T-BWML would also be a good avenue for future research. Deriving a modification to T-BWML that uses a quasi-Newton update may significantly reduce the complexity/run time of the algorithm.

We have seen that the Bures-Wasserstein distance is capable of learning metrics with comparable performance to ITML and LMNN. However, there is a significant scaling issue that is encountered by Metric Learning methods when trying to learn metrics on large-scale datasets. This issue is the quadratic scaling of linear metrics with dimensionality of their input. Additionally, many datasets are considered to lie in a low-dimensional space within their ambient space, so it is unnecessary to learn information about all of these dimensions. Therefore, in the next chapter we study techniques using the Bures-Wasserstein distance to find this type of metric and examine how this choice affects their performance.

Chapter 4

Low-rank Bures-Wasserstein Metric Learning

4.1 Introduction

The dimensionality of the PSD cone is quadratic in its input dimension, so learning a Generalised Mahalanobis distance becomes infeasible on datasets above a moderate dimension. Underlying many Machine Learning methods is the assumption that the data lie on a low-dimensional manifold within the ambient space [151]. We can use this principle to design metrics that have a reduced number of parameters, thus reducing scalability issues. However this makes the design process more complex, as we now need to select a low-dimensional space that contains the data and then find a metric in this space.

In the case of learning a Generalised Mahalanobis distance, the most natural way to characterise a low-dimensional space of a dataset is to identify a linear subspace. In this case our metric $M \in S_+^n$ will be rank-deficient, but this still requires a geometry in which to perform learning tasks. As we have already outlined in section 2.1, several non-linear geometries have been developed for such fixed-rank matrices [84, 85, 86]. Accordingly, in this chapter we consider the set

$S_+(p, n)$ when finding a metric. Low-rank Metric Learning has been used in several contexts [18, 121, 125]. Meyer *et al.* consider similar geometric optimisation techniques for low-rank distance Metric Learning [152]. Our work differs in that it focuses on the probabilistic viewpoint of Metric Learning with respect to the Bures-Wasserstein distance.

Following the theme of this thesis, we explore how the Bures-Wasserstein distance can be used to learn such a metric. In this chapter we develop and test two Metric Learning algorithms using the Bures-Wasserstein distance. The first is an extension of BWML to $S_+(p, n)$ that uses Cyclic Projections, while the second is based on Riemannian optimisation. The first algorithm is Low-Rank Bures-Wasserstein Metric Learning (LR-BWML) presented in section 4.2.1, which is BWML restricted to a linear subspace of $S_+(p, n)$, so techniques from the previous chapter can be used. While this algorithm is efficient, the restriction to a linear subspace is a decision that needs to be made *a priori* and is not integrated into the algorithm. The second algorithm, based on Riemannian optimisation, uses the geometry in [118, 87] and is presented in section 4.2.2. It is inspired by the Metric Learning algorithm DRML of Harandi *et al.* [121] (Problem (2.18)). Because this algorithm uses a non-linear geometry on $S_+(p, n)$ it has more freedom in its solution compared to Low-Rank BWML. For this algorithm we use a sum of logistic discriminant terms on similarity and dissimilarity terms as the objective, as in [16]. Our algorithm can be seen as a low-rank generalisation of the Metric Learning algorithm presented by Han *et al.* [55]. In their work they briefly outline a Metric Learning formulation with a small number of convergence tests. We also derive a regulariser based on the Bures-Wasserstein distance, in keeping with our theme. This regulariser is included in the logistic-discriminant objective. We refer to this algorithm as Bures-Wasserstein Logistic Discriminant Metric Learning or BW-LDML.

To test these Metric Learning algorithms we perform experiments similar to those in the previous chapter, but with different choices of rank included to observe its effect. The results of these convergence experiments are presented in section 4.4.1

and the UCI and image dataset classification are presented in sections 4.4.2 and 4.4.3 respectively. By using the same experiments we can investigate how using a low-rank approach changes performance relative to the full-rank algorithms. For comparison, we include information-theoretic counterparts to each algorithm in the experiments and another low-rank Metric Learning algorithm that uses optimal transport. Low-Rank ITML (LR-ITML) [125] is the low-rank counterpart to BWML and DRML is the counterpart to BW-LDML. Wasserstein Discriminant Analysis (WDA - Problem (2.23)) [52] is also included in the classification tests as a comparative Optimal Transport algorithm that learns a low-rank metric. We find that LR-BWML and LR-ITML generally execute the fastest and have good performance relative to the other algorithms. However in some cases BW-LDML is able to maintain performance over a wider range of choices of rank.

4.2 Methods

We introduce and derive two Metric Learning algorithms. The first is essentially BWML from the previous chapter adapted to learning in a low-rank setting. The component we focus on is demonstrating the validity of using BWML in this way.

The second algorithm uses Riemannian optimisation on $S_+(p, n)$ to find a low-rank metric. This method has a larger capacity of metrics due to the availability of matrices in $S_+(p, n)$, whereas BWML can only compute a metric that exists on a linear subspace of $S_+(p, n)$. This algorithm also uses a regulariser based on the Bures-Wasserstein distance, but instead phrases the Metric Learning problem as an unconstrained optimisation problem.

4.2.1 Low-Rank BWML

To extend the Bures-Wasserstein distance to learning a low-rank metric we use BWML with a low-rank initialisation metric $M_0 \in S_+(p, n)$.

To derive the Cyclic Projections updates for BWML we solved:

$$\nabla_{M_{t+1}} f(M_{t+1}) = \nabla_{M_t} f(M_t) + \delta_{ij} \alpha X_{ij}, \quad (4.1)$$

where $f(M) = \mathcal{B}^2(M, M_0)$.

Observing the gradient of $\mathcal{B}^2$:

$$\nabla_M \mathcal{B}^2(M, M_0) = I - M_0^{\frac{1}{2}} (M_0^{\frac{1}{2}} M M_0^{\frac{1}{2}})^{-\frac{1}{2}} M_0^{\frac{1}{2}}.$$

We see that if $M_0 \in S_+(p, n)$ then the term in the inverse will be low-rank and thus not invertible. Therefore we cannot solve Equation (4.1) in its current form. We address this by using a similar approach to Kulis *et al.* [125]. If a subspace of $\mathbb{R}^n$ is selected using an orthogonal matrix $V \in St(p, n)$ that has the same range space as M_0 , then the closest M in this subspace that adheres to similarity and dissimilarity constraints may be computed. While this does not find the closest solution in $S_+(p, n)$, it may still be a good metric. This makes the choice of the range space of M_0 a significant factor in finding a desirable metric.

To use BWML on such a space we need to show that applying such a dimensionality reduction technique will not alter the Metric Learning problem beyond reducing the space of potential solutions. To do this we need to show that $\mathcal{B}^2(A, B)$ is invariant to dimensionality reduction, subject to some constraints, which we formalise. We first prove a lemma about the matrix square root:

Lemma 4.2.1 *For $V \in St(p, n)$ and $A \in S_+(p, n)$ such that $\text{range}(V) = \text{range}(A)$ then $(VAV^T)^{\frac{1}{2}} = VA^{\frac{1}{2}}V^T$.*

Proof: Since $A \in S_+(p, n)$ it has the eigendecomposition $U^T \Lambda U$ where $U \in St(p, n)$ and has the same range as V . If $\text{range}(V) = \text{range}(U)$ then $U = VH$ where $H \in$

$\mathcal{O}(p)$. Therefore:

$$\begin{aligned}
& (VAV^T)^{\frac{1}{2}} \\
&= (VU^T\Lambda UV^T)^{\frac{1}{2}} \\
&= (VV^TH^T\Lambda UVV^T)^{\frac{1}{2}} \\
&= (H^T\Lambda H)^{\frac{1}{2}} \\
&= H^T\Lambda^{\frac{1}{2}}H \\
&= VV^TH^T\Lambda^{\frac{1}{2}}HV V^T \\
&= VA^{\frac{1}{2}}V^T.
\end{aligned}$$

■

We also require the lemma:

Lemma 4.2.2 *Let $M \in S_+(p, n)$ and let $V \in St(p, n)$ such that $\text{range}(M) \subseteq \text{range}(V)$. Then $M = V^T V M V^T V$.*

Proof: see [125], Lemma 8

Proposition 4.2.3 *For semidefinite matrices $A, B \in S_+(p, n)$ and $V \in St(p, n)$, such that $\text{range}(A) = \text{range}(B) = \text{range}(V)$, then: $\mathcal{B}(A, B) = \mathcal{B}(VAV^T, VBV^T)$.*

Proof:

$$\begin{aligned}
& \mathcal{B}^2(VAV^T, VBV^T) \\
&= \text{tr}(VAV^T + VBV^T - 2((VBV^T)^{\frac{1}{2}}VAV^T(VB^{\frac{1}{2}}V^T)^{\frac{1}{2}})) \\
&= \text{tr}(VAV^T + VBV^T - 2(VB^{\frac{1}{2}}V^TVAV^TVB^{\frac{1}{2}}V^T)^{\frac{1}{2}}) \\
&= \text{tr}(VAV^T + VBV^T - 2(VB^{\frac{1}{2}}AB^{\frac{1}{2}}V^T)^{\frac{1}{2}}) \\
&= \text{tr}(VAV^T + VBV^T - 2V(B^{\frac{1}{2}}AB^{\frac{1}{2}})^{\frac{1}{2}}V^T) \\
&= \text{tr}(A + B - 2(B^{\frac{1}{2}}AB^{\frac{1}{2}})^{\frac{1}{2}}) \\
&= \mathcal{B}^2(A, B).
\end{aligned}$$

■

If we now select $V \in St(p, n)$ such that $\text{range}(M) = \text{range}(M_0) = \text{range}(V)$ then:

$$\begin{aligned} & \nabla_M \mathcal{B}^2(M, M_0) \\ &= \nabla_M \mathcal{B}^2(VMV^T, VM_0V^T) \\ &= I_p - V(VM_0V^T)^{\frac{1}{2}}((VM_0V^T)^{\frac{1}{2}}VMV^T(VM_0^{\frac{1}{2}}V^T)^{\frac{1}{2}})^{-\frac{1}{2}}(VM_0V^T)^{\frac{1}{2}}V^T. \end{aligned}$$

We can now use this gradient in the Cyclic Projections update equation (4.1) to solve for M_{t+1} as the term in the matrix inverse is full-rank. Instead of operating on low-rank matrices $M_t \in S_+(p, n)$ we can simplify the problem further. By substituting $M \rightarrow \hat{M} = VMV^T \in S_{++}^p$, $M_0 \rightarrow \hat{M}_0 = VM_0V^T \in S_{++}^p$ and $\hat{x}_i \rightarrow Vx_i$ into Problem (3.2) for BWML we obtain:

$$\begin{aligned} \min_{\hat{M} \in S_{++}^p, \xi \in \mathbb{R}_+^m} & \mathcal{B}^2(\hat{M}, \hat{M}_0) + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0)) \\ \text{st. } & (\hat{x}_i - \hat{x}_j)^T \hat{M} (\hat{x}_i - \hat{x}_j) \leq \xi_{ij}, \quad (i, j) \in \mathcal{S} \\ \text{st. } & (\hat{x}_i - \hat{x}_j)^T \hat{M} (\hat{x}_i - \hat{x}_j) \geq \xi_{ij}, \quad (i, j) \in \mathcal{D}. \end{aligned} \tag{4.2}$$

Since the objective term $\mathcal{B}^2(M, M_0)$ is invariant to a correctly chosen V we may simply use the dimensionality reduced $\hat{M}$ to find the desired metric and convert back to the original dimensionality using $M = V\hat{M}V^T$. Problem (4.2) is a full-rank BWML problem, so we can use the update scheme in Algorithm 8 to solve for a metric. Notice that ξ has been left unchanged. This is because the primary issues of scalability lie in the M updates and not the margin, so it is not necessary to find a factorisation of ξ . Therefore Low-Rank BWML is a generalisation of BWML where a dimensionality reduction step is applied to the input variables. This means it immediately inherits the convergence properties already provided for BWML. The choice of the matrix V to select a subspace of $S_+(p, n)$ is a non-trivial one. For our implementation we use PCA (problem 2.16) to find an orthogonal matrix, where the transformed datapoints still retain most of their variance when projected. Additionally, PCA is efficient enough not to impact the overall complexity of the algorithm. Since $M_0 = I$ was a successful choice for BWML we

also select $M_0 = VV^T$ so that $\hat{M}_0 = I_p$. Funnily enough, this dimensionality reduction step is equivalent to the preprocessing made for the image classification tests in the previous chapter. The above presentation formalises that approach and motivates us to see how well Low-Rank BWML performs in Metric Learning tasks at different target ranks.

Having adapted BWML for a low-rank setting we now present a Metric Learning algorithm that reformulates the core components of BWML in quite a different way.

4.2.2 Bures-Wasserstein Logistic-Discriminant Metric Learning

Thus far the algorithms we have presented frame the Metric Learning problem as a projection of a semidefinite matrix onto a feasible set, with respect to the Bures-Wasserstein distance. The set S_{++}^n of SPD matrices are amenable to this type of approach. However, the non-linear structure of $S_+(p, n)$ means many of the previously used methods cannot be applied directly. Low-Rank BWML addresses this problem by operating on a linear subspace of the fixed-rank semidefinite matrices. However, this requires selecting a suitable subspace in the first place, which is not a trivial task. We draw inspiration from Harandi *et al.* [121] in which they find a low-rank Mahalanobis metric is found using Riemannian optimisation methods and a logistic-discriminant objective [16]. For our metric we decompose $M \in S_+(p, n)$ as $M = Y^T Y$, where $Y \in \mathbb{R}^{p \times n}$.

To begin our derivation we see that the Bures-Wasserstein distance $\mathcal{B}^2(A, B)$ can also be expressed as [69]:

$$\begin{aligned} \min_{X \in S_+^{2n}} \quad & \langle C, X \rangle \\ \text{s.t.} \quad & P_0^T X P_0 = A \\ & P_1^T X P_1 = B, \end{aligned} \tag{4.3}$$

where

$$C = \begin{bmatrix} I & -I \\ -I & I \end{bmatrix}, P_0 = \begin{bmatrix} I \\ \mathbf{0}_n \end{bmatrix}, P_1 = \begin{bmatrix} \mathbf{0}_n \\ I \end{bmatrix}. \quad (4.4)$$

This formulation is analogous to the optimal transport distance between discrete distributions (Expression (2.12)) except the terms are PSD matrices. We leverage this formulation in two ways. We can use this formulation and solve for a Generalised Mahalanobis distance by constructing:

$$\begin{aligned} & \min_{X \in \mathcal{S}_+^{2n}} \langle C, X \rangle \\ & s.t. P_0^T X P_0 = M_0 \\ & (x_i - x_j)^T P_1^T X P_1 (x_i - x_j) \leq u, (i, j) \in \mathcal{S} \\ & (x_i - x_j)^T P_1^T X P_1 (x_i - x_j) \geq l, (i, j) \in \mathcal{D}. \end{aligned}$$

We then use a Burer-Monteiro factorisation [153]:

$$X = \begin{bmatrix} Y_0^T \\ Y^T \end{bmatrix} \begin{bmatrix} Y_0 & Y \end{bmatrix}, \quad (4.5)$$

where $Y, Y_0 \in \mathbb{R}^{p \times n}$ to get a low-rank expression for this problem. Substituting this into the objective of (4.3) we get:

$$\begin{aligned} & \left\langle \begin{bmatrix} I & -I \\ -I & I \end{bmatrix}, \begin{bmatrix} Y_0 \\ Y \end{bmatrix} \begin{bmatrix} Y_0^T & Y^T \end{bmatrix} \right\rangle \\ & = \text{trace}(Y_0^T Y_0 - Y_0^T Y - Y^T Y_0 + Y^T Y) \\ & = \|Y - Y_0\|_F^2. \end{aligned}$$

This results in the problem becoming:

$$\begin{aligned} & \min_{Y \in \mathbb{R}^{p \times n}} \|Y - Y_0\|_F^2 \\ & s.t. (x_i - x_j)^T Y^T Y (x_i - x_j) \leq u, (i, j) \in \mathcal{S} \\ & (x_i - x_j)^T Y^T Y (x_i - x_j) \geq l, (i, j) \in \mathcal{D}, \end{aligned}$$

where the first constraint has been removed because it is automatically satisfied. This problem is not amenable to efficient optimisation techniques. We choose to relax the similarity and dissimilarity terms by replacing each constraint with a corresponding logistic discriminant term in the objective. The optimisation problem is then:

$$\min_{Y \in \mathbb{R}^{p \times n}} \sum_{(i,j) \in \mathcal{S} \cup \mathcal{D}} [p_{ij}^* \log(p_{ij}(Y)) + (1 - p_{ij}^*) \log(1 - p_{ij}(Y))] + \gamma \|Y - Y_0\|_F^2, \quad (4.6)$$

where

$$p_{ij}(Y) = (1 + \exp(1 - \|x_i - x_j\|_{Y^T Y}^2))^{-1}$$

γ is a regularisation parameter, and p_{ij}^* corresponds to the true similarity-dissimilarity classification of (i, j) .

We refer to this problem as Bures-Wasserstein Logistic-Discriminant Metric Learning or BW-LDML. It is apparent that Problem (4.6) is non-convex, making optimisation much more difficult if we are to use Euclidean optimisation techniques. However, we may equip $Y \in \mathbb{R}^{p \times n}$ with the geometry in [118, 87]. This geometry identifies each Y with a corresponding element $Y^T Y \in S_+(p, n)$ and is amenable to Riemannian optimisation techniques. Problem (4.6) retains some of the underlying structure of a Wasserstein distance problem. Additionally, the Frobenius-norm regularisation term is very similar to the expression for the corresponding Bures-Wasserstein distance on $S_+(p, n)$ which is:

$$d(Y, Y_0) = \min_{U \in \mathcal{O}(p)} \|Y - UY_0\|_F^2. \quad (4.7)$$

To optimise Problem (4.6) we select Riemannian L-BFGS [106], a Riemannian optimisation algorithm. We make this selection based on the enhanced convergence properties that this algorithm offers without needing to compute an expensive Hessian term, and on its reduced memory requirements.

4.3 Implementation

The implementation of LR-BWML (which is the same as the BWML implementation except for the preprocessing steps) and BW-LDML are made available as MATLAB code at <https://gitlab.ecs.vuw.ac.nz/cooperdavi1/thesis-implementation>. Implementations of DRML, LR-ITML and WDA are also made available there. The experiments themselves were executed in the same grid-computing environment described in the previous chapter.

4.4 Experiments

For the low-rank Metric Learning algorithms we complete a similar set of experiments to the full-rank ones in the previous chapter. These consist of measuring convergence on synthetic problems that we generated and k -NN classification error tests on UCI Datasets, MNIST and Fashion MNIST. The significant difference in these tests and the previous chapter's is that the effect of rank is examined on performance, and how this varies between algorithms. In addition to the algorithms presented in this chapter, we also include information-theoretic counterparts and another optimal transport algorithm for comparison. The low-rank version of Information Theoretic Metric Learning (LR-ITML) [125] and Dimensionality Reduction Metric Learning (DRML) [121] are included as the information-theoretic counterparts to LR-BWML and BW-LDML. In addition to the comparison between Bures-Wasserstein and information-theoretic methods, we also consider comparisons between the projection algorithms LR-BWML and LR-ITML and the logistic-discriminant methods BW-LDML and DRML. Wasserstein Discriminant Analysis (WDA) [52] is also included as it provides a relevant comparison of another optimal transport based algorithm for learning a low-rank metric.

4.4.1 Synthetic Data

Similar to the synthetic data used in the previous chapter, constraints $a_i \in \mathbb{R}^n, i \in [m]$ with dimension n are sampled with unit normal distributed entries. Each a_i is used in place of $x_i - x_j$ for each algorithm's objective and constraint terms. Each constraint is presented as a similarity constraint so that feasibility is easy to guarantee. The rank specified for each algorithm is denoted p , where each algorithm either learns $Y \in \mathbb{R}^{p \times n}$ or $M \in S_+(p, n)$. For an initial M_0 or Y_0 the p principal components of vectors a_i were computed. The resulting matrix $WinSt(p, n)$ was either used as $Y_0 = W$ or $M_0 = WW^T$. The convergence and timing behaviour of the algorithms is then recorded during execution to characterise their performance. Convergence is evaluated by measuring the suboptimality at the i -th iteration, i.e. the difference between the objective at the i -th iterate and the solution. To account for random fluctuations in convergence, each experiment is repeated 100 times. In the convergence figures the median and a 25% and 75% quantile bounds of the suboptimality are presented. LR-ITML and DRML are included in these tests, but WDA is not. WDA is a dimensionality-reduction algorithm, and so this projection-esque task does not make sense as a test for this algorithm. As with the previous synthetic tests the effect of number of constraints, dimensionality and regularisation are presented here.

Effect of Constraints

First, we examine the effect of varying the number of constraints on each algorithm's convergence. The dimensionality in these tests is fixed at 10, and rank is set to 2, 5 and 10 to see how this affects convergence.

Figure 4.1 shows the convergence for BW-LDML. Convergence appears linear and consistent. Generally as the rank increases the number of iterations increases modestly. Across the different values of rank, the fastest series to converge is with $m = 100$ constraints and the slowest is $m = 20$, but $m = 20$ and $m = 50$ converge at a similar rate.

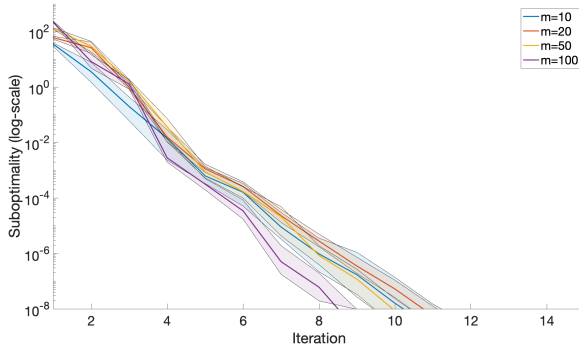
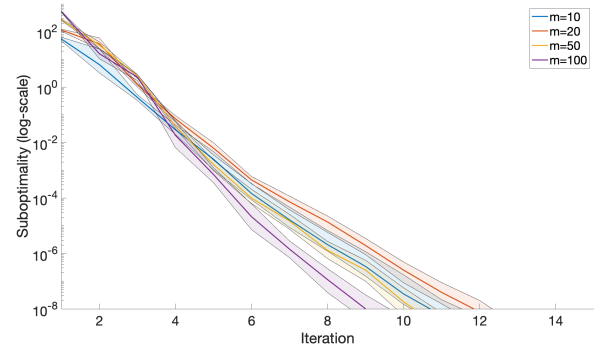
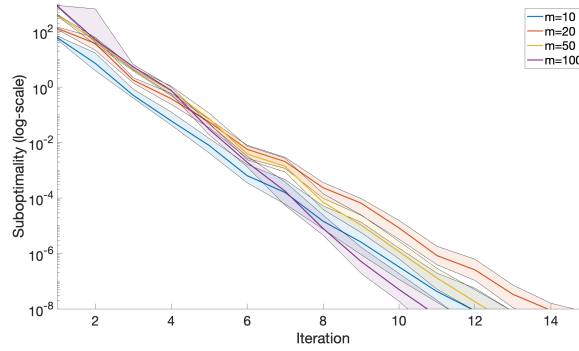
(a) $p = 2$ (b) $p = 5$ (c) $p = 10$

Figure 4.1: Convergence of BW-LDML on synthetic problems with varying number of constraints and rank, and constant dimension of 10.

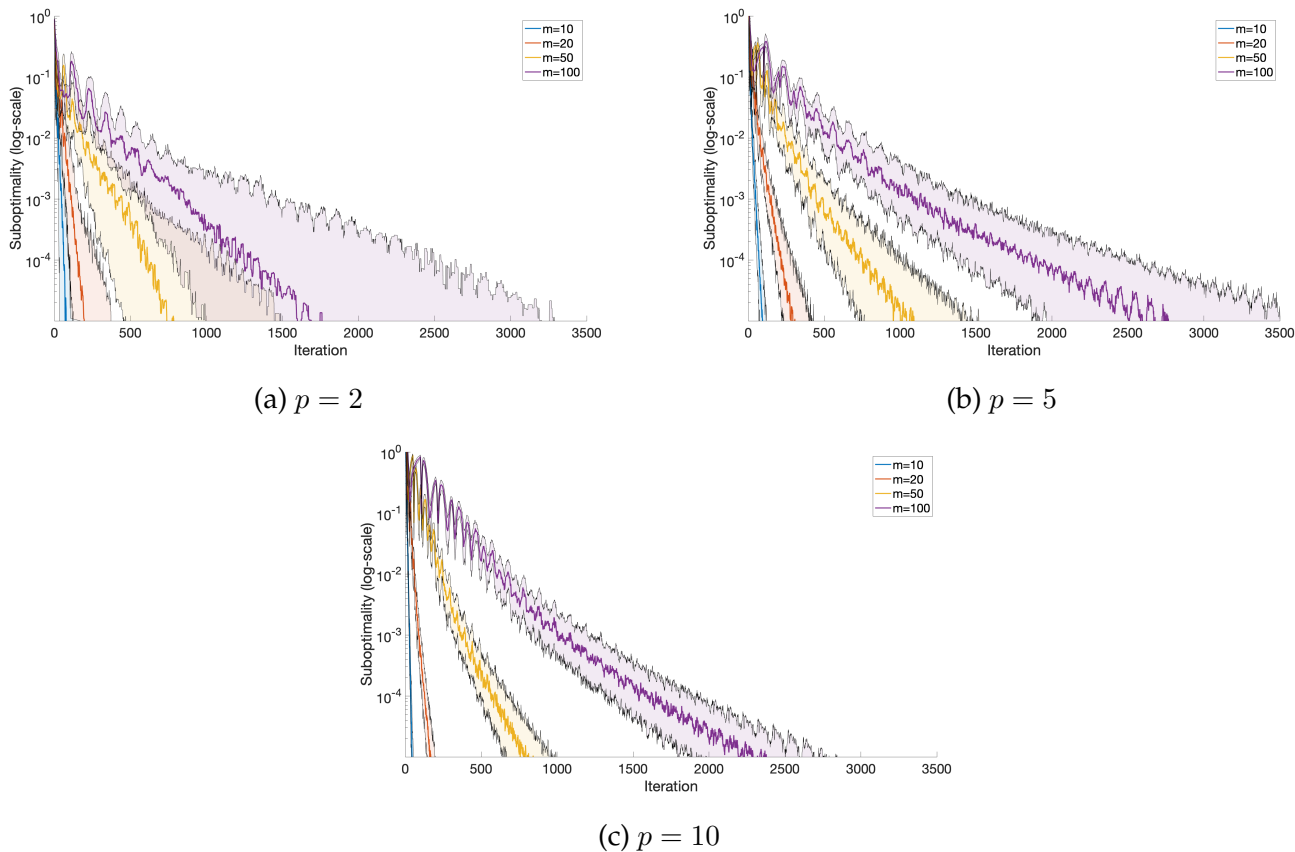


Figure 4.2: Convergence of LR-BWML on synthetic problems with varying number of constraints and rank, and constant dimension of 10.

The convergence plots shown in Figure 4.2 for LR-BWML are fairly similar to those of BWML. Generally speaking, including more constraints increases the number of iterations to converge and also increases the overall variability. As discussed in §3.4.1, this is due to the interference that occurs between many constraints sequentially operating on the matrix variable. It appears that as the rank increases the variability in the convergence decreases. While the rank equal to two series has some smaller median lines of convergence, the convergence bounds are wider than the other two series. This effect may be caused by increased interfer-

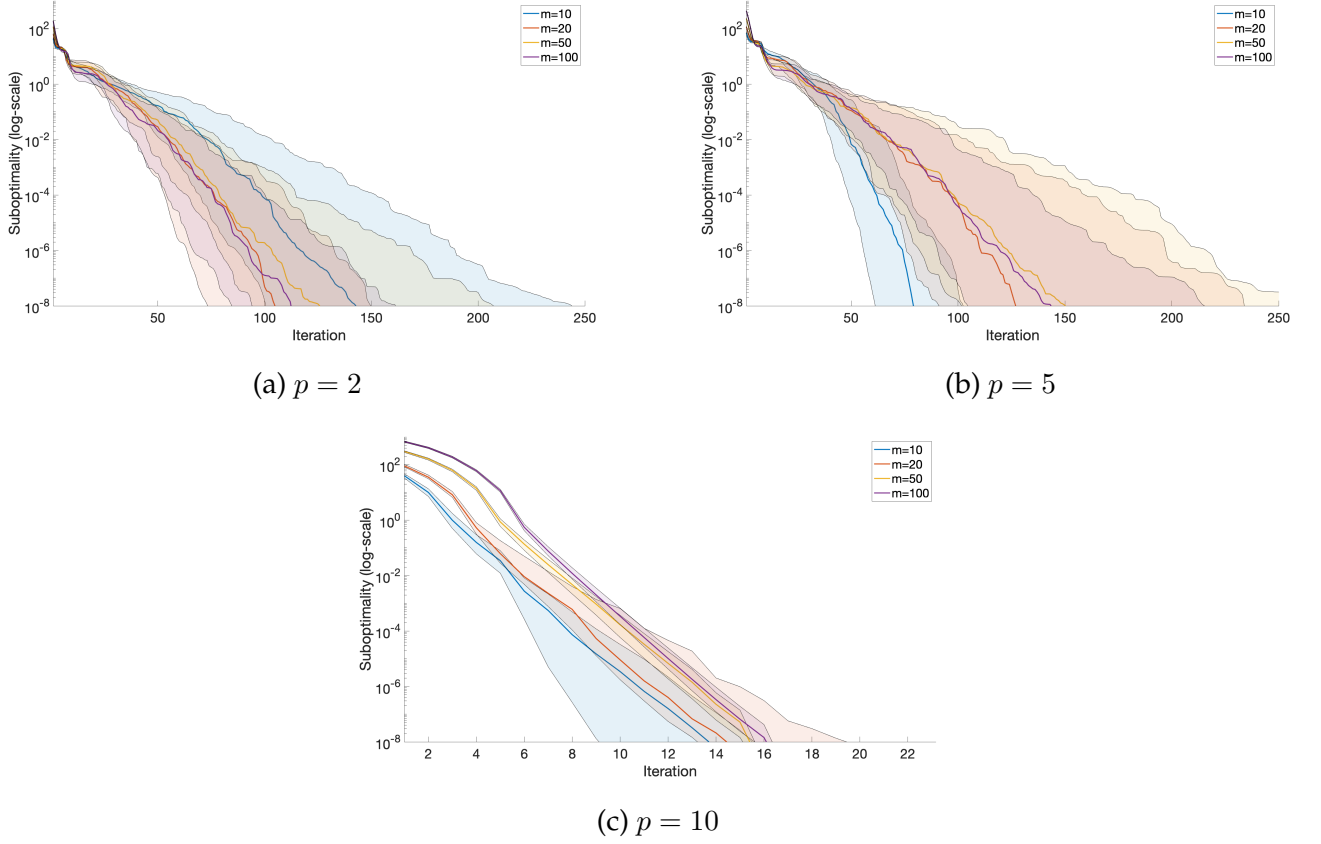


Figure 4.3: Convergence of DRML on synthetic problems with varying number of constraints and rank, and constant dimension of 10.

ence relative to the dimension of the problem. Because LR-BWML transforms the original n -dimensional problem to a dimension equal to the rank, which results in more interference.

Convergence trends for DRML are shown in Figure 4.3. The convergence for rank equal to 10 is much faster than those for rank equal to two and five. This is attributable to the fact that, if rank is equal to the dimensionality, the manifold structure used in DRML treats all Stiefel manifold elements as equivalent. Therefore this variable will be unchanged by any gradient updates, thus simplifying the

optimisation routine. For ranks two and five, the rate of convergence between the different number of constraints appears fairly small, with the overlapping variance being quite significant. The increase of rank from two to five seems to increase the required number of iterations. DRML and BW-LDML differ in their factorisation of the metric and their optimiser, but not in their objective terms. The dimensions of the variables for BW-LDML and DRML are $p(2n - p - 1)/2$ and $p(2n - p + 1)/2$ respectively. So while the dimensions of these manifolds are similar, the product structure of DRML may cause the variables to interfere with each other, causing the longer convergence times we see.

Figure 4.4 illustrates the convergence for LR-ITML and shows similar trends to LR-BWML, with the rank acting as the effective dimensionality of the problem. It seems that LR-ITML converges faster than LR-BWML, similar to the case for BWML and ITML.

Figure 4.5 shows timing performance for all four algorithms and indicates that, for small numbers of constraints, the projection methods appear to converge much faster than the logistic-discriminant methods. However as the number of constraints increases the relative gap between the projection methods and the Riemannian methods decreases. For 100 constraints BW-LDML is actually the fastest algorithm for ranks two and five. It appears that the execution time of the logistic-discriminant methods are much less sensitive to the number of constraints compared to the projection methods. This is consistent with the convergence plots since BW-LDML and DRML do not appear to change much with respect to the number of constraints, whereas LR-BWML and LR-ITML do.

Effect of Dimensionality

This section looks into how the problem dimensionality affects these algorithms in practice. Each result is paired into two plots. In the first of these the dimension and number of constraint are both fixed at 50, and the rank of the problem

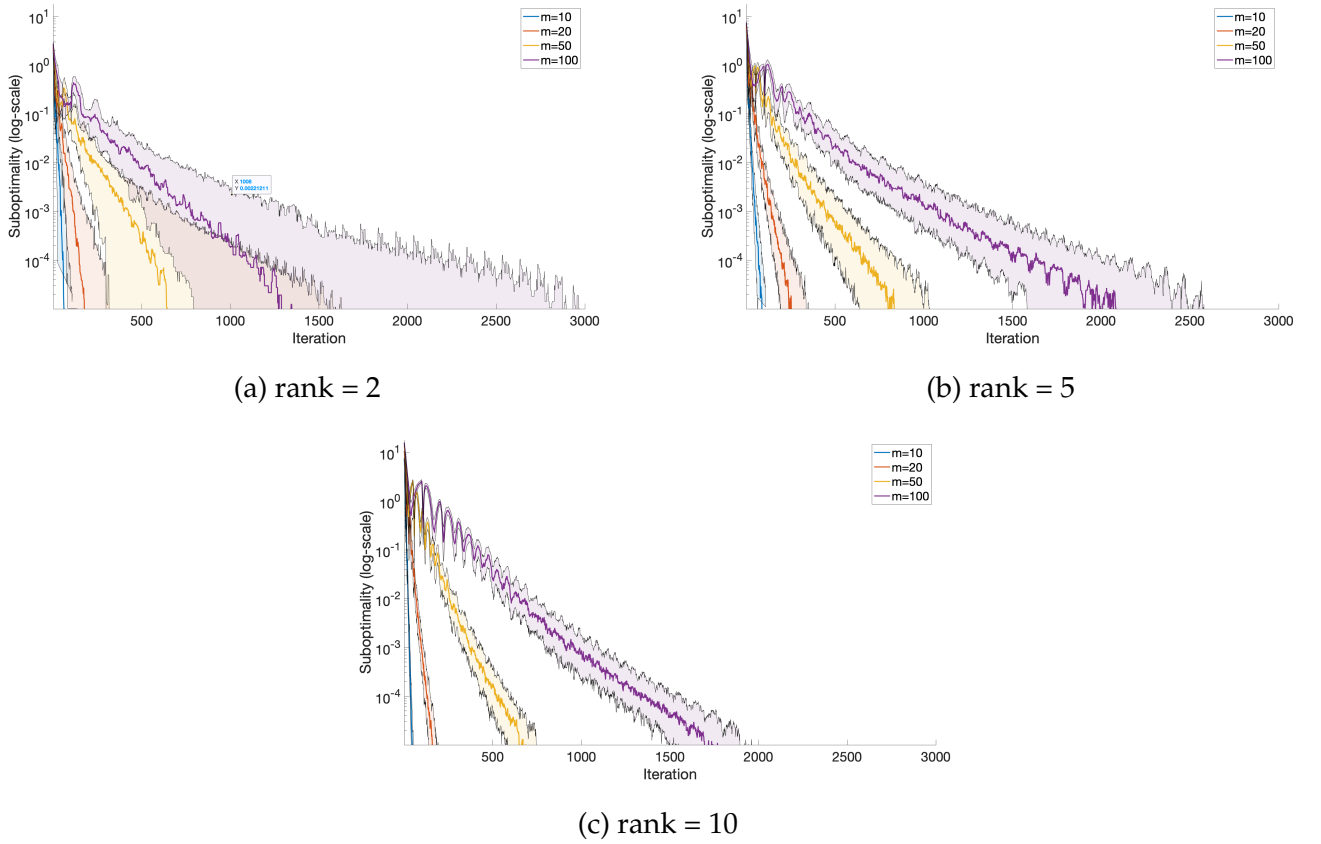


Figure 4.4: Convergence of LR-ITML on synthetic problems with varying number of constraints and rank, and constant dimension of 10.

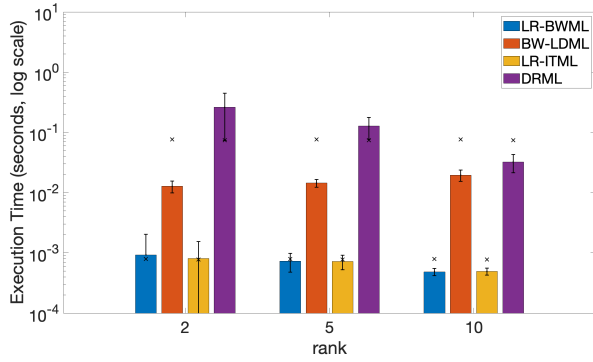
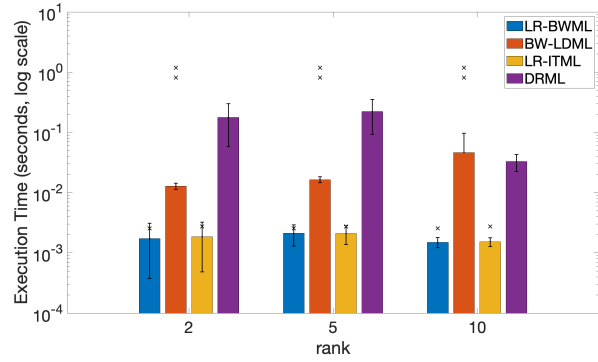
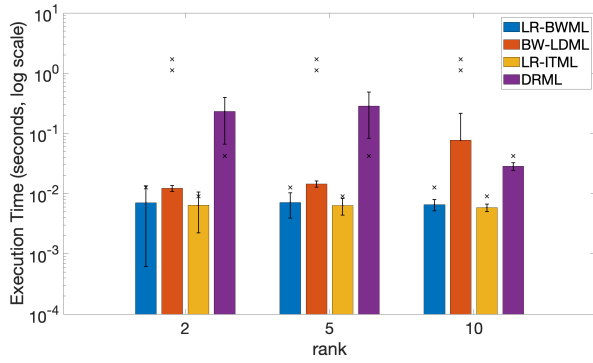
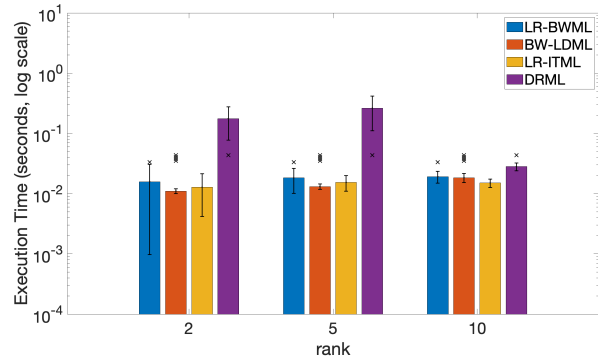
(a) $m = 10$ (b) $m = 20$ (c) $m = 50$ (d) $m = 100$

Figure 4.5: Timing for the four algorithms on synthetic problems for varying numbers of constraints and rank, and constant dimension of 10.

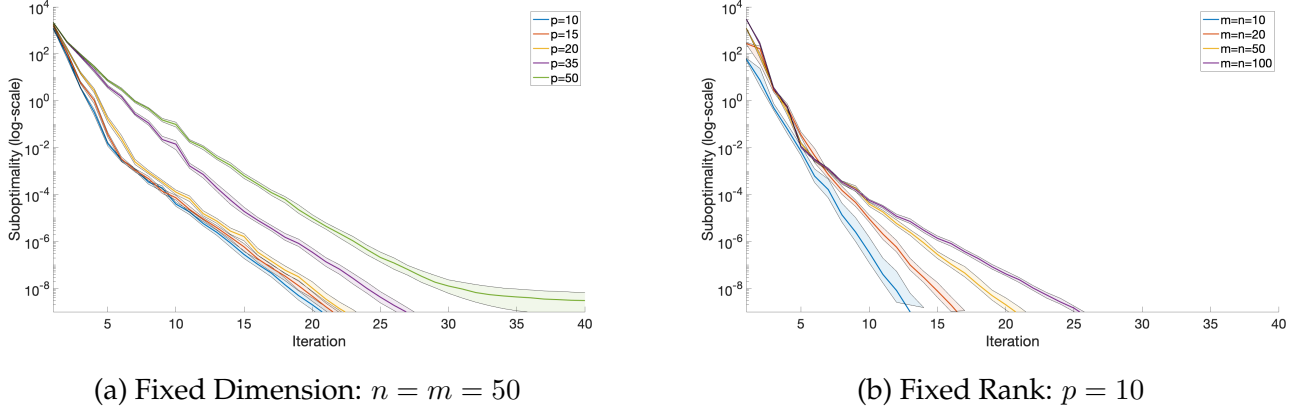


Figure 4.6: Convergence of BW-LDML on synthetic problems for fixed dimension and rank.

is varied. For the second plot, the rank is fixed at 10, but the dimension and the number of constraints are varied. In both cases the dimensionality and number of constraints are equal. This done particularly for LR-BWML and LR-ITML so the effect of interference with respect to constraints does not confound the effects of dimensionality. If the number of constraints were to be fixed it would be fixed at the maximum value of dimension that is tested, which would add extra interference for these algorithms.

The BW-LDML convergence results are shown in Figure 4.6. BW-LDML appears to converge faster at smaller rank (seen in Figure 4.6a). There also appears to be a plateau in convergence encountered in the full-rank case. Again the convergence is linear and rapid. When varying dimensionality (Figure 4.7b), we see that the convergence slows with increasing dimensionality.

Figure 4.7 shows how dimensionality affects convergence for LR-BWML. It appears in the fixed dimension case (Figure 4.7a) that as the rank decreases convergence slows. This is explained by the fact that the number of constraints is constant and as the target rank is decreased the effective dimensionality in the

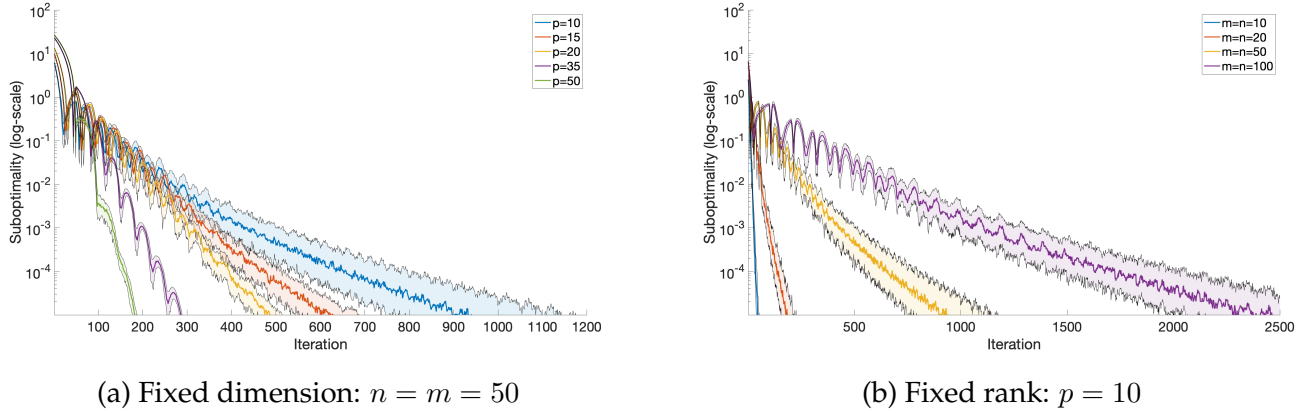


Figure 4.7: Convergence of LR-BWML on synthetic problems for fixed dimension and rank.

problem is reduced. This then causes interference to rise, increasing the number of iterations required for convergence. Similar to the fixed rank experiments (Figure 4.7b), as the dimensionality and constraints increase (with the rank being held constant) the effective dimensionality is constant; this then causes the increase in iterations to convergence due to interference.

In the case of DRML with a fixed dimensionality, different choices of rank do not seem to have a significant effect on the rate of convergence, seen in Figure 4.8a. This indicates that there is little to be gained in terms of convergence by reducing the rank. An obvious special case is rank equal to 50, i.e., the same as the dimensionality, which converges much faster than the other series. This is because the Stiefel manifold variable is unchanged during optimisation, as described earlier. Figure 4.8b shows clearly that in higher dimensions convergence is slower. The increase in problem dimensionality n increases dimensionality of DRML's Stiefel manifold variable $W \in St(p, n)$ but not the PSD variable $M \in S_{++}^p$, which has its dimensionality determined by the rank. It is interesting that the change in dimensionality affects DRML more than the change in rank.

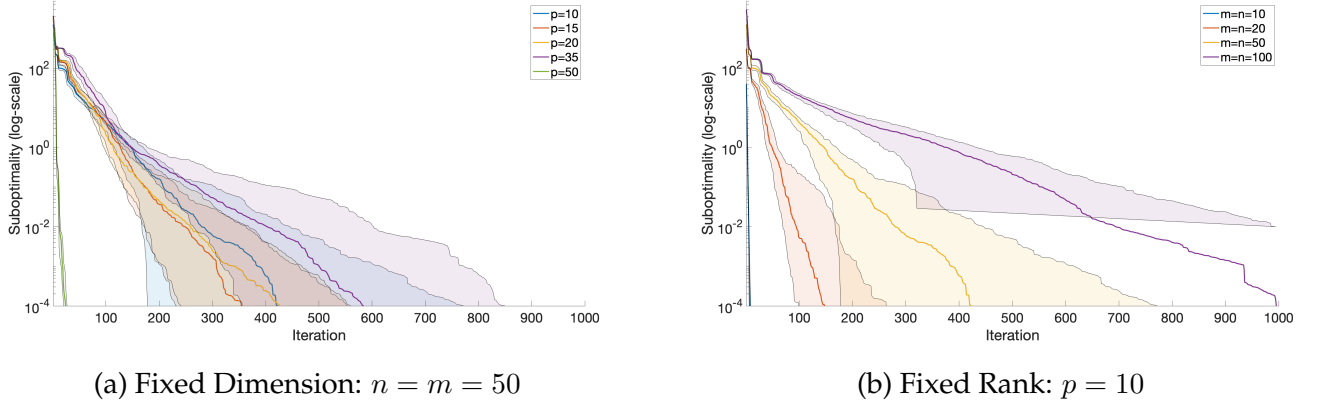


Figure 4.8: Convergence of DRML on synthetic problems for fixed dimension and rank.

In Figure 4.9 LR-ITML operates similarly to LR-BWML, with interference effects dominating the behaviour of convergence. Additionally, LR-ITML converges in fewer iterations than LR-BWML .

We see in Figure 4.10 that LR-BWML and LR-ITML require much less time to execute compared to BW-LDML and DRML for both fixed dimension and fixed rank. Both LR-BWML and LR-ITML execute the fastest at an intermediate value of rank and dimension. This can be attributed to the reduced complexity of operating on low-rank or lower dimension, balanced with the increased number of iterations required when dimension is small. For DRML, there is a small increase in execution time as rank increases in Figure 4.10a, until rank is equal to dimension where the execution time drops. A similar trend is seen in Figure 4.10b. The increase in the complexity of the updates due to the change in variable sizes would explain this change. BW-LDML has a more obvious trend with respect to rank. The timing figures show a seemingly quadratic increase (with respect to the logarithmic scale) in execution time with rank for BW-LDML.

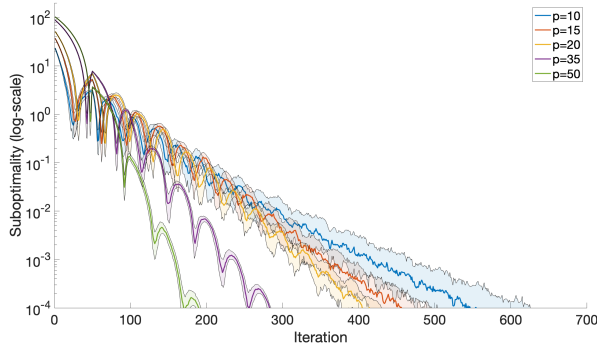
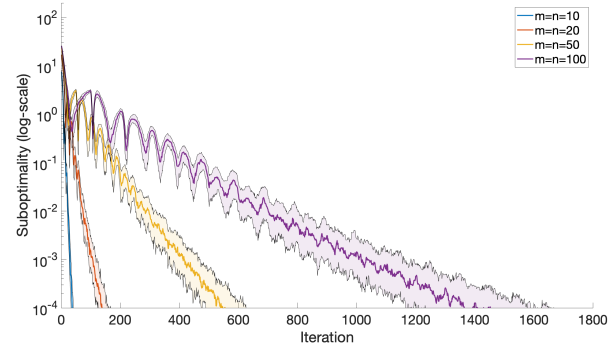
(a) Fixed Dimension: $n = m = 50$ (b) Fixed Rank; $p = 10$

Figure 4.9: Convergence of LR-ITML on synthetic problems for fixed dimension and rank.

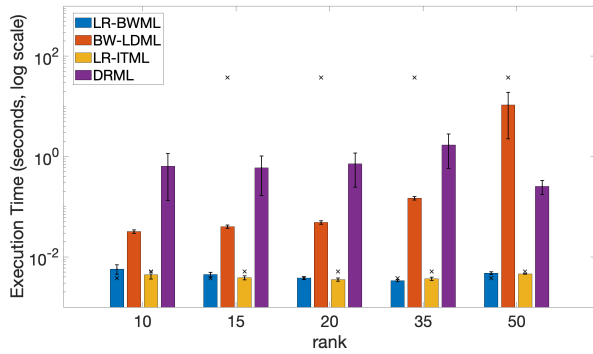
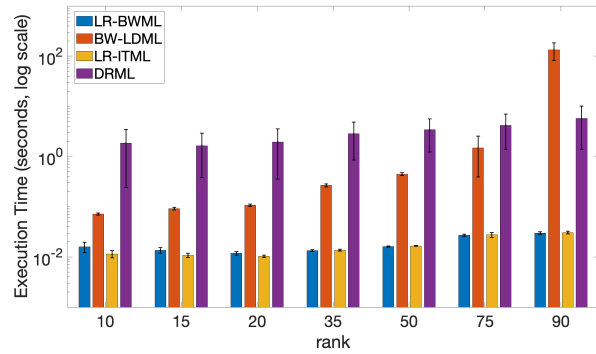
(a) Fixed Dimension: $m = n = 50$ (b) Fixed Dimension: $m = n = 100$

Figure 4.10: Timing of four algorithms with varying rank.

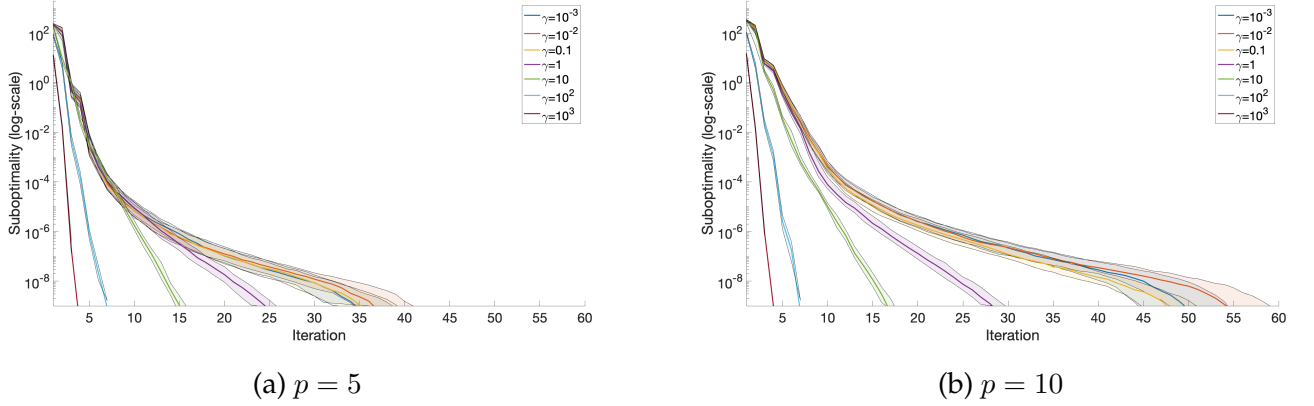


Figure 4.11: Convergence of BW-LDML on synthetic problems for varying γ , and number of constraints and dimension equal to 20.

Effect of Regularisation

Regularisation has effects on both convergence and timing, as seen in the previous chapter (§3.4.1). We now consider this effect for the low-rank algorithms. Each algorithm has a regularisation parameter with respect to an information theoretic divergence or the Bures-Wasserstein distance. In these tests the dimensionality and number of constraints are both fixed at 20 and the rank is varied between figures. Within each figure the regularisation parameter γ is then varied to produce a series of convergence plots.

In Figure 4.11 we see that as the regularisation parameter γ increases BW-LDML converges in fewer iterations. This can be understood as follows. As γ increases the regulariser term becomes more predominant in the objective and thus the solution moves closer to the initial point. On the other hand, for lower values of γ it appears that the algorithm's convergence is more variable. This indicates BW-LDML is converging to the unregularised solution of the logistic discriminant objective. It is interesting to see this interpolation behaviour between the two functions. The effect of rank on convergence is similar to previous cases, where it increases the number of iterations required for convergence.

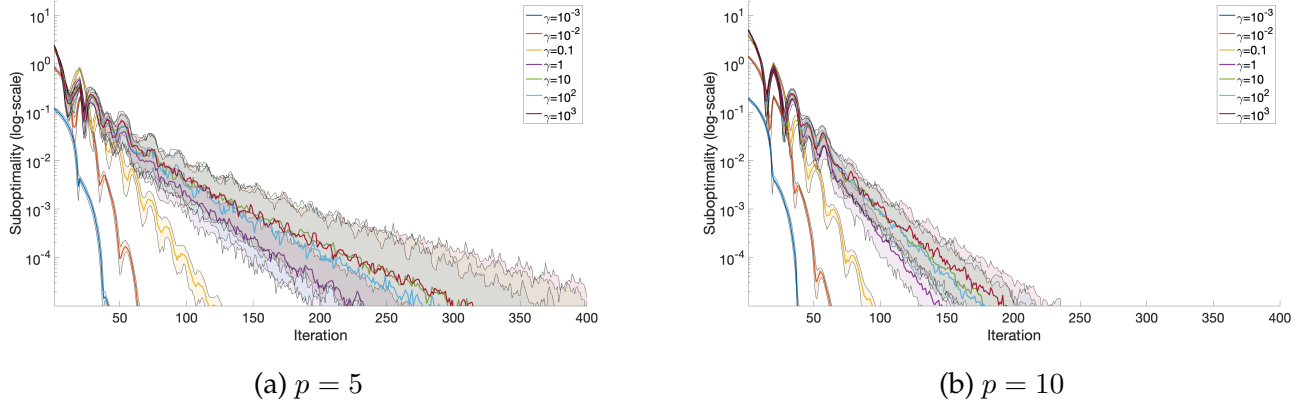


Figure 4.12: Convergence of LR-BWML on synthetic problems for varying γ , and number of constraints and dimension equal to 20.

The regularisation convergence results for LR-BWML are shown in Figure 4.12. The influence of γ is to trade off between the margin and matrix variables, in a similar way to BWML. For small values of γ it is more favourable to optimise the margin variables than the matrix variable, which presents a simpler problem, whereas large γ weights towards updating the matrix variable, which is a more complex problem. Between the two problems of different ranks the lower rank series takes more iterations to converge. Again, this is because of the lower dimensionality induced by the rank.

The regularisation convergence results for DRML are shown in Figure 4.13. A smaller value of γ causes an initial drop in the suboptimality followed by an interval of slower convergence. When γ is smaller the initial drop is larger, but the next region converges slower than if γ was larger. The faster convergence after the initial drop may be caused by the regulariser having a more consistent gradient than the logistic determinant terms.

Figure 4.14 shows the convergence for LR-ITML. A similar trade-off between ma-

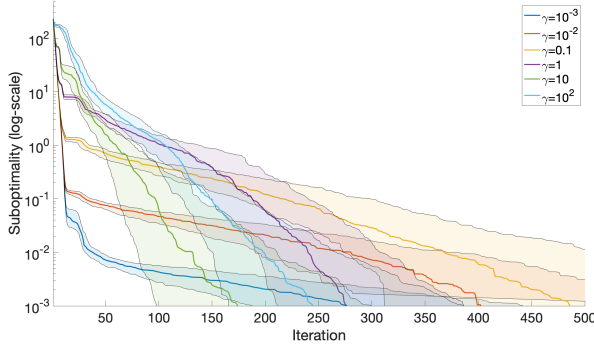
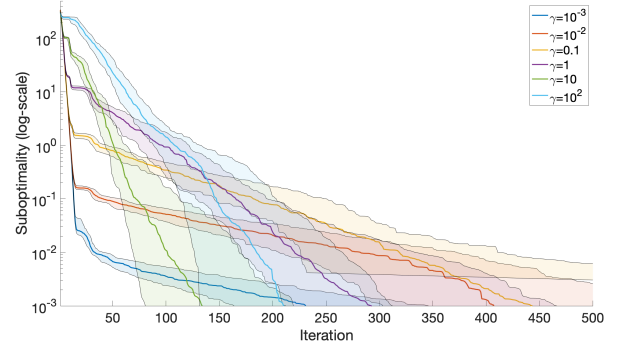
(a) $p = 5$ (b) $p = 10$

Figure 4.13: Convergence of DRML on synthetic problems for varying γ with fixed rank, and number of constraints and dimension equal to 20.

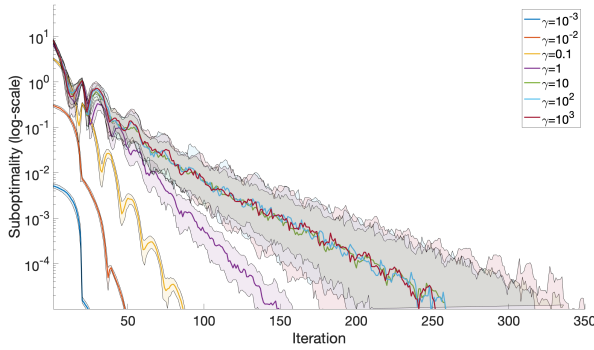
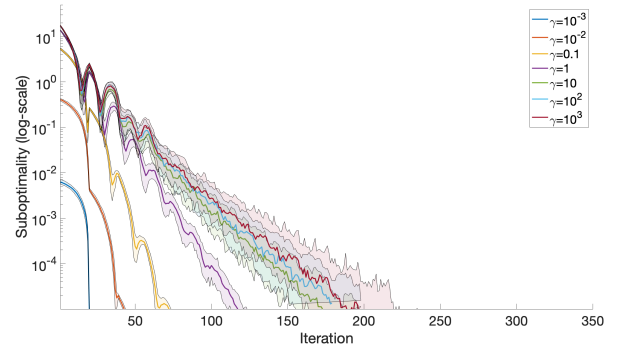
(a) $p = 5$ (b) $p = 10$

Figure 4.14: Convergence of LR-ITML on synthetic data for varying γ with fixed rank, and number of constraints and dimension equal to 20.

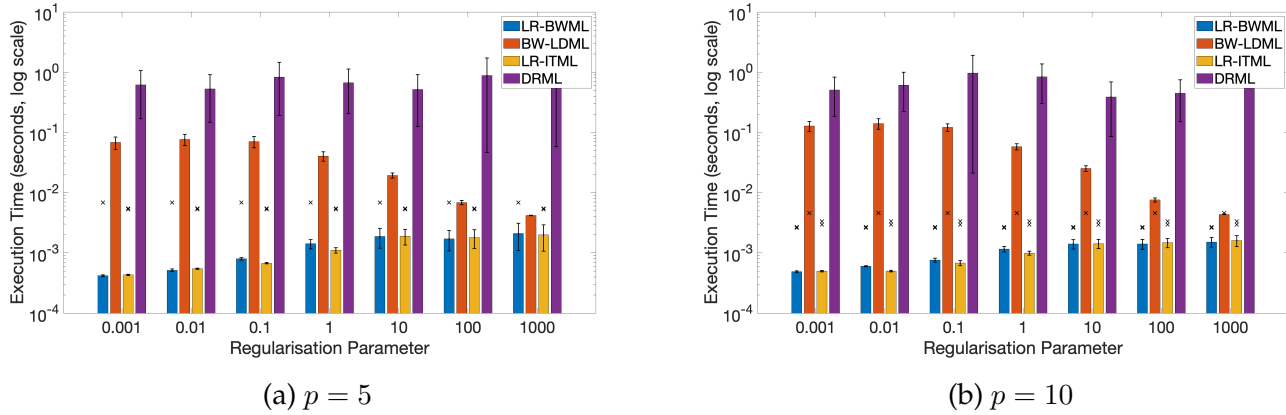


Figure 4.15: Timing performance of four algorithms for varying γ with fixed rank, and number of constraints and dimension equal to 20.

trix and margin variable is also visible in these convergence figures, where small γ favours the margin terms.

The timing comparisons for regularisation parameters are shown in Figure 4.15. For LR-BWML and LR-ITML the regularisation behaviour is essentially the same as the full-rank version. The case for BW-LDML is essentially reversed with larger γ executing faster (which is consistent with the convergence plots). The case for DRML is less clear as γ does not appear to have a significant effect on its mean execution time.

Overall, the effects noted for LR-BWML and LR-ITML with respect to constraints, dimension and regularisation parameter are similar to those found in the full-rank case. Interference drives the oscillations present during convergence. The effect of rank is the same as changing the dimensionality of the problem. This is not a surprise, as this is how these algorithms are designed, but it is worth being aware of. For BW-LDML, an increase in the number of constraints appears to decrease the rate of convergence, which is not unexpected as this amounts to an increase in overall complexity. Increases in rank also decrease the rate of convergence.

The convergence characteristics of DRML are less clear. Increases in dimensionality decrease rate of convergence, but the effect of rank is variable and the same goes for the number of constraints. The effect of regularisation parameter γ on BW-LDML is the opposite to the projection algorithms. For small γ LR-BWML and LR-ITML converge quickly and BW-LDML converges slowly and vice-versa for large γ , while DRML has a more complex behaviour with respect to γ . The projection algorithms converge in the least time, with very little margin between LR-BWML and LR-ITML. BW-LDML shows a substantial increases in execution time with rank whereas DRML is more consistent. However, BW-LDML is normally faster than DRML.

Now we proceed with tests on the UCI datasets to understand how these algorithms function on real data. However, we keep the scale modest enough to be able to perform replicate experiments to account for statistical variation.

4.4.2 UCI Datasets

These UCI dataset experiments use the same test parameters as the full-rank UCI experiments, i.e., $4c(c-1)$ pairs of dissimilarity and similarity constraints, 2 folds and 5 repeats. However, the additional variable of rank for these tests changes the nature of these experiments. We perform classification on the Ionosphere, Spambase and Wine datasets for these experiments. Balance-scale and Iris are not included because they don't show the effect of rank, since both have dimension 4 whereas Ionosphere, Spambase and Wine have 34, 57 and 13 dimensions respectively. The choices of rank that are presented for each dataset are necessarily different, but the full-rank case is included in each figure and so is rank equal to two. Wasserstein Discriminant Analysis (WDA) is included in these tests for comparison with how another Optimal Transport algorithm performs. WDA solves for an element of the Stiefel manifold $St(p, n)$. When the problem is full-rank this manifold is equivalent to the orthogonal matrices. Orthogonal matrices do not alter distances in a Generalised Mahalanobis distance, making them insufficient

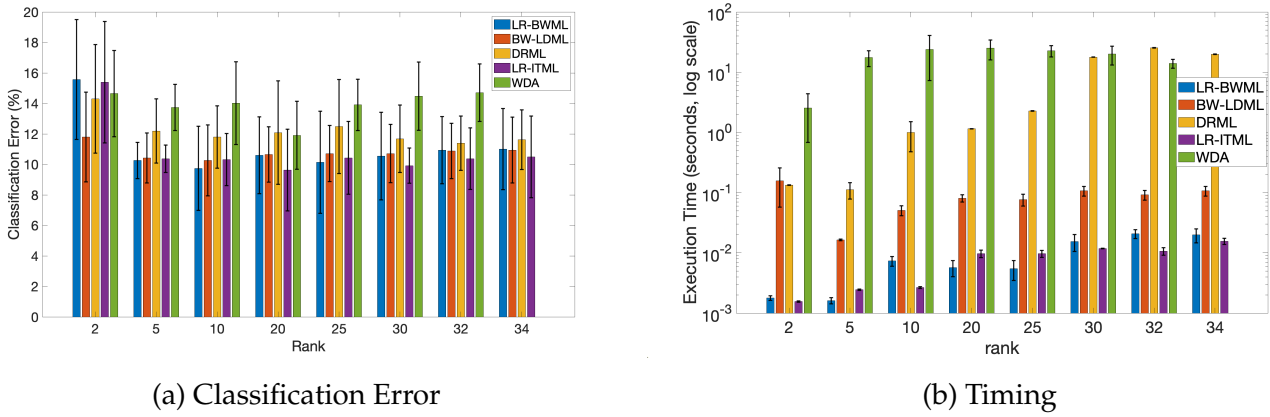


Figure 4.16: Results of k -NN classification on Ionosphere with variable rank for Metric Learning algorithms.

for learning metrics. Therefore we do not include WDA when learning a full-rank metric (i.e. $p = d$) in the figures we present. Similar to the choice for M_0 and Y_0 in the synthetic tests, PCA was computed on each $x_i - x_j$ from the similarity and dissimilarity pairs. The resulting $W \in St(p, n)$ was then used as $Y_0 = W$ or $M_0 = WW^T$. The figures presented here focus on one dataset at a time, demonstrating how either the timing or classification error vary with the target rank. Similar to the previous UCI figures, the mean of the k -NN classification error and fit time for each algorithm is presented with standard deviations as error bars. The regularisation parameter γ , maximum number of iterations and stopping condition tolerance were subject to a grid search to find the lowest classification error for each algorithm.

For the classification on Ionosphere in Figure 4.16a, it seems that as rank decreases, the classification error of each algorithm is fairly constant with respect to the error bars shown. The exception to this is for rank equal to two, where the error rises noticeably for LR-BWML, DRML and LR-ITML. BW-LDML and WDA appear to suffer less because of this rank reduction, but WDA has a higher classification error to begin with. LR-BWML, BW-LDML, DRML and LR-ITML seem

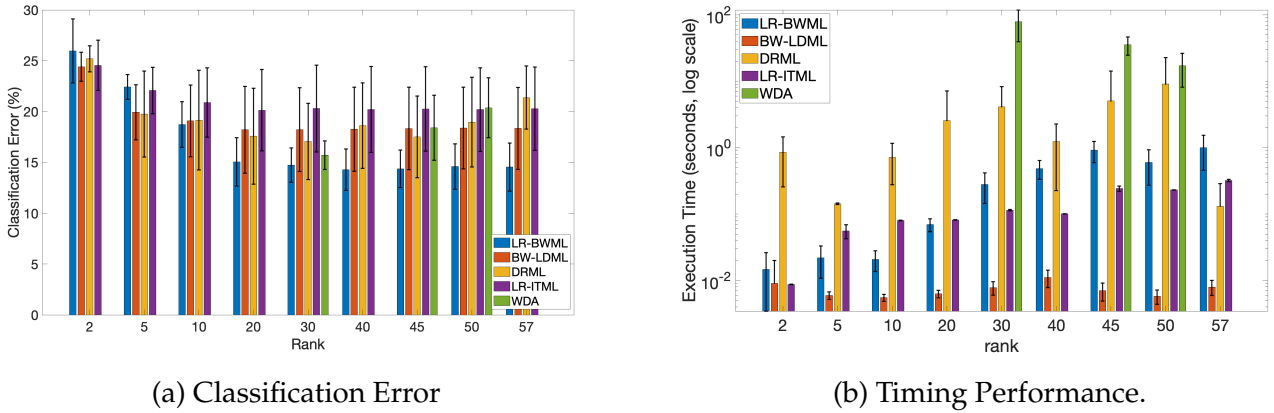


Figure 4.17: Results of k -NN classification on Spambase with variable rank for Metric Learning algorithms.

to have similar classification performance (aside from rank equal to two, while WDA maintains the worst error on these higher ranks).

In Figure 4.16b the timing results for Ionosphere are presented. The timing results are much more varied than the error results in this case. Generally speaking, the execution time decreases as the rank decreases. However, in some cases the execution time increases when the rank decreases. This is the most predominant in comparing rank two and rank five. Most noticeably the execution time for BW-LDML rises significantly and LR-BWML and DRML experience marginal increases. LR-BWML and LR-ITML appear to be the fastest algorithms followed by (in order) BW-LDML, DRML and WDA.

In Figure 4.17a the classification error results for Spambase are presented. The effect of rank on the classification error for Spambase exhibits a more gradual increase in the error with decreasing rank. LR-BWML has good error performance at higher rank, but does not maintain this comparative advantage at lower ranks. WDA does not appear in many of the values of rank as experiments would not converge in a realistic timeframe. For the few values that are shown, WDA does

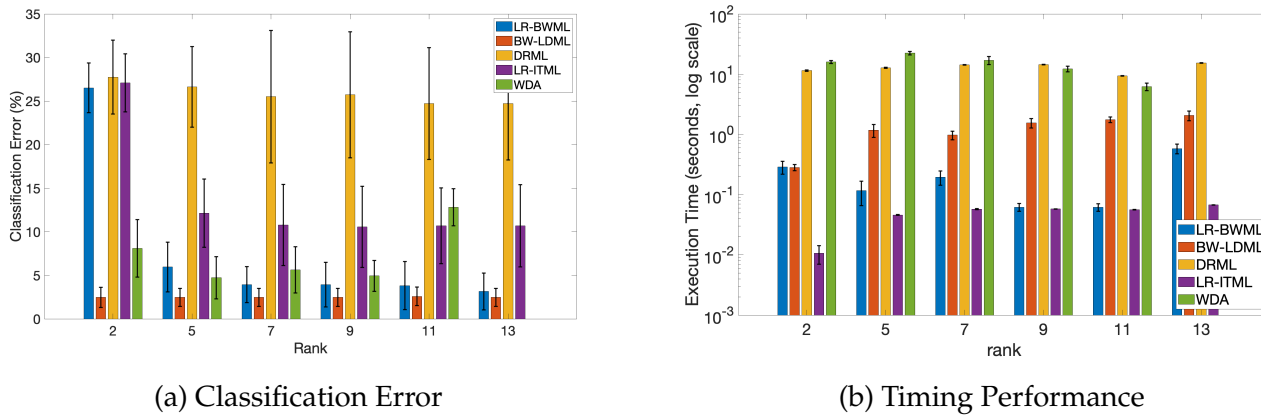


Figure 4.18: Results of k -NN classification on Wine with variable rank for Metric Learning algorithms.

have comparatively good performance. BW-LDML and DRML have similar values for classification error over the different values of rank.

In Figure 4.17b the timing results for Spambase are presented. BW-LDML appears to be consistently the fastest algorithm over the different values of rank. Its execution time also does not have a large amount of variability between different ranks. LR-BWML on the other hand exhibits a decrease in execution time with reduced rank. LR-ITML has a similar trend, but it is not as consistent. The execution time for DRML is much more variable. However, the error bars indicate that there is a significant amount of variance occurring in the execution time, which will makes a trend harder to observe. WDA is the slowest algorithm and the results indicate that the execution time would be larger for smaller values of rank, which is consistent with its inability to converge.

In Figure 4.18a the classification error results for Wine are presented. LR-BWML and BW-LDML have the lowest error rates for each value of rank (with the exception of rank equal to two for LR-BWML). In this case BW-LDML appear to be less sensitive to rank than BWML. WDA also appears to show a good error rate for the

smaller ranks. LR-ITML maintains a moderate error rate, while DRML performs the worst.

In Figure 4.18b the timing results for Wine are presented. DRML and WDA are the slowest algorithms followed by BW-LDML. LR-ITML is consistently the fastest algorithm, followed by LR-BWML.

Overall, LR-BWML and BW-LDML have the lowest classification error and LR-BWML and LR-ITML are the fastest algorithms. BW-LDML appears to be the least sensitive to changes in rank. It appears that decreasing rank does *not* increase classification error, perhaps with the exception of WDA in a small number of cases. The main advantage conferred by reducing rank is to decrease the complexity and therefore the execution time of the algorithm. Comparing the projection methods against the logistic-discriminant methods it appears that on average the projection methods perform better in terms of error and execution time. There is a noticeable difference in timing behaviour between the synthetic tests and the UCI tests. This is apparent in the less consistent reduction in execution time with respect to reduction in rank. This may be because the UCI datasets are necessarily more complex and by reducing the rank this complexity is compounded.

4.4.3 Image Datasets

Following the full-rank tests on MNIST and Fashion-MNIST from the previous chapter, we present experiments on LR-BWML, BW-LDML, LR-ITML, DRML and WDA for learning a distance for k -NN classification on these image datasets. In keeping with the UCI datasets, $k = 4$ nearest neighbours are used in these tests, and the same initialisation procedure for Y_0 and M_0 was used. A grid search was used to determine the best parameters for each algorithm, as for the UCI experiments. We choose not to compute replicate experiments for an average or error bars due to the increased computational cost of the problem. For training and testing the respective training and testing sets for MNIST and Fashion-MNIST were

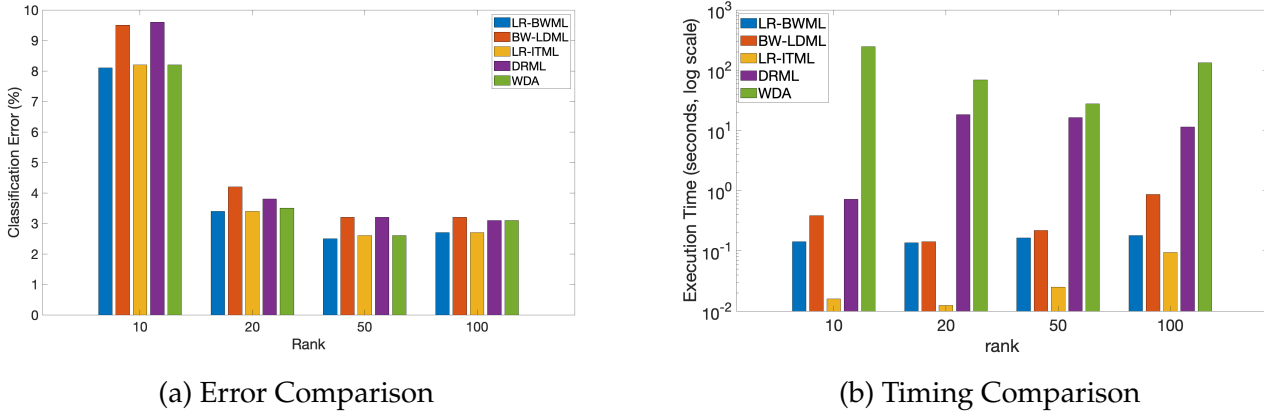


Figure 4.19: Results of k -NN classification on MNIST with variable rank for Metric Learning algorithms.

used and 90000 similarity and dissimilarity pairs were used to train each algorithm.

In Figure 4.19 we see the results of the MNIST experiments. It is clear that as the rank is reduced the classification error for each algorithm increases, which is consistent with the UCI results. This is most significant for rank equal to 10, but there does appear to be a modest increase in error overall for rank equal to 20. LR-BWML, LR-ITML and WDA tend to achieve the best error for the different values of rank where BW-LDML and DRML generally have a higher error. LR-ITML is at least an order of magnitude faster than the other algorithms in most cases. LR-BWML is the next fastest algorithm, however its timing seems to be fairly constant with respect to rank. BW-LDML is similar in timing to LR-BWML, but it is the slower of the two. DRML is an order of magnitude slower than LR-BWML and BW-LDML in most cases. WDA is by far the slowest algorithm in each case.

For a visualisation, a twenty dimensional embedding of MNIST produced by each of the algorithms is then embedded in two dimensions using t-SNE [154]. The resulting embeddings are presented in Figure 4.20. The same embedding us-

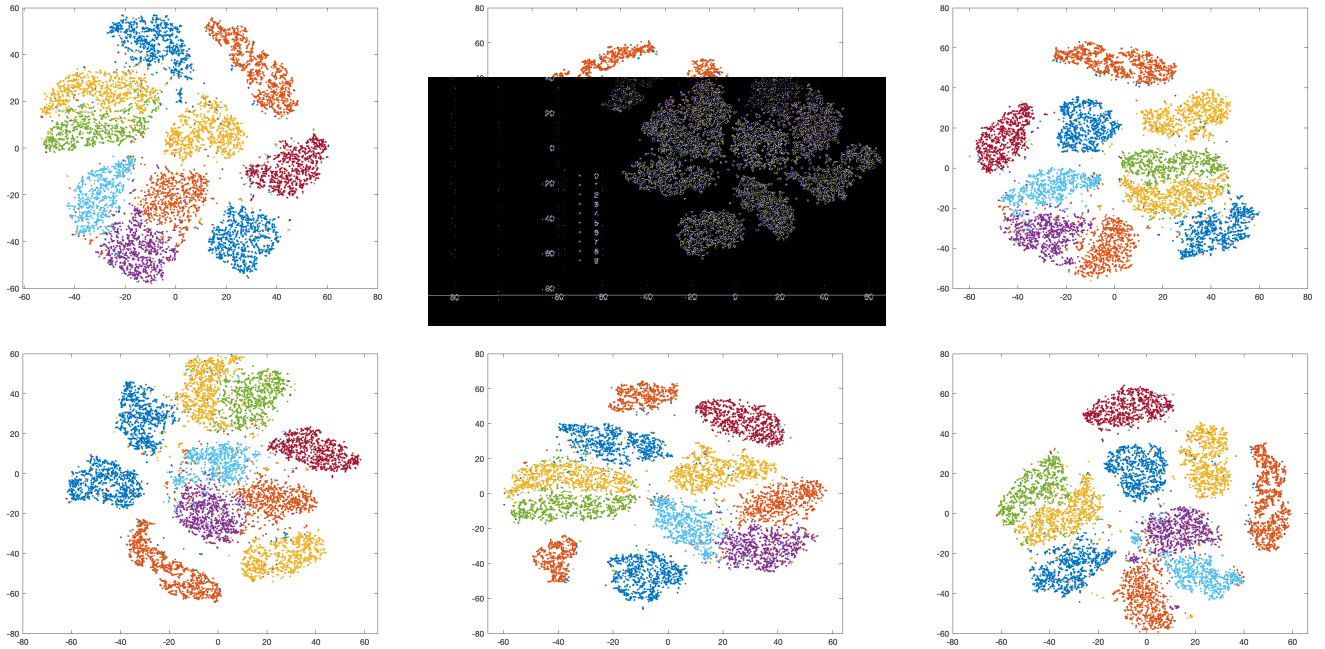


Figure 4.20: t-SNE Embeddings of MNIST after dimensionality reduction to 20 dimensions using learnt metrics.

1st row left to right: LR-BWML, BW-LDML, LR-ITML.

2nd row left to right: DRML, WDA, PCA.

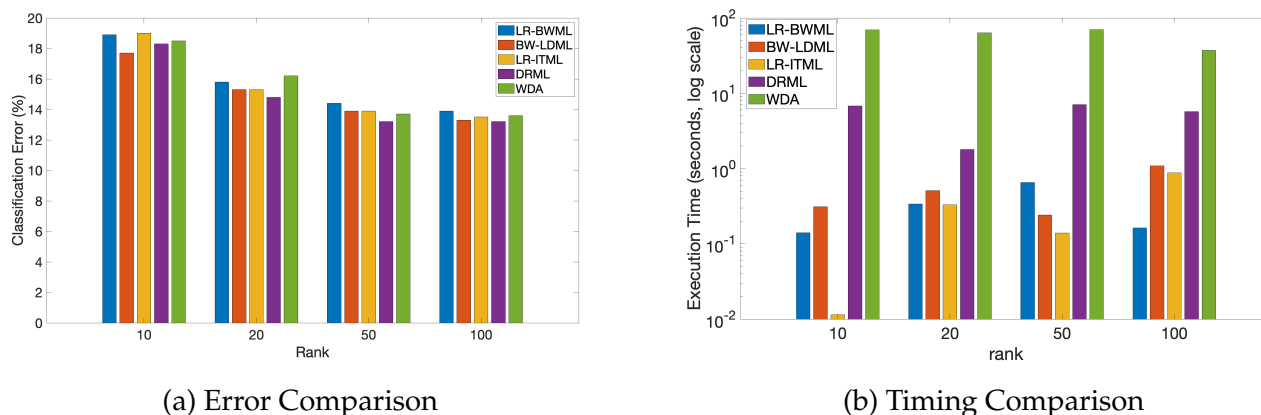


Figure 4.21: Results of k -NN classification on Fashion-MNIST with varying rank for Metric Learning algorithms.

ing PCA is included as this is the initial point used in each algorithm. The most striking feature of these embeddings is how similar each of them are. Overall the structure of the data is largely unchanged, with the relative position of each cluster being consistent with respect to its neighbours. A subtle difference between the figures is the separation at the class boundaries. Improvements can be made at these interfaces and this appears to be the one case that distinguishes the algorithms. A good example is at the interfaces of the 1, 3 and 5 clusters (orange, purple and light blue). For LR-BWML, LR-ITML and WDA it appears that there is less leakage and better definition at these boundaries compared to the other algorithms.

The classification error and timing performance on Fashion-MNIST is shown in Figure 4.21. The classification error for each algorithm on Fashion-MNIST is higher than that of MNIST, which is consistent with Fashion-MNIST being a more complex dataset. A trend similar to the MNIST results of decreasing error with increasing rank is seen here. It appears that DRML attains the lowest error over the different values of rank except for rank equal to 10. However, the differences in classification error are generally less than 1%, similar to MNIST. The timing be-

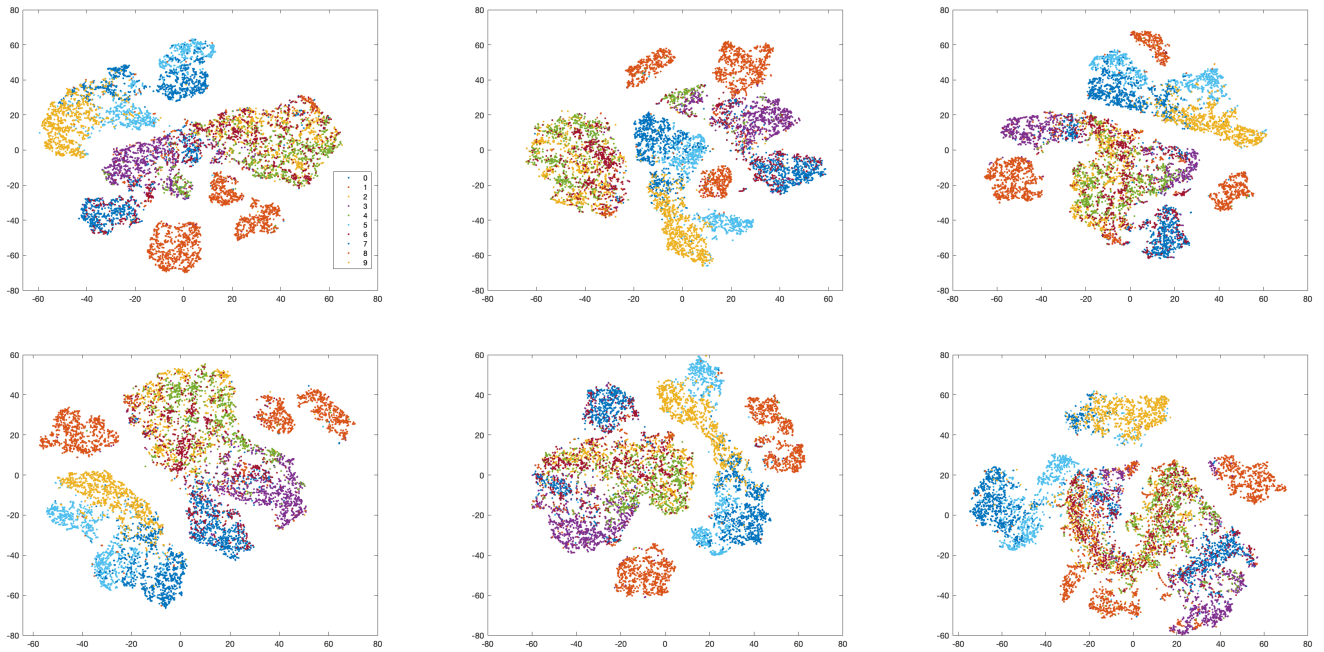


Figure 4.22: t-SNE Embeddings of Fashion MNIST after dimensionality reduction to 20 dimensions using learnt metric.

1st row left to right: LR-BWML, BW-LDML, LR-ITML.

2nd row left to right: DRML, WDA, PCA.

haviour for Fashion-MNIST is less consistent than the MNIST results.

The same embedding process is repeated with Fashion-MNIST in Figure 4.22. It is immediately apparent why the error rate for Fashion-MNIST is higher when considering these embeddings. The class boundaries are much less defined and it appears that some clusters overlap each other. Also in these figures the relative structure for each of these figures does not seem to be as consistent as in Figure 4.20.

Overall, the classification performance of the projection algorithms seems to be on par with the Riemannian optimisation methods, but with a reduced execution

time.

4.5 Conclusion

This chapter introduced two algorithms for learning low-rank Generalised Mahalanobis metrics: LR-BWML and BW-LDML. We introduce these low-rank Metric Learning algorithms aim to increase the scalability of Metric Learning using the Bures-Wasserstein distance. They use two different ways of achieving this scalability. One of them essentially reduces the dimension of the problem and the other operates on a low-rank manifold directly.

On synthetic data we saw that these algorithms converged reliably in practice. Many of the effects seen with synthetic data are similar to those seen in the previous chapter. For UCI classification tasks, the effect of rank is mixed. When rank is reduced, the error increases and the execution time generally decreases. The increase in error indicates that higher rank matrices capture some useful components of the data. BW-LDML copes with a reduction in rank better than the other algorithms, perhaps because it has comparatively more capacity. On the image datasets the effect of reducing the rank is the same as was seen in the UCI experiments. There is some reduction in the execution time, but it appears that the constraints drive the complexity as opposed to dimensionality of the metric. Overall, LR-BWML gives better performance, but BW-LDML seems to be reliable over a wider range of choices of rank. The concern of scalability is somewhat addressed by these algorithms, but they are still lacking. To complement learning low-rank metrics, methods that address scalability in the size of the training data are necessary.

Thus far we have only considered learning metrics on vector data, but this is not the only option. We next consider learning metrics on SPD data using the Bures-Wasserstein distance. Since the Bures-Wasserstein distance is a metric on SPD matrices it is well-positioned for this task. However, it needs to be parameterised

to adapt it to the data provided.

Chapter 5

Symmetric Positive Definite Matrix Metric Learning

5.1 Introduction

So far in this thesis we have studied algorithms for learning metrics based on the Generalised Mahalanobis distance. However, the Generalised Mahalanobis distance is a distance on Euclidean space and there are types of data that are not suited to Euclidean geometry. Symmetric Positive Definite (SPD) matrices are an example of this. Examples datasets such as brain imaging data [82, 23], textures [21] and image sets [26] are either composed of SPD matrices or have advantageous properties when expressed as such. There are many distances on the SPD manifold [82, 155, 23, 156] with the Bures-Wasserstein distance being one such distance.

We therefore study how to learn distance metrics on the SPD matrix manifold using the Bures-Wasserstein distance. But in order to learn a metric on this manifold, a parameterised distance must be specified. We consider two such families of metric. The first is the Generalised Bures-Wasserstein distance [55], which generalises the Bures-Wasserstein distance in a similar way that the Mahalanobis distance

generalises the l_2 norm. The second is a generalisation of the Bures-Wasserstein distance for learning a ground metric, which is the underlying distance used to compute the Optimal Transport distance. We refer to this as Ground Metric Bures-Wasserstein or GM-BW.

The expression for the Generalised Bures-Wasserstein distance (GBW) between SPD matrices $X, Y \in S_+^n$ is:

$$\begin{aligned} \mathcal{GB}_M^2(A, B) &= \sqrt{\text{trace}(MA + MB - 2(A^{\frac{1}{2}}MBMA^{\frac{1}{2}})^{\frac{1}{2}})} \\ &= \min_{U \in \mathcal{O}(n)} \|A^{\frac{1}{2}} - B^{\frac{1}{2}}U\|_M. \end{aligned} \quad (5.1)$$

where $M \in S_{++}^n$ parameterises the distance. Notice that $M = I$ reduces (5.1) to the Bures-Wasserstein distance $\mathcal{B}^2(X, Y)$. Han *et al.* [55] verify that this is a formal distance and provide several example applications, including a Metric Learning formulation, which we refer to as Generalised Bures-Wasserstein Metric Learning or GBWML.

Our first algorithm is a modification of BWML, which instead uses the Generalised Bures-Wasserstein distance to describe constraints on SPD matrices. We refer to this adaptation of BWML to SPD data as SPD-BWML. The relationship between SPD-BWML and Log-Euclidean Metric Learning (LEML - Problem (2.20)) is analogous to the one between BWML and ITML. Given the analogous nature of BWML and ITML and their relative performance, we expect to find a similar relationship between LEML and SPD-BWML.

Using the ground metric formulation of the Bures-Wasserstein distance we introduce Ground Metric Bures-Wasserstein Metric Learning or GM-BWML. This algorithm treats the ground metric in GM-BW as a trainable parameter. The choice of learning a ground metric for the Bures-Wasserstein distance draws inspiration from Ground Metric Learning of Cuturi and Avis [62] (Problem (2.22)). They find improved classification performance when learning the ground metric for histogram data and we would like to do the same for the Bures-Wasserstein distance.

The set of metrics on SPD matrices available to GM-BW is larger than that of the generalised Bures-Wasserstein distance, which we hope allows for better tailored metrics. The GM-BW does not have a closed-form for an arbitrary ground metric and is formulated as an optimisation problem. The most straightforward method to evaluate this is by solving the optimisation problem with an off-the-shelf solver.

As an alternative, we derive an approximation to the GM-BW reminiscent of the Sinkhorn distance. We refer to this problem as Sinkhorn Ground Metric Bures-Wasserstein or S-GM-BW. The Metric Learning algorithm associated with this approximation is accordingly referred to as S-GM-BWML and we adapt standard solvers to this problem. The Sinkhorn variation has been considered because typically the cost of computing the Optimal Transport distance with respect to a linear program impedes the scalability. In practice, approximating the Optimal Transport distance with the Sinkhorn distance has seen significant successes and we hoped the same may be true for Sinkhorn GM-BW.

The convergence properties of S-GM-BW are investigated with two optimisation algorithms in section 5.4.1. These tests were performed with the intention of selecting one of the optimisation algorithms to run S-GM-BWML.

Convergence tests on GM-BWML and S-GM-BWML are performed in section 5.4.2, with GBWML included for comparison. SPD-BWML is not included in these tests as we have already established its convergence properties in sections 3.4.1 and 4.4.1.

The k -NN classification performance of these algorithms was tested on four datasets, with results presented in section 5.4.3. MNIST and Fashion-MNIST are adapted to the SPD Matrix manifold using Covariance Descriptors [80, 21]. We have included these datasets to see how they perform compared to the previous methods. The other two datasets are ETH-80 [157] and JHMDB [158]. The training data used for JHMDB is considerably larger than the other tests and is included to see how our

algorithms perform on higher dimensional data.

GBWML, Log-Euclidean Metric Learning (LEML) and Information-Divergence Dictionary Learning [132] (IDDL - Problem 2.21) are also included in the k -NN performance tests for relevant comparison. GBWML is the most similar to our algorithms and provides the best comparison for their performance. LEMML is ITML [12] adapted to learning a metric on the Log-Euclidean distance, which forms the basis for the inspiration of SPD-BWML, and is thus a good algorithm or comparison. IDDL is a dictionary learning algorithm that learns an information-theoretic divergence on dictionary atoms for classification, and is included as another information-theoretic algorithm trained on SPD data.

Our experiments show that while GM-BWML and S-GM-BWML converge reliably, they are not useful for classification. We find that the SPD-BWML has competitive classification performance with the comparison algorithms, but has higher training and testing times. We notice that the S-GM-BW seems to evaluate faster than the GBW and may be useful at evaluation time in future applications.

5.2 Methods

Our two Metric Learning methods for learning in the SPD Bures-Wasserstein setting are now introduced. The first algorithm (§5.2.1) SPD-BWML presents a Cyclic Projections-based alternative to GBWML, and potentially gains some of the benefits we have seen from this approach.

The second algorithm (§5.2.2) increases the expressive capacity of the metrics with respect to the Bures-Wasserstein distance. However, this approach comes at the cost of increased computational complexity.

5.2.1 SPD - Bures-Wasserstein Metric Learning

Here we show how to extend BWML to SPD matrices using the Generalised Bures-Wasserstein Distance. To do this the Generalised Bures-Wasserstein distance is substituted into the constraints for BWML in Problem (3.2), with distances between similar or dissimilar $X_i, X_j \in S_{++}^n$ replacing $x_i, x_j \in \mathbb{R}^n$. This results in the following problem:

$$\begin{aligned} \min_{M \in S_{++}^n, \xi \in \mathbb{R}_{++}^m} \quad & \mathcal{B}^2(M, M_0) + \gamma \mathcal{B}^2(\text{diag}(\xi), \text{diag}(\xi_0)) \\ \text{s.t.} \quad & \mathcal{GB}_M^2(X_i, X_j) \leq \xi_{ij}, (i, j) \in \mathcal{S} \\ & \mathcal{GB}_M^2(X_i, X_j) \geq \xi_{ij}, (i, j) \in \mathcal{D}. \end{aligned} \quad (5.2)$$

To express this problem in an analogous form to BWML we see that the Generalised Bures-Wasserstein distance can be computed as:

$$\begin{aligned} \mathcal{GB}_M^2(X_i, X_j) &= \min_{U \in \mathcal{O}(n)} \|X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U\|_M^2 \\ &= \min_{U \in \mathcal{O}(n)} \text{trace}((X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)^T M (X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)) \\ &= \min_{U \in \mathcal{O}(n)} \langle (X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)(X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)^T, M \rangle \end{aligned}$$

according to Equation (5.1), where the optimal U is equal to the polar factor of $X_j^{\frac{1}{2}} M X_i^{\frac{1}{2}}$. If the value of U is calculated, then the form of the constraints in Problem (5.2) are linear. Therefore, the existing BWML algorithm may be used except instead of $X_{ij} = (x_i - x_j)(x_i - x_j)^T$ for the Generalised Mahalanobis distance, $X_{ij} = (X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)(X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)^T$, based on the Generalised Bures-Wasserstein distance.

Consequently, the update equation for SPD-BWML in terms of the variable $Z_t = M_t^{\frac{1}{2}}$ is:

$$Z_{t+1} = Z_t + \delta_{ij} \frac{\alpha Z_t M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} Z_t}{1 - \delta_{ij} \alpha \text{trace}(M_0^{-\frac{1}{2}} X_{ij} M_0^{-\frac{1}{2}} Z_t)}. \quad (5.3)$$

The solution for α in equation (3.10) is updated for the full-rank matrix terms so that $p = \text{trace}(X_{ij} Z_t^2)$ and $q = \text{trace}(X_{ij} Z_t)$. The Cyclic Projections Algorithm for

SPD-BWML is provided in Algorithm 11.

Even though we have expressed the constraints linearly, this is an approximation of the constraints presented in Problem (5.2), since U still has a dependence on M . Empirically, we have observed that making these approximations in SPD-BWML still results in the increase or reduction of distances with respect to similarity and dissimilarity pairs.

When substituting a full-rank matrix into the Cyclic Projections updates, the Sherman-Morrison identity is no longer valid. The effect of this is that the matrix update will no longer project the matrix onto the desired set. However, from empirical observation it appears that the update equation (5.3) decreases the distance between similar pairs and increases the distance between dissimilar pairs, as desired. The specific values of the margins are determined heuristically, so it is not necessary that the algorithm precisely projects the metric parameter M . We have made the same observation of LEML, where its authors claim that they perform a projection, but they in fact do not. However, LEML is a competitive algorithm, which suggests this property is not essential.

To reduce computational burden at run time the $X_i^{\frac{1}{2}}$ terms are precomputed. We also investigated how to reduce the computational burden of computing U with each projection. While the analytic solution for U is: $X_j^{\frac{1}{2}} M X_i^{\frac{1}{2}} (X_i^{\frac{1}{2}} M X_j M X_i^{\frac{1}{2}})^{-\frac{1}{2}}$, computing the polar factor of $X_j^{\frac{1}{2}} M X_i^{\frac{1}{2}}$ using the SVD is faster in practice. This is achieved via:

$$U = QP^T \text{ s.t. } X_j^{\frac{1}{2}} M X_i^{\frac{1}{2}} = PSQ^T, \quad (5.4)$$

where P, Q are the left and right singular vectors respectively and S is the matrix of singular values. The shorthand for the above expression is $\text{polarFactor}(X_j^{\frac{1}{2}} M X_i^{\frac{1}{2}})$.

Algorithm 11: SPD - Bures-Wasserstein Metric Learning

Input: Dataset: $\{X_i \in S_{++}^n\}_{i=1}^N$

Similarity and Dissimilarity indexes: $\mathcal{S}, \mathcal{D} \subseteq [N] \times [N]$

Initial Matrix: $M_0 \in \mathbb{R}^{n \times n}$, Initial Margins: $\xi_0 \in \mathbb{R}^{|\mathcal{S}|+|\mathcal{D}|}$, Regularisation

Parameter: $\gamma \in \mathbb{R}_+$, Convergence tolerance: $\epsilon \in \mathbb{R}_+$

1. $Z_0 \leftarrow M_0^{1/2}, \xi \leftarrow \xi_0, \lambda_0 \leftarrow 0$

2. **do**

(a) Sample $(i, j) \in \mathcal{S} \cup \mathcal{D}$ uniformly at random

(b) $\delta \leftarrow 1$ if $(i, j) \in \mathcal{S}$, -1 otherwise (to decide on the $\pm$ choices)

(c) $U \leftarrow \text{polarFactor}(X_j^{\frac{1}{2}} M_t X_i^{\frac{1}{2}})$

(d) $X_{ij} \leftarrow (X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)(X_i^{\frac{1}{2}} - X_j^{\frac{1}{2}} U)^T$

(e) $p \leftarrow \text{trace}(X_{ij} Z_t^2)$

(f) $q \leftarrow \text{trace}(X_{ij} Z_t)$

(g) Solve equation (3.10) for α

(h) $\alpha \leftarrow \min(\alpha, \lambda_{t,ij})$

(i) $\lambda_{t+1,ij} \leftarrow \lambda_{t,ij} - \alpha$

(j) set Z_{t+1}, ξ_{t+1} using equations (5.3) and (3.9) respectively

(k) $t \leftarrow t + 1$

while $\|\lambda_t - \lambda_{t+1}\| \geq \epsilon$

5.2.2 Ground Metric Bures-Wasserstein Metric Learning

We can also learn a generalisation of GBWML by providing the Generalised Bures-Wasserstein distance with a higher degree of expressivity. To begin with, the Bures-Wasserstein distance between SPD matrices $X_i, X_j \in S_{++}^n$ is expressed as an optimisation problem, as in Problem (4.3):

$$\begin{aligned} \min_{Z \in S_{++}^{2n}} \quad & \langle C, Z \rangle \\ \text{s.t.} \quad & P_0^T Z P_0 = X_i \\ & P_1^T Z P_1 = X_j. \end{aligned} \tag{5.5}$$

where

$$C = \begin{bmatrix} I & -I \\ -I & I \end{bmatrix} \quad P_0 = \begin{bmatrix} I \\ \mathbf{0}_n \end{bmatrix} \quad P_1 = \begin{bmatrix} \mathbf{0}_n \\ I \end{bmatrix}.$$

Similarly, the Generalised Bures-Wasserstein distance used in GBWML can be expressed as an instance of Problem (5.5) where:

$$C = \begin{bmatrix} M & -M \\ -M & M \end{bmatrix},$$

for a matrix $M \in S_{++}^n$. The solution to Problem (5.5) is expressed using the shorthand $\mathcal{B}_{GM}^2(X_i, X_j; C)$, which is the Ground Metric Bures-Wasserstein or GM-BW between X_i and X_j .

Our intention is to learn $C \in S_{++}^{2n}$ in a supervised way using similarity and dissimilarity pairs with respect to $\mathcal{B}_{GM}^2$. We refer to this approach as Ground Metric Bures-Wasserstein Metric Learning or GM-BWML. In both the above examples, C has rank equal to n and can be factorised as $C = Y^T Y$, where $Y \in \mathbb{R}^{n \times 2n}$. We incorporate this into our algorithm and train Y such that $C = Y^T Y$, reducing the dimensionality of the problem.

As with BW-LDML and GBWML, we use a sum of logistic-discriminant terms for the GM-BWML objective, so the optimisation problem for GM-BWML is:

$$\min_{Y \in \mathbb{R}^{n \times 2n}} \sum_{(i,j) \in S \cup \mathcal{D}} [p_{ij}^* \log(p_{ij}(Y)) + (1 - p_{ij}^*) \log(1 - p_{ij}(Y))] + \gamma \|Y - Y_0\|_F^2, \quad (5.6)$$

where

$$p_{ij}(Y) = (1 + \exp(1 - \mathcal{B}_{GM}^2(X_i, X_j; Y^T Y)))^{-1},$$

$s_{ij} = \pm 1$ encodes the similarity or dissimilarity of X_i and X_j , p_{ij}^* is the ideal classifier and the Frobenius norm has been included as a regularisation term. We choose this formulation as it makes the effect of using GM-BW instead of GBW clearer. If there is an advantage to the GM-BW it will appear in the performance of GM-BWML relative to GBWML.

Similar to BW-LDML, we choose to optimise $Y \in \mathbb{R}^{n \times 2n}$ on the manifold of fixed-rank semidefinite matrices $S_+(n, 2n)$ to avoid problems with the non-convexities associated with the factorisation $Y^T Y$. Riemannian L-BFGS is chosen as the optimiser for this problem since it is an efficient choice for large-scale problems.

To our knowledge, there is no closed-form solution to Problem (5.5) with an arbitrary C , so the solution needs to be computed by some optimisation algorithm. Accordingly, we develop two variants of GM-BWML. The first uses SDPT3 [159], a linear semidefinite programming algorithm that directly solves the semidefinite problem, to evaluate $\mathcal{B}_{GM}^2(X_i, X_j; C)$. This is denoted as GM-BWML.

The second method approximates a solution to $\mathcal{B}_{GM}^2$ using a method inspired by the Sinkhorn distance [68] (Problem (2.13)). We call this algorithm Sinkhorn Ground Metric Bures-Wasserstein or S-GM-BW, and denote by S-GM-BWML the variant of GM-BWML that uses S-GM-BW in place of GM-BW.

The approach for the Sinkhorn distance on a discrete optimal transport problem is to include an entropic regulariser in the optimisation problem with a regularisation parameter ϵ . We wish to do the same for S-GM-BW. The log-determinant

function is a good candidate for an entropic regulariser to the objective term. However, C is typically low rank, so the corresponding regulariser $\log \det(CZ)$ will not be defined.

To circumvent this we add ϵI to C to ensure it is full rank. This gives the problem:

$$\begin{aligned} \min_{Z \in S_{++}^{2n}} \quad & \langle C + \epsilon I, Z \rangle \\ \text{s.t.} \quad & P_0^T Z P_0 = X_i \\ & P_1^T Z P_1 = X_j, \end{aligned} \tag{5.7}$$

which, for small ϵ , has approximately the same value for the objective as Problem (5.5).

We then introduce the entropic regulariser $\epsilon \log \det((\frac{1}{\epsilon}C + I)Z)$ on Z . As $\epsilon \rightarrow 0$ then $\epsilon \log \det((\frac{1}{\epsilon}C + I)Z) \rightarrow 0$ so we may use it in the objective term in Problem (5.7) for our approximation. Dividing both terms in the objective by ϵ leaves the optimum unchanged, and the problem becomes:

$$\begin{aligned} \min_{Z \in S_{++}^{2n}} \quad & \langle (\frac{1}{\epsilon}C + I), Z \rangle - \log \det((\frac{1}{\epsilon}C + I)Z) \\ \text{s.t.} \quad & P_0^T Z P_0 = X_0 \\ & P_1^T Z P_1 = X_1. \end{aligned} \tag{5.8}$$

This problem is the S-GM-BW and has the log-determinant divergence as its objective and independent linear constraints on Z . We denote the solution to Problem (5.8) as $\mathcal{B}_{GM}^2(X_i, X_j; Y^T Y, \epsilon)$. We similarly define S-GM-BWML using the same objective function as GM-BWML, except using:

$$p_{ij}(Y) = (1 + \exp(1 - \mathcal{B}_{GM}^2(X_i, X_j; Y Y^T, \epsilon)))^{-1},$$

as the logistic discriminant classifier. To solve S-GM-BWML we also use the fixed-rank positive semidefinite manifold and Riemannian L-BFGS.

We now examine how to evaluate S-GM-BW. The first algorithm we consider is

Bregman Alternating Projections [75]. The objective of Problem (5.8) is the log-determinant divergence, which is a Bregman divergence, and its constraints are linear, so we can alternate between closed-form projections with respect to each constraint to find the solution. We derive the projection, with respect to the first constraint with the derivation of the second projection being analogous.

We begin by relabelling the expression for the objective to $D_{ld}(Z||(\epsilon^{-1}C + I)^{-1})$, where D_{ld} is the log-determinant divergence. The projection with respect to the first constraint is then:

$$\begin{aligned} \min_{Z \in S_{++}^{2n}} D_{ld}(Z||Z_t) \\ \text{s.t. } P_0^T Z P_0 = X_0. \end{aligned}$$

The optimality conditions for this are:

$$Z^{-1} = Z_t^{-1} - P_0 F P_0^T \quad (5.9)$$

$$P_0^T Z P_0 = X_0, \quad (5.10)$$

where $F \in S_+^n$ is the dual variable with respect to the constraint. The first step to find the projection is to invert expression (5.9) and apply the Woodbury-Matrix identity to the right hand side expression:

$$\begin{aligned} Z &= (Z_t^{-1} - P_0 F P_0^T)^{-1} \\ &= Z_t + Z_t P_0 (F^{-1} - P_0^T Z_t P_0)^{-1} P_0^T Z_t. \end{aligned}$$

This expression is then substituted into equation (5.10) and solved for F^{-1} :

$$\begin{aligned} P_0^T (Z_t + Z_t P_0 (F^{-1} - P_0^T Z_t P_0)^{-1} P_0^T Z_t) P_0 &= X_0 \\ P_0^T Z_t P_0 + P_0^T Z_t P_0 (F^{-1} - P_0^T Z_t P_0)^{-1} P_0^T Z_t P_0 &= X_0 \\ P_0^T Z_t P_0 (F^{-1} - P_0^T Z_t P_0)^{-1} P_0^T Z_t P_0 &= X_0 - P_0^T Z_t P_0 \\ (F^{-1} - P_0^T Z_t P_0)^{-1} &= (P_0^T Z_t P_0)^{-1} (X_0 - P_0^T Z_t P_0) (P_0^T Z_t P_0)^{-1} \\ F^{-1} - P_0^T Z_t P_0 &= (P_0^T Z_t P_0) (X_0 - P_0^T Z_t P_0)^{-1} (P_0^T Z_t P_0) \\ F^{-1} &= P_0^T Z_t P_0 + P_0^T Z_t P_0 (X_0 - P_0^T Z_t P_0)^{-1} P_0^T Z_t P_0. \end{aligned}$$

Substituting this into the expression for Z_{t+1} gives a closed-form expression for the projection:

$$\mathcal{P}_0(Z_t) = Z_t + Z_t P_0 (P_0^T Z_t P_0 (X_0 - P_0^T Z_t P_0)^{-1} P_0^T Z_t P_0) P_0^T Z_t.$$

Similarly, the projection with respect to the second constraint is:

$$\mathcal{P}_1(Z_t) = Z_t + Z_t P_1 (P_1^T Z_t P_1 (X_1 - P_1^T Z_t P_1)^{-1} P_1^T Z_t P_1) P_1^T Z_t.$$

An iteration can then be expressed as $Z_{t+1} = \mathcal{P}_1(\mathcal{P}_0(Z_t))$.

To run the Alternating Projections algorithm an initial point Z_0 is required. The corresponding initial point for the log-determinant divergence is $Z_0 = (\frac{1}{\epsilon}C + I)^{-1}$. Since C is factored as $C = Y^T Y$, using the Woodbury Matrix identity means $(\frac{1}{\epsilon}C + I)^{-1}$ may be simplified to $I_{2n} - (\epsilon I_p + Y Y^T)^{-1}$, reducing the cost of the matrix inversion.

As an alternative to Alternating Projections we implement another algorithm, Accelerated Alternating Minimisation [111] (AAM - Algorithm 7), for solving Problem (5.8). AAM has been shown to converge faster than the Sinkhorn algorithm for discrete Optimal Transport problems, and it may also provide a similar benefit to S-GM-BW. AAM constructs the dual problem and then performs block updates on the dual variables using Neseterov momentum. Firstly the dual problem to Problem (5.8) is constructed. The Lagrangian of Problem (5.8) is:

$$L(Z, F, G) = D_{ld}(Z || (\epsilon^{-1}C + I)^{-1}) + \langle F, P_0^T Z P_0 - X_0 \rangle + \langle G, P_1^T Z P_1 - X_1 \rangle,$$

where $F, G \in S_{++}^n$ are dual variables. Taking the derivative of L with respect to Z and setting equal to zero gives:

$$Z = (\frac{1}{\epsilon}C + I - P_0 F P_0^T - P_1 G P_1^T)^{-1},$$

and substituting this back into the Lagrangian results in the dual function:

$$\phi(F, G) = \langle F, X_0 \rangle + \langle G, X_1 \rangle - \log \det(\frac{1}{\epsilon}C + I - P_0 F P_0^T - P_1 G P_1^T),$$

which is free of constraints. An iteration of AAM requires solving the minimisation problem with respect to F and G analytically. The gradients of the dual function with respect to each variable are:

$$\begin{aligned}\nabla_F \phi &= X_0 - \left(\frac{1}{\epsilon} C_{11} + I - \epsilon^{-2} C_{12} \left(\frac{1}{\epsilon} C_{22} + I - G \right)^{-1} C_{21} \right)^{-1} \\ \nabla_G \phi &= X_1 - \left(\frac{1}{\epsilon} C_{22} + I - \epsilon^{-2} C_{21} \left(\frac{1}{\epsilon} C_{11} + I - F \right)^{-1} C_{12} \right)^{-1},\end{aligned}$$

where

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Equating the above gradients to zero and solving for F and G respectively gives:

$$\begin{aligned}F &= \frac{1}{\epsilon} C_{11} + I - \epsilon^{-2} C_{12} \left(\frac{1}{\epsilon} C_{22} + I - G \right)^{-1} C_{21} - X_0^{-1} \\ G &= \frac{1}{\epsilon} C_{22} + I - \epsilon^{-2} C_{21} \left(\frac{1}{\epsilon} C_{11} + I - F \right)^{-1} C_{12} - X_1^{-1},\end{aligned}$$

which are the solutions to the respective block minimisation problems. Using these expressions we run the AAM algorithm, and solve for X . In the experiments section we will compare the Alternating Projections and AAM algorithms for solving Problem (5.8), to determine which is better suited for learning a metric.

5.3 Implementation

Our implementations of SPD-BWML, GM-BWML and S-GM-BWML are made available at: <https://gitlab.ecs.vuw.ac.nz/cooperdavi1/thesis-implementations>. Implementations of GBWML and LEML were not available from their respective authors, so we also provide our own implementations at the preceding url. The authors of IDDL made an implementation available at https://users.cecs.anu.edu.au/~cherian/code/iddl_spd.zip. As in the preceding chapters, experiments were carried out in a grid-computing environment.

5.4 Experiments

We now proceed with experiments on the SPD Metric Learning algorithms that we have introduced. The first section of experiments provides empirical performance of the convergence for Sinkhorn Bures-Wasserstein using Alternating Projections (AP) and Accelerated Alternating Minimisation (AAM). These experiments are performed to establish bounds for parameters for training S-GM-BWML in the later tests. Specifically, we investigate how the choice of the regularisation parameter ϵ and the dimensionality affect the convergence of Problem (5.8) and the implications for performance of S-GM-BWML.

In the second section, the convergence of GM-BWML and S-GM-BWML in comparison to GBWML is examined. These experiments are necessary as it is unclear how to theoretically assess the convergence properties of GM-BWML and S-GM-BWML. In these tests we provide both algorithms with a synthetic dataset to understand how parameters such as: the number of training pairs, dimensionality of the data and value of their regularisation parameters affect each algorithm.

The final set of experiments assess the k -NN-classification error of GM-BWML, S-GM-BWML and SPD-BWML in comparison with GBWML [55], IDDL [133] and LEML [25]. The datasets we use for these experiments are ETH-80 [157] (an objection recognition dataset), MNIST and Fashion-MNIST, and JHMDB [158] (an action recognition dataset composed of short video clips). The examples from each dataset needed to be converted into an SPD form to use the algorithms. We accomplish this using Covariance Descriptors for ETH-80, MNIST and Fashion-MNIST. For JHMDB we compute a covariance matrix of each of the down-sampled frames of each video. MNIST and Fashion-MNIST have been included so that we can make a comparison between the approaches of using distances on SPD matrices and Generalised Mahalanobis distances. Each algorithm learned a metric on the training set, and then a k -NN classifier using that metric was applied to a test set. MNIST and Fashion-MNIST have predefined training and test sets, but for the

other algorithm we randomly split them into training and test sets using an 80:20 rule.

5.4.1 Sinkhorn Bures-Wasserstein Convergence

In this section the results of convergence experiments on Alternating Projections and Accelerated Alternating Minimisation for computing Problem (5.8) are presented. One of these algorithms is selected for use in S-GM-BWML based on its convergence speed.

To test these algorithms, SPD datapoints are required to compute their SBW distance. We generate a SPD matrix $X_i \in S_{++}^n$ according to a Wishart distribution:

$$X_i = G_i^T G_i \quad (5.11)$$

where $G_i \in \mathbb{R}^{n \times n}$ has normally distributed entries with zero mean and unit variance. X_i is then a Wishart random variable on SPD matrices with parameters n and I .

The parameters we vary for these experiments are the dimensionality n of the data X_i and the value of the regularisation parameter ϵ in the objective term of Problem (5.8). We use $n = 5, 10, 20, 50$ and $\epsilon = 0.001, 0.01, 0.1, 1$ during these tests. The choices of n incorporate choices of the SPD matrix dimensionality relevant to later tests and the range of ϵ is typical of values used for the Sinkhorn distance. The ground metric parameter $C \in S_+^{2n}$ is also randomly generated according to $C = Y^T Y$, where $Y \in \mathbb{R}^{n \times 2n}$ has normally distributed entries. This is done to account for the different values of ground metric that will be encountered while training S-GM-BWML. For each combination of n and ϵ 100 pairs of SPD matrices were generated for each algorithm using the above procedure and then run on the respective optimisation problem for 1000 iterations to evaluate the S-GM-BW. The objective value during optimisation and time elapsed were recorded for each pair in order to create convergence plots. The suboptimality (i.e. the difference between the final value of the objective function and the i -th value) is our measure of

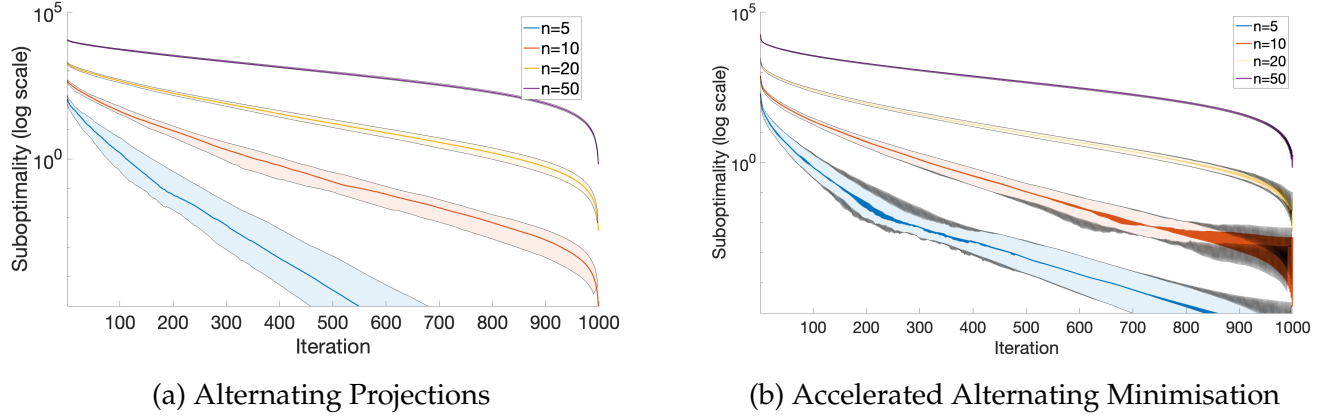


Figure 5.1: Sinkhorn Bures-Wasserstein convergence on different dimensions for $\epsilon = 0.1$.

convergence in these experiments. Similar to previous convergence experiments, the mean of the suboptimality is plotted with quantile bounds included to convey the variability in the results.

In Figure 5.1 a comparison of the convergence of Alternating Projections and Accelerated Alternating Minimisation on problems of varying dimensionality is presented. In this figure the value of ϵ is held constant at 0.1. There is a roll-off in the convergence at the end of each series. The final value of the objective used to compute the suboptimality is still fairly far from the solution. So while the earlier part of the convergence is indicative of the rate of convergence, the final section of the series will be more reflective of this suboptimal point. The $n = 5$ series does not experience this effect as significantly as its final value is much closer to the solution. The convergence figures for Alternating Projections and Accelerated Alternating Minimisation are fairly similar, for increasing dimension the rate of convergence appears to decrease, with dimension equal to 50 having a very slow rate of convergence. While the quantile margins appear smaller for the larger dimensions, the logarithmic scale of the figure suppresses equivalent variability at larger values. In fact, the variability is reasonably constant with respect to di-

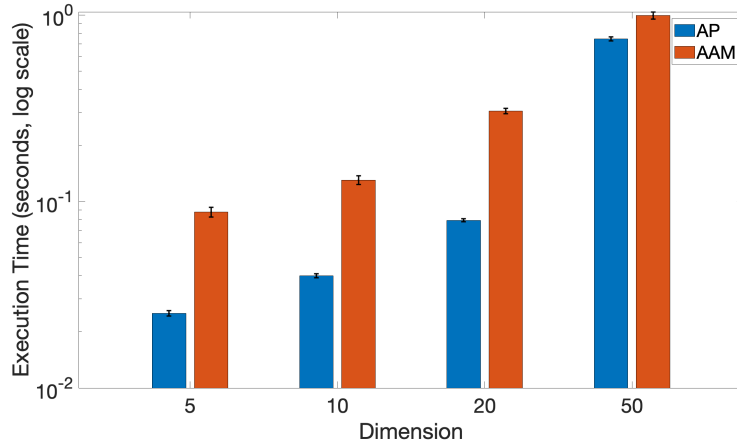


Figure 5.2: Timing comparison of S-GM-BW algorithms with respect to dimension for $\epsilon = 0.1$.

mensionality. The convergence series for Accelerated Alternating Minimisation appears to have high frequency oscillation in some regions. This may be due to instability in the log-determinant objective term. In our implementation of Accelerated Alternating Minimisation there is no guarantee of the gradient being Lipschitz continuous, so discontinuities may account for the oscillations. Oscillations are not present in the Alternating Projections series, possibly because of its closed-form updates.

We can see in Figure 5.2 that the execution time for Alternating Projections is generally faster than Accelerated Alternating Minimisation. Given that the convergence of both algorithms are similar in Figure 5.1 and the Accelerated Alternating Minimisation algorithm is more complex than Alternating Projections, it is not unexpected that Alternating Projections takes significantly less time to execute.

In Figure 5.3 the convergence figures for Alternating Projections and Accelerated Alternating Minimisation with respect to choices of the regularisation parameter ϵ are shown, with dimension equal to 10. Similar features to the previ-

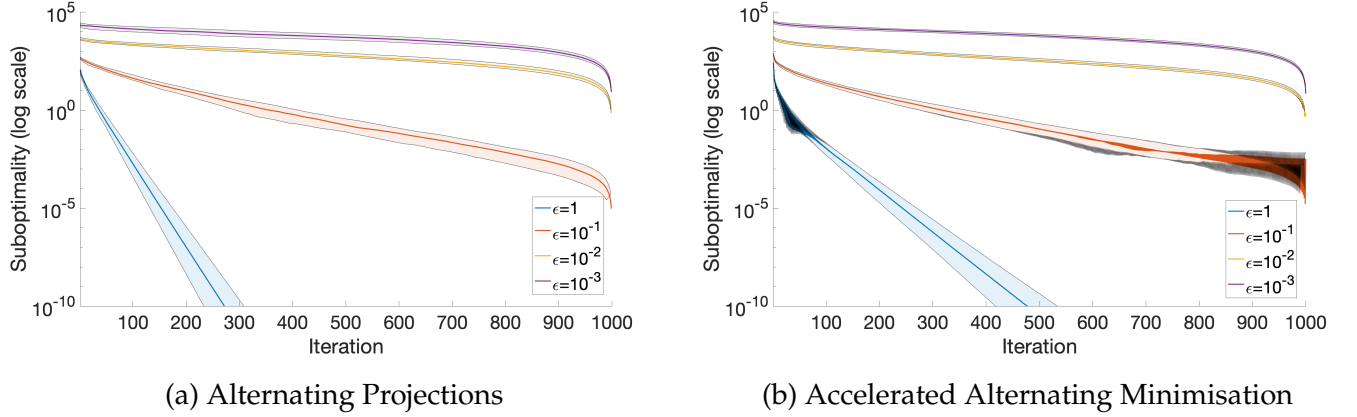


Figure 5.3: Sinkhorn Bures-Wasserstein convergence on different regularisation parameter values in 10 Dimensions.

ous convergence figure are present. For smaller values of ϵ both algorithms have a significantly reduced rate of convergence. As before, Accelerated Alternating Minimisation has oscillations present while Alternating Projections does not.

In Figure 5.4 we see that the change in ϵ does not affect the execution time of Alternating Projections. This is unsurprising since ϵ changes the values of the problem during optimisation, but not the overall complexity of the steps. It does appear that ϵ affects the rate of convergence of Accelerated Alternating Minimisation. This may be due to changes in the objective making the line search steps more time consuming. Overall, Alternating Projections is faster and less sensitive to the choice of ϵ with respect to timing.

From these experiments it is apparent that Alternating Projections is the preferred choice in terms of execution time. Additionally, it does not exhibit the oscillations and dependence on ϵ that Accelerated Alternating Minimisation does. Therefore in the following sections Alternating Projections is used for computing values of S-GM-BW.

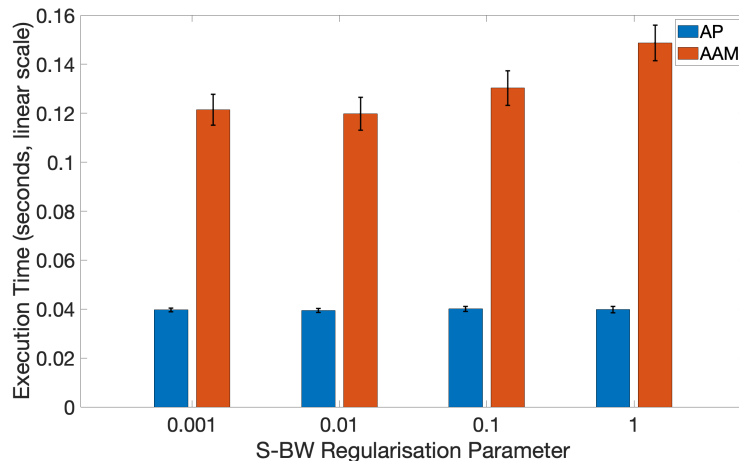


Figure 5.4: Timing comparison of algorithms with respect to ϵ .

5.4.2 GM-BWML and S-GM-BWML Convergence

Now that the Alternating Projections algorithm has been selected for S-GM-BWML we proceed with testing GM-BWML, S-GM-BWML and GBWML on a synthetic dataset. We compare the convergence and timing of GM-BWML and S-GM-BWML to GBWML. Both of our algorithms can be seen as generalisations of GBWML and so this comparison is very useful.

Synthetic problems are generated using a similar approach to the S-GM-BW tests in the previous section. Pairs of matrices $X_i, X_j \in S_{++}^n$, are sampled independently from a Wishart distribution and are treated as similarity pairs.

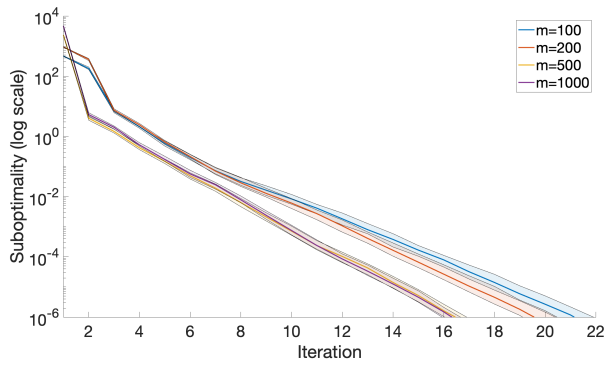
For these tests the influence of: problem dimensionality n , number of training pairs m , and regularisation parameter γ are examined. For valid comparison the same Frobenius norm regulariser was included for GBWML. We use $n = 5, 10, 20$, $m = 100, 200, 500, 1000$ and $\gamma = 0.0001, 0.001, 0.01, 0.1, 1$, since these reflect realistic values that also scale to the large number of tests being run. For each setting with respect to dimensionality, training pairs or γ , the experiment is repeated 100 times, with the similarity pairs being resampled each time. The choice of the ini-

tial matrix C for S-GM-BWML is the default matrix used for the Bures-Wasserstein distance. Similar to previous convergence tests, the median suboptimality curve of the 100 tests is shown, with quantile bounds of 25 % and 75% being included as a shaded region. For timing figures the mean execution time for each algorithm is presented over the 100 tests and error bars of one standard deviation are included. Outliers were detected using the MATLAB function 'rmoutlier' and are displayed as separate points.

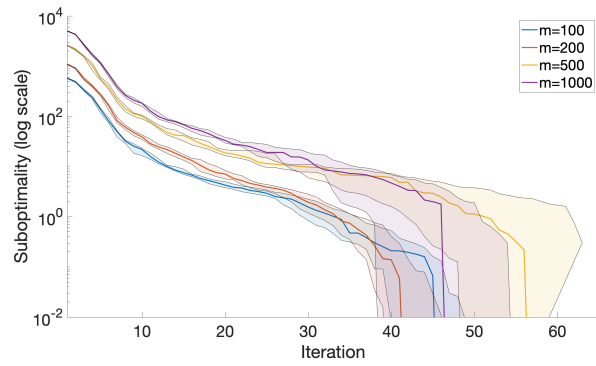
The effects of the S-GM-BW parameters on the convergence and timing of S-GM-BWML are also presented. The choice of ϵ and number of Alternating Projections iterations N_{AP} are examined, where $\epsilon = 0.001, 0.01, 0.1, 1$ and $N_{AP} = 10, 100, 1000$. This will motivate the choice of parameters in the k -NN classification tests that follow.

In Figure 5.5 the convergence figures for the number of training pairs tests of GBWML, GM-BWML and S-GM-BWML are shown. Both GBWML and S-GM-BWML appear to converge, but GBWML does so linearly, whereas S-GM-BWML converges super-linearly. Both algorithms appear to converge to the same level of suboptimality in about 20 iterations. GM-BWML converges much slower than the other two algorithms and scales worse with respect to the number of training pairs. For GBWML, as the number of training pairs increases the number of necessary iterations appears to decrease modestly. This may be because as the number of training pairs increases the gradient becomes subject to less variance since we are sampling from an analytic distribution. The result of this is that GBWML requires fewer iterations for more constraints. S-GM-BWML does not display the same decrease in iterations with number of training pairs. This difference can be attributed to the additional noise in the gradients from estimating the SBW distance. This level of noise may outweigh the potential reduction from the number of training pairs. For GM-BWML the effect of scaling is more obvious: more training pairs requires more iterations.

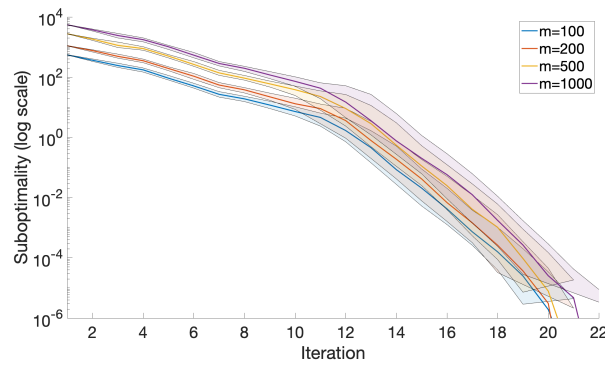
We see in Figure 5.6 that GBWML takes significantly less time during execu-



(a) GBWML



(b) GM-BWML



(c) S-GM-BWML

Figure 5.5: Convergence comparison for varying number of constraints in 10 dimensions and $\gamma = 10^{-3}$.

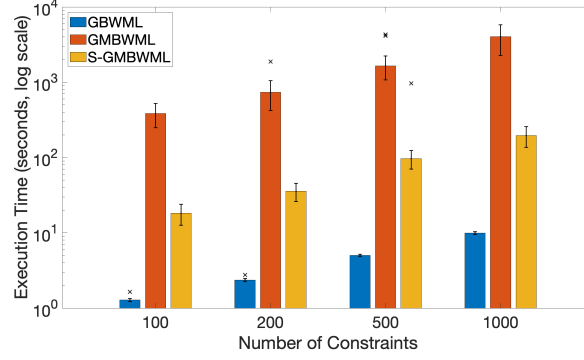
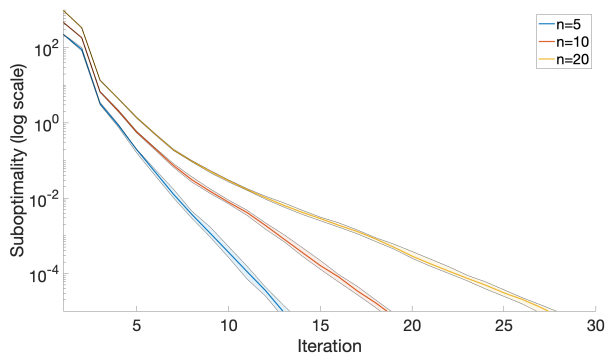


Figure 5.6: Timing comparison with respect to number of constraints in 10 dimensions and $\gamma = 10^{-3}$.

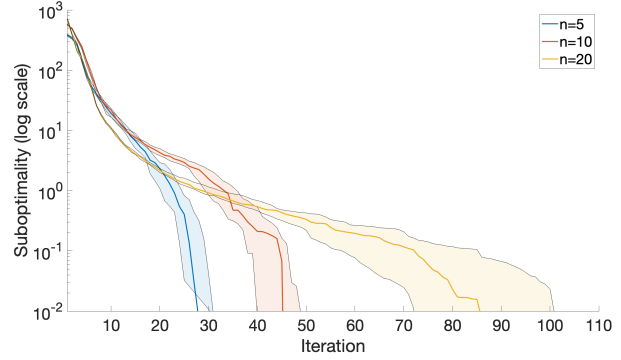
tion compared to GM-BWML and S-GM-BWML. Overall, the algorithms all show a similar trend for execution time with respect to number of constraints.

In Figure 5.7 convergence plots for problems of different dimensionalities are shown. S-GM-BWML appears to scale the best with respect to the number of iterations required to converge. GBWML has some increase in the number of iterations required and GM-BWML has a significant increase in the number of iterations required. For both GBWML and GM-BWML the increase in dimensionality results in an increase in the complexity of the training variable Y . It is reasonable that GM-BWML has a more significant decrease in rate of convergence since it has double the dimensionality of GBWML. For S-GM-BWML the convergence in 20 dimensions appears to be as fast as in 5 dimensions. This effect must be due to some influence from the S-GM-BW. Since the S-GM-BW parameters are constant for each dimension the effect of dimensionality may be lessened as the complexity of the distance computation does not increase as significantly compared to the other algorithms.

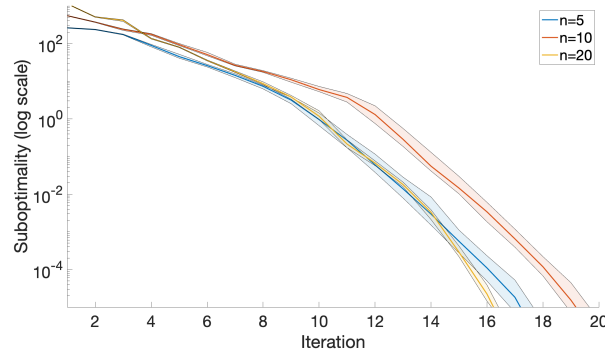
Additionally, for the timing in Figure 5.8, we see that the increase in execution time for S-GM-BWML is not as fast as GBWML or GM-BWML for increasing di-



(a) GBWML



(b) GM-BWML



(c) S-GM-BWML

Figure 5.7: Convergence comparison for varying dimension, 100 Constraints and $\gamma = 10^{-3}$.

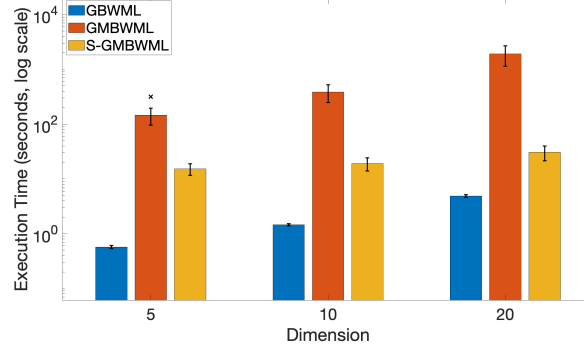


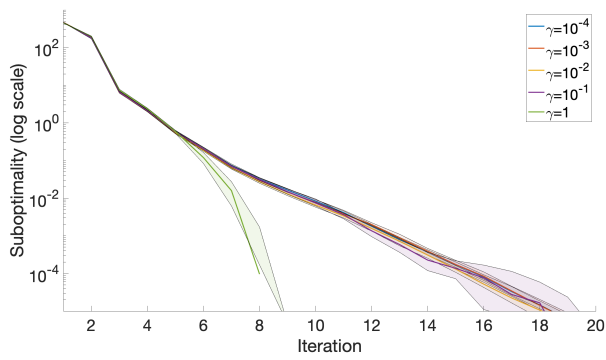
Figure 5.8: Timing comparison with respect to dimension for 100 constraints and $\gamma = 10^{-3}$.

mension. This may be due to the reduced number of iterations required for S-GM-BWML at larger dimensionality, a trend that the other algorithms do not follow.

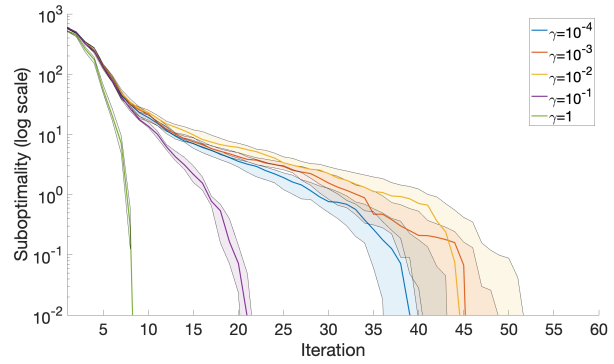
For the effect of the regularisation parameter in Figure 5.9 the primary effect is that the optimisation terminates earlier when γ is larger. For the smaller values of γ there appears to be some variation as to which series converges first. This may be because with small enough γ the problem acts as if it is unregularised. When γ is sufficiently large we see it's intended effect on convergence for series such as $\gamma = 0.1$ and $\gamma = 1$.

For the timing (Figure 5.10), GBWML follows the trend from the convergence figure fairly well, where most of the values of γ yield similar execution times, except for $\gamma = 1$, which appears to be smaller, but with a larger variance. GM-BWML follow a modest decrease in execution time as γ increases. For S-GM-BWML the trend in timing seems to be unrelated to the trend observed in the convergence. As γ increases the execution time increases. This trend is due to some other effect, perhaps larger γ making the retraction function more complex to evaluate.

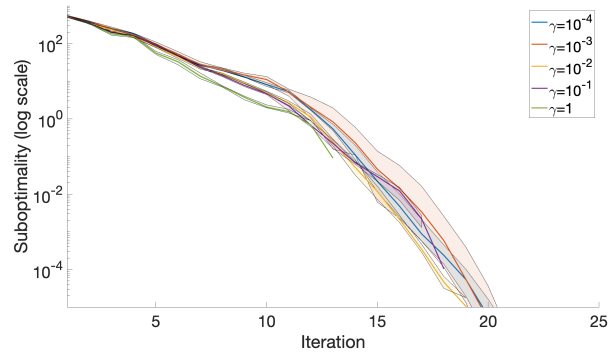
The convergence plots of S-GM-BWML for varying SBW parameters are shown in



(a) GBWML



(b) GM-BWML



(c) S-GM-BWML

Figure 5.9: Convergence comparison for varying γ with 100 constraints and 10 dimensions

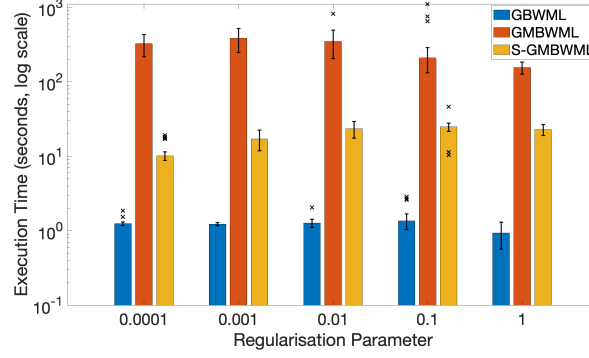
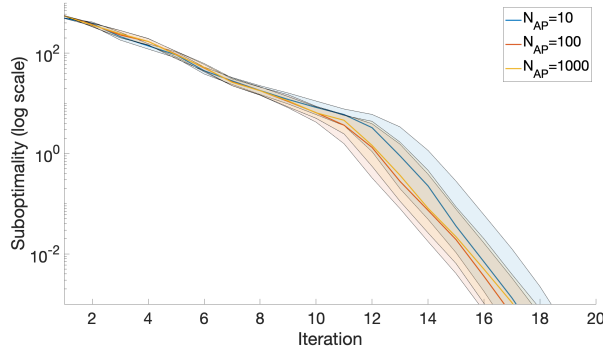
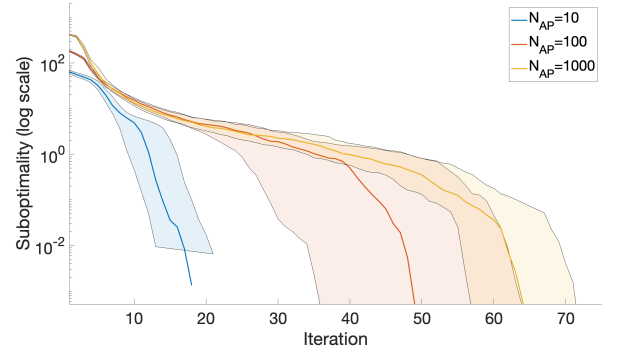


Figure 5.10: Timing comparison with respect to γ for 100 constraints and 10 dimensions



(a) $\epsilon = 0.1$



(b) $\epsilon = 0.001$

Figure 5.11: S-GM-BWML convergence comparison for varying N_{AP} , 100 constraints, 10 dimensions and $\gamma = 10^{-3}$.

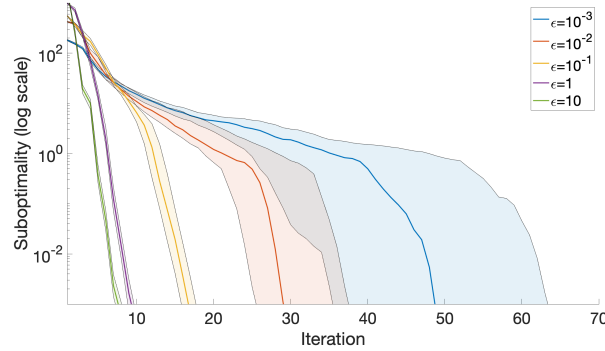


Figure 5.12: S-GM-BWML convergence with varying ϵ for 100 constraints, 10 dimensions and $\gamma = 10^{-3}$.

figures 5.11 and 5.12. Figure 5.11 shows how convergence is related to the number of Alternating Projections iterations for two values of ϵ . For $\epsilon = 0.1$, a relatively large value, there is little difference between the convergence plots,. For $\epsilon = 0.001$ there is a significant difference between the different values of N_{AP} . When N_{AP} is larger the number of iterations to convergence is larger. This is consistent with the figures for convergence of S-GM-BW.

In Figure 5.12 the convergence for several values of ϵ with a fixed n is shown. We see that as ϵ decreases the required number of iterations increases and convergence is slower.

Similarly, in Figure 5.13 we see how the timing changes with different values of ϵ and N_{AP} . While it is unsurprising that as N_{AP} increases the execution time increases, the effect of ϵ on the time to convergence is definitely strengthened.

In these experiments we have shown that GBWML, GM-BWML and S-GM-BWML converge under several different conditions with the synthetic datapoints. The net effect of number of training pairs and dimensionality are similar to previous cases where they generally influence the complexity of the problem. For different val-

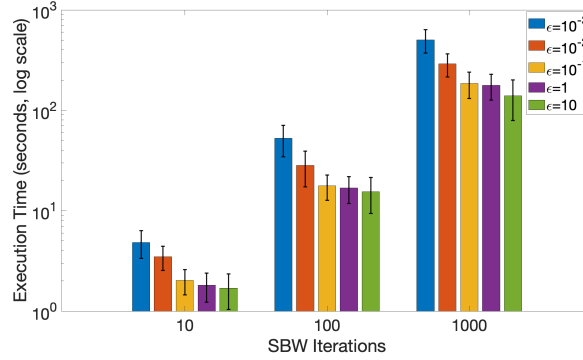


Figure 5.13: S-GM-BWML timing with varying S-BW parameters for 100 constraints, 10 dimensions and $\gamma = 10^{-3}$.

ues of γ the algorithms have been shown to converge, although the relationship between γ , convergence and execution time is more opaque.

It appears that S-GM-BWML converges in a similar number of iterations to GBWML, whereas GM-BWML generally takes longer than either of these algorithms. In terms of execution time GBWML is the fastest followed by S-GM-BWML and then GM-BWML. The intention to reduce the execution time for S-GM-BWML using the S-GM-BW appears to have been successful in these tests.

The effects of N_{AP} and ϵ is also clear on S-GM-BWML and are consistent with the results of SBW convergence. Observing the general trends in the results, it appears that $\epsilon = 0.1$ and $N_{AP} = 100$ are good initial choices for these parameters. For the k -NN classification experiments we will use these values during training in order to reduce the search space for grid searching on parameters.

Having established the convergence behaviour of GBWML, GM-BWML and S-GM-BWML we may now proceed with k -NN classification tasks using our algorithms, as well as two others, for a relevant comparison.



Figure 5.14: ETH-80 sample images

5.4.3 k -NN Classification Tests

The results of k -NN classification using GBWML, SPD-BWML, GM-BWML and S-GM-BWML are now presented. We present the classification error, training time and testing time for each algorithm. We include the testing time for these results as there are multiple methods of computing the distances between datapoints i.e., Log-Euclidean distance vs Generalised Bures-Wasserstein distance vs (S)-GM-BW. Because of this we wish to understand the relative performance of each distance during testing. For comparison, we include LEML and IDDL to provide relevant baselines for these experiments. LEML is essentially a reformulation of ITML using the Log-Euclidean distance between samples instead of the Mahalanobis distance. LEML has been shown to be highly scalable and has good performance on SPD Metric Learning tasks [25]. IDDL is another information-theoretic algorithm. However, it chooses to learn a dictionary of SPD atoms to classify data. Compared to typical Metric Learning algorithms IDDL has a much higher capacity for its learnable parameters [132, 133].

For our experiments we choose four datasets. The first two are ETH-80 and JHMDB. ETH-80 is an object recognition dataset consisting of images of eight kinds of object photographed from several angles. JHMDB, an action recognition dataset, of high dimensionality, presents a more difficult problem for learning a metric.



Figure 5.15: JHMDB sample images

We also include MNIST and Fashion-MNIST in our experiments to allow comparisons with the previous Metric Learning tasks to see if there are any advantages or disadvantages of using SPD algorithms on these datasets. These datasets are not natively expressed as SPD matrices, so they need to be transformed into SPD examples. This is accomplished using a couple of different methods. For ETH-80 a typical Region Covariance Descriptor [21] approach is used. For each pixel $(i, j) \in [m] \times [n]$ in the 3-channel RGB image: $R, G, B \in \mathbb{R}^{m \times n}$ a 10-dimensional feature is generated:

$$f_{ij} = \left[i, j, R_{ij}, G_{ij}, B_{ij}, \left| \frac{\partial I_{ij}}{\partial i} \right|, \left| \frac{\partial I_{ij}}{\partial j} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial j^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i \partial j} \right| \right]^T,$$

where I is the m by n greyscale intensity of the image and the discrete derivatives are calculated using the MATLAB 'gradient' function. The covariance matrix for the image is then:

$$C = \frac{1}{nm - 1} FF^T,$$

the 10×10 covariance matrix of all the pixel level features, where F is the $10 \times mn$ matrix of stacked features f .

For MNIST and Fashion-MNIST, two covariance-descriptor representations are created for each dataset. The first is analogous to the one for ETH-80, except MNIST and Fashion-MNIST already contain greyscale images, so the pixel-level fea-

tures are:

$$f_{ij} = \left[i, j, I_{ij}, \left| \frac{\partial I_{ij}}{\partial i} \right|, \left| \frac{\partial I_{ij}}{\partial j} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial j^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i \partial j} \right| \right]^T.$$

An 8×8 covariance matrix is then computed as above using these features.

Higher dimensional features are also created using a method similar to Harandi *et al.* [80]. These features extend the previous covariance descriptor by adding Gabor filter responses $G_{i,j}^{(s,o)}$ [160] at pixel (i, j) with scale and orientation parameters s and o respectively:

$$f_{ij} = \left[i, j, I_{ij}, \left| \frac{\partial I_{ij}}{\partial i} \right|, \left| \frac{\partial I_{ij}}{\partial j} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial j^2} \right|, \left| \frac{\partial^2 I_{ij}}{\partial i \partial j} \right|, |G_{i,j}^{(s_1,o_1)}(I)|, \dots, |G_{i,j}^{(s_4,o_4)}(I)| \right]^T$$

where the scales are $s = [2, 4, 8, 16]$ and $o = [0, 45, 95, 135]$. The resulting covariance matrix C is size 24×24 .

To create the covariance representation for JHMDB we instead treat each video as an image set and compute an associated covariance matrix [26]. This particular method of computing an image set covariance was used to test LEML [25]. For each frame in a video we convert it to greyscale, resize it to a 20×20 image and vectorise it to a 400 dimensional vector. The mean and covariance matrices of the vectors are computed as μ and C . These are combined into the 401 by 401 covariance matrix:

$$\tilde{C} = \begin{bmatrix} C + \mu\mu^T + \delta I & \mu \\ \mu^T & 1 \end{bmatrix} \quad (5.12)$$

where $\delta = 10^{-3}$ is included to ensure the matrix is positive definite. $\tilde{C}$ is then used as a training example for the action the video is associated with.

Grid searches were performed to select the training parameters for each algorithm. However, grid searches were not performed for the JHMDB experiments since this dataset was too computationally expensive to train multiple times. As with the previous experiments the value of k was set to 4 for each algorithm during testing. Similarity and dissimilarity pairs were created using the same method

Dataset \ Algorithm	GBWML	IDDL	LEML	SPD-BWML	GM-BWML	S-GM-BWML
k -NN Error Rate	3.51 %	0.61 %	2.29 %	3.65 %	28.2 %	62.0 %
Training Time (s)	1.24	59631.93	0.08	0.32	11130.64	10.66
Testing Time (s)	39.86	0.26	7.21	34.57	30436.10	936.32

Table 5.1: k -NN classification results on ETH-80

Quantity \ Algorithm	GBWML	IDDL	LEML	SPD-BWML	GM-BWML	S-GM-BWML
k -NN Error Rate (8 x 8)	15.2 %	13.3 %	29.5 %	16.8 %	59.4 %	86.0 %
k -NN Error Rate (24 x 24)	5.3 %	N/A	3.1 %	3.5 %	N/A	80.0 %
Training Time (s, 8 x 8)	334.55	82537.76	0.06	0.63	833.11	75.73
Training Time (s, 24 x 24)	3987.82	N/A	0.01	237.57	N/A	483.57
Testing Time (s, 8 x 8)	962.00	0.51	213.22	961.01	31563.65	290.64
Testing Time (s, 24 x 24)	5506.02	N/A	313.54	2942.47	N/A	1374.53

Table 5.2: k -NN classification results on MNIST

as previous tests. Pairs of indices of SPD training data were uniformly sampled and if the pair has the same label they are treated as a similarity pair and a dissimilarity pair otherwise. In some cases there is a N/A for an entry in our results. This occurs when the algorithm would not converge due to limitations of the hardware being used during these tests.

In Table 5.1 the results on ETH-80 are shown. IDDL has the lowest error rate at the expense of a large training time, whereas GBWML, LEML and SPD-BWML have fairly similar error rates to each other and train in less time. GM-BWML and S-GM-BWML have very high error rates. S-GM-BWML takes about 10 times longer to train than GBWML and GM-BWML takes ten thousand times longer. The situation is reversed for testing time where IDDL classifies the results very quickly. LEML tests faster than GBWML and SPD-BWML, which are followed by S-GM-BWML and GM-BWML.

The results for both MNIST covariance descriptors are presented in Table 5.2. IDDL achieves the lowest error rate for the lower-dimensional problem, again

Quantity \ Algorithm	GBWML	IDDL	LEML	SPD-BWML	GM-BWML	S-GM-BWML
k -NN Error Rate (8×8)	21.9 %	24.1 %	20.9 %	21.8 %	52.3 %	80.0 %
k -NN Error Rate (24×24)	19.0 %	N/A	15.5 %	16.3 %	N/A	84.0 %
Training Time (8×8)	8.79	85465.80	0.11	0.77	843.924572	48.69
Training Time (24×24)	5065.62	N/A	0.14	28.76	N/A	359.76
Testing Time (8×8)	663.32	0.51	238.00	945.55	26041.09	276.32
Testing Time (24×24)	5430.21	N/A	438.86	3129.72	N/A	1426.47

Table 5.3: k -NN classification results on Fashion-MNIST

with a large training time, however IDDL was not able to converge for the higher-dimensional covariance descriptors. GBWML and SPD-BWML achieve lower error rates for the 8×8 covariance descriptors, attaining improved error rates over LEMML. However for the 24×24 covariance descriptors, LEMML has a significant reduction in the error rate and beats SPD-BWML marginally. GBWML has long training times compared to LEMML and SPD-BWML, but for the higher-dimensional data this gap between GBWML and SPD-BWML becomes smaller, whereas LEMML executes in even less time. For testing time IDDL is much faster in the case where it converged and the Bures-Wasserstein methods scaled similarly to each other, taking longer than LEMML to evaluate k -NN classification. GM-BWML and S-GM-BWML have high error rates for both tasks. S-GM-BWML seems to scale a bit better than GBWML, but its error rate is too high to be usable. However, it is interesting to note that the testing time for S-GM-BWML is lower than SPD-BWML. It seems that the S-GM-BW may scale better than the GBW for evaluating k -NN classification. Similar to IDDL, GM-BWML would not converge for the 24×24 covariance descriptors and when training on the 8×8 descriptors the number of training pairs was limited by the training time. Table 5.3 shows the results for k -NN classification on Fashion-MNIST. Since MNIST and Fashion-MNIST have the same number of training and testing examples the training times are similar to the MNIST tests. LEMML has the lowest error for both types of covariance descriptor with a small margin between its error, S-GM-BWML and GBWML. For the higher-dimensional covariance descriptors the margin between

Quantity \ Algorithm	GBWML	IDDL	LEML	SPD-BWML	GM-BWML	S-GM-BWML
k -NN Error Rate	N/A	N/A	63.87%	65.97 %	N/A	N/A
Training Time	N/A	N/A	64.56	4870.96	N/A	N/A
Testing Time	N/A	N/A	291.61	2951.10	N/A	N/A

Table 5.4: k -NN classification results on JHMDB

LEML and GBWML is increased. However SPD-BWML maintains its difference in error to LEML between both experiments. The results of GM-BWML and S-GM-BWML on the Fashion-MNIST test is very similar to that on the MNIST tests. Both algorithms have high error rates and long training times, with GM-BWML unable to return a metric on the 24×24 dataset. As with MNIST, S-GM-BWML has reduced training times with respect to SPD-BWML. Similar to the case for the MNIST results, the error rates here do not out perform previous algorithms on classification for Fashion-MNIST.

In comparison to the error rates for Fashion-MNIST and MNIST classification in the previous chapters, these classifiers they do not improve on the classification error and wholesale increases in training and testing time. The error rates only become comparable with the 24×24 covariance descriptors, which have dimension $24(24 + 1)/2 = 300$. This much larger than the 100-dimensional data used in previous chapters for learning distances on MNIST.

For JHMDB, the covariance descriptors were too large for GBWML, GM-BWML, S-GM-BWML and IDDL to find the desired metric as seen in Table 5.4. LEML and SPD-BWML are more scalable algorithms and converge under restrictive hardware. LEML attains a smaller error rate and executes faster for both training and testing than SPD-BWML, however the margin in error rate is not large.

Overall, when IDDL converges its results are better than the other algorithms. This can be attributed to its increased expressivity from its large number of dictionary elements and trainable parameters. This increased expressivity does make

the learning problem more complex, and hence requires better hardware to compute a solution. LEML is the best performing algorithm in general, with SPD-BWML being fairly close in error rates. GBWML and SPD-BWML perform similarly, but SPD-BWML scales better, as seen in the JHMDB tests. GM-BWML and S-GM-BWML unfortunately contribute little to learning a metric and do so without any advantage in scaling in training. However S-GM-BWML does scale better than SPD-BWML in some cases for testing.

5.5 Conclusion

In this chapter three Metric Learning algorithms for SPD data have been introduced: SPD-BWML, GM-BWML and S-GM-BWML. The first is an extension of BWML, while the other two use a ground metric formulation. An approximation method for the ground metric was introduced, with two candidate algorithms being identified to solve it. GM-BWML and S-GM-BWML were then run on synthetic problems to establish their convergence properties. The algorithms were found to converge with S-GM-BWML converging faster than GM-BWML.

On k -NN classification tasks, SPD-BWML had fairly competitive results with the existing algorithms, but GM-BWML and S-GM-BWML did not. SPD-BWML performed better than GBWML in classification tasks and also exhibited better scalability, with training times at least an order of magnitude lower. These types of benefits are consistent with the benefits conferred by Cyclic Projections in the algorithms in previous chapters.

However, looking at classification performance, learning SPD matrix representations on MNIST and Fashion-MNIST did not seem to offer a benefit over the previous metrics. It appears that datasets that tend to benefit from SPD matrix classifiers seem to have a higher level of richness to their examples, such as ETH-80 or JHMDB. In contrast, MNIST and Fashion-MNIST contain simpler greyscale images.

A limitation of the Generalised Bures-Wasserstein distance for k -NN classification is that computing the distances associated with them is more expensive than the Generalised Mahalanobis distance and the Log-Euclidean distance. This is seen specifically in the increased testing times for the Bures-Wasserstein distance methods. However during testing S-GM-BW was able to run k -NN faster than the other Bures-Wasserstein algorithms. This may offer a method to improve the scalability of testing time. Further research into S-GM-BW as a means of increasing the scalability of generalised Bure-Wasserstein methods may be beneficial for future algorithms.

It appears that the increased model capacity of GM-BWML and S-GM-BWML hinders the learning problem, although this seems contrary to the performance of other Ground Metric Learning algorithms. While GBWML and SPD-BWML train on a subset of the metrics available to GM-BWML and S-GM-BWML, they manage to achieve good classification error. This indicates that a SPD Ground Metric Learning algorithm needs more constraints than GM-BWML. Perhaps the set of metrics available to the generalised Bures-Wasserstein distance is more or less optimal for Metric Learning on SPD matrices.

SPD-BWML had worse classification performance than LEML, but only by a small margin. IDDL generally had the best classification performance. However, it is less scalable than LEML and SPD-BWML and was not able to be evaluated on datasets using larger covariance matrices. While the updates for SPD-BWML (similar to LEML) are approximations, the algorithm performs at a similar level to GBWML, which does not use approximations in its formulation. The success of this algorithm provides more weight to these types of approximations when designing algorithms. Being able to approximate the generalised Bures-Wasserstein distance as a linear constraint may be useful in a wider context for algorithm design.

Chapter 6

Conclusion

In this thesis we have introduced a variety of Metric Learning algorithms that use the Bures-Wasserstein distance. We now conclude with a review of how these algorithms have achieved the aims set out in the Introduction, followed by a suggestion of future work and final remarks.

In Chapter three, five metric learning algorithms were introduced: Bures-Wasserstein Metric Learning (BWML), Averaged Bures-Wasserstein Barycentric Metric Learning (A-BWBML), Sequential Bures-Wasserstein Barycentric Metric Learning (S-BWBML), Transport-Bures-Wasserstein Metric Learning (T-BWML) and Similarity Transport-Bures-Wasserstein Metric Learning (T-BWML-S). A rigorous proof of convergence was provided for BWML and experiments on empirical convergence were conducted on each algorithm, with Information-Theoretic Metric Learning (ITML) included for comparison. The synthetic problem experiments demonstrated the different convergence properties of each algorithm and indicated that all algorithms converge under the conditions we tested. The effect of using Wasserstein Barycentres in the BWBML algorithms was apparent relative to BWML and ITML. The averaging that the barycentre computation offered provided a smoothing effect to convergence of these algorithms. T-BWML and T-BWML-S both converged in far less iterations than the other algorithms owing to their second-order optimisation algorithms. The k -Nearest-Neighbours (k -NN) classification perfor-

mance of our algorithms was then evaluated on feature data obtained from UCI datasets. Each of the algorithms, except for T-BWML, demonstrated competitive classification performance on these datasets, with most executing faster or at a similar speed to the comparison algorithms. For the k -NN classification experiments on image data our algorithms again demonstrated competitive classification error performance. However, the increased size of MNIST and Fashion-MNIST illustrated the differences in scalability of each algorithm. The fastest algorithm out of our contributions was BWML. However, A-BWBML and T-BWML-S had the best error performance, although by a small margin in some cases. Our algorithms utilise several different features of the Bures-Wasserstein distance, each with their different characteristics. We see that the effect of these differences primarily results in different training times, which are essentially differences in scalability.

In Chapter four, two metric learning algorithms for learning low-rank Generalised Mahalanobis distances were introduced: Low-Rank Bures-Wasserstein Metric Learning (LR-BWML) (an extension of BWML) and Bures-Wasserstein Logistic-Discriminant Metric Learning (BW-LDML) a Riemannian optimisation algorithm. LR-BWML was shown to be equivalent to BWML on the linear subspace described by the initial metric and as such converged reliably. Similar to the previous chapter, synthetic problem experiments were conducted to verify the convergence properties of LR-BWML and demonstrate convergence properties of BW-LDML. LR-BWML was found to behave essentially as BWML, and BW-LDML was found to have good linear convergence properties with respect to the synthetic problem parameters. Both algorithms showed some increased scalability with respect to a reduction in rank. In some cases, however, reduction in rank increased the training time for LR-BWML. This indicated that the complexity of the learning problem can be influenced more by the amount of training data than the dimensionality of the problem. When reducing rank, the k -NN classification error increased steadily. It seems that the benefit of rank reduction is to reduce training time (or increase scalability) at the expense of classification performance. Interestingly, metrics learnt

by BW-LDML resulted in classifiers that were more tolerant to reduction in rank than LR-BWML, although it was generally slower to train.

In Chapter 5, three metric learning algorithms for learning metrics on Symmetric Positive Definite (SPD) Matrices were presented. These are: SPD-Bures-Wasserstein Metric Learning (SPD-BWML) – an extension of BWML to SPD matrices, Ground Metric Bures-Wasserstein Metric Learning (GM-BWML) and Sinkhorn Ground Metric Bures-Wasserstein Metric Learning (S-GM-BWML) – a variation of GM-BWML using an entropic approximation to the ground metric formulation. Convergence experiments on synthetic datasets were performed for GM-BWML and S-GM-BWML and GBWML, and the expected increase in rate of convergence for S-GM-BWML versus GM-BWML was demonstrated. k -NN classification experiments on several types of SPD data were then conducted with each of these algorithms. Both GM-BWML and S-GM-BWML showed poor classification performance. However, during testing S-GM-BWML performed k -NN classification on Ground Metric Bures-Wasserstein faster than SPD-BWML. This gives potential for Sinkhorn Ground Metric Bures-Wasserstein to be used to increase scalability of other Generalised Bures-Wasserstein algorithms during evaluation. SPD-BWML had competitive k -NN classification performance with Generalised Bures Wasserstein Metric Learning (GBWML) and similar k -NN classification performance to Log Euclidean Metric Learning (LEML). SPD-BWML was also able to train a metric on the large-scale action recognition dataset JHMDB, which only LEMML was also able to do.

Overall, we have found that the majority of our Bures-Wasserstein Metric Learning algorithms perform well on classification tasks, but do not exceed the existing algorithms by a large margin. Our algorithms achieve competitive performance in classification, but with variable performance in training time. We have however, demonstrated that algorithms based on the Bures-Wasserstein distance do indeed reliably learn metrics on a variety of data.

6.1 Future Work

There are many avenues for future work based on this thesis. Here we consider the most direct and potentially fruitful extensions.

With respect to learning a Generalised Mahalanobis distance using the Bures-Wasserstein distance, it would be beneficial to study the theoretical properties of A-BWBML and S-BWBML. For example, these barycentric methods did not exhibit the oscillations exhibited by BWML and were much smoother during convergence. However, the optimality of the solutions produced by A-BWBML and S-BWBML was unclear. Establishing conditions for optimality in these kinds of algorithms may improve classification performance or reduce training time.

The Supervised Word Mover's Distance was able to improve on document classification performance using the Wasserstein distance. Implementing a variant of BWML using the Wasserstein distance between document representations is also an avenue for future work. Ideally the BWML pattern could confer similar benefits that ITML confers over Neighbourhood Component Analysis.

A geometric extension to Cyclic Projections similar to our BWBML algorithms for learning a generalised Mahalanobis metric on $S_+(p, n)$ may also be of use. Low-Rank Bures-Wasserstein Metric Learning converged in less time than BW-LDML. However, the classification error of BW-LDML was more tolerant to reduction in rank. A method, perhaps barycentric, that incorporates closed-form updates of LR-BWML with the geometric awareness of BW-LDML may possess both of these advantages.

Establishing theoretical convergence properties of BW-LDML with respect to the Riemannian L-BFGS algorithm or other Riemannian optimisation algorithms would be of interest. This would allow us to quantify the behaviour of this algorithm and make more informed choices about how it is optimised.

The performance of GM-BWML and S-GM-BWML was very poor for classification on SPD data. To improve the classification error we would consider including constraints on the set of ground metrics, similar to Ground Metric Learning. Since Ground-Metric Bures-Wasserstein Metric Learning is a generalisation of GBWML we would seek an understanding of how to enlarge the set of metrics without degrading performance. Choosing a regulariser different to the Frobenius distance may also have a similar effect to introducing more constraints.

SPD-BWML used the Bures-Wasserstein distance as its objective term to train the generalised Bures-Wasserstein distance. Relative to LEML this formulation did not have significant benefit. SPD-BWML did not follow the same pattern as BWML in the sense that it altered both the training objective and the type of distance. In future we would consider only using the Bures-Wasserstein distance as the objective function and using the Log-Euclidean distance as the trainable distance function. Similar benefits of BWML over ITML would be expected and the increased complexity of evaluating the Bures-Wasserstein distance may be avoided.

Extending our algorithms to Kernel and Deep Metric Learning would also be a good future direction for research. These non-linear approaches may give additional discriminative power for classification on complex datasets. An alternative to a linear transformation with the generalised Bures-Wasserstein distance is to use a SPD neural network [161] $f_\theta : S_{++}^n \rightarrow S_{++}^{n'}, n' \leq n$. This type of function may be combined with the Bures-Wasserstein distance, similarity and dissimilarity transforms in the problem:

$$\min_{\theta} \sum_{(i,j) \in \mathcal{S}} \min_{U \in \mathcal{O}(p)} \|f_\theta(X_i^{\frac{1}{2}}) - f_\theta(X_j^{\frac{1}{2}})U^T\|_F^2 - \sum_{(i,j) \in \mathcal{D}} \min_{U \in \mathcal{O}(p)} \|f_\theta(X_i^{\frac{1}{2}}) - f_\theta(X_j^{\frac{1}{2}})U^T\|_F^2.$$

The benefits of the SPD embedding network f_θ are two-fold. The first is that it gives a more expressive set of discriminative functions with respect to the Bures-Wasserstein distance. The second is that, by reducing the dimensionality of the

data, evaluation of the distance function becomes cheaper. This was a barrier to scalability when learning with the Bures-Wasserstein distance and addressing this will reduce these issues.

6.2 Final Remark

In conclusion, we have seen that the Bures-Wasserstein distance is a flexible and powerful tool for Linear Metric Learning. We believe that the most ground may be gained by examining the use of the Bures-Wasserstein distance in SPD Metric Learning, as it presents a learnable distance that has gone largely unused. Riemannian optimisation has also become more widely used in Machine Learning in recent years, and the Bures-Wasserstein distance is poised to follow this trend. We hope that these types of algorithms lead to better understanding of data and meaningful applications to medicine, science and society.

Bibliography

- [1] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [2] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman, *The elements of statistical learning: data mining, inference, and prediction*, vol. 2. Springer, 2009.
- [3] J. MacQueen, “Classification and analysis of multivariate observations,” in *5th Berkeley Symp. Math. Statist. Probability*, pp. 281–297, 1967.
- [4] D. Xu and Y. Tian, “A comprehensive survey of clustering algorithms,” *Annals of Data Science*, vol. 2, no. 2, pp. 165–193, 2015.
- [5] R. Datta, D. Joshi, J. Li, and J. Z. Wang, “Image retrieval: Ideas, influences, and trends of the new age,” *ACM Computing Surveys (Csur)*, vol. 40, no. 2, pp. 1–60, 2008.
- [6] M. Kobayashi and K. Takeda, “Information retrieval on the web,” *ACM Computing Surveys (CSUR)*, vol. 32, no. 2, pp. 144–173, 2000.
- [7] A. Bellet, A. Habrard, and M. Sebban, “A survey on metric learning for feature vectors and structured data,” *arXiv preprint arXiv:1306.6709*, 2013.
- [8] B. Ghojogh, A. Ghodsi, F. Kararray, and M. Crowley, “Spectral, probabilistic, and deep metric learning: Tutorial and survey,” *arXiv preprint arXiv:2201.09267*, 2022.

- [9] B. Kulis *et al.*, “Metric learning: A survey,” *Foundations and Trends in Machine Learning*, vol. 5, no. 4, pp. 287–364, 2012.
- [10] E. P. Xing, A. Y. Ng, M. I. Jordan, and S. Russell, “Distance metric learning with application to clustering with side-information,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 15, pp. 505–512, 2002.
- [11] P. C. Mahalanobis, “On the generalized distance in statistics,” National Institute of Science of India, 1936.
- [12] J. V. Davis, B. Kulis, P. Jain, S. Sra, and I. S. Dhillon, “Information-theoretic metric learning,” in *Proceedings of the 24th International Conference on Machine Learning (ICML)*, p. 209–216, 2007.
- [13] A. Globerson and S. Roweis, “Metric learning by collapsing classes,” in *Advances in Neural Information Processing Systems (NIPS)*, pp. 451–458, 2005.
- [14] K. Q. Weinberger and L. K. Saul, “Distance metric learning for large margin nearest neighbor classification,” *Journal of Machine Learning Research*, vol. 10, no. 9, pp. 207–244, 2009.
- [15] D. Kedem, S. Tyree, F. Sha, G. Lanckriet, and K. Q. Weinberger, “Non-linear metric learning,” *Advances in neural information processing systems*, vol. 25, 2012.
- [16] M. Guillaumin, J. Verbeek, and C. Schmid, “Is that you? metric learning approaches for face identification,” in *2009 IEEE 12th international conference on computer vision*, pp. 498–505, IEEE, 2009.
- [17] M. Koestinger, M. Hirzer, P. Wohlhart, P. M. Roth, and H. Bischof, “Large scale metric learning from equivalence constraints,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2288–2295, 2012.

- [18] J. Goldberger, G. E. Hinton, S. Roweis, and R. R. Salakhutdinov, "Neighbourhood components analysis," in *Advances in Neural Information Processing Systems (NIPS)*, pp. 513–520, 2004.
- [19] I. Csiszár, "I-divergence geometry of probability distributions and minimization problems," *The annals of probability*, pp. 146–158, 1975.
- [20] M. Kaya and H. Ş. Bilge, "Deep metric learning: A survey," *Symmetry*, vol. 11, no. 9, p. 1066, 2019.
- [21] O. Tuzel, F. Porikli, and P. Meer, "Region covariance: A fast descriptor for detection and classification," in *European conference on computer vision*, pp. 589–600, Springer, 2006.
- [22] O. Tuzel, F. Porikli, and P. Meer, "Pedestrian detection via classification on Riemannian manifolds," *IEEE transactions on pattern analysis and machine intelligence*, vol. 30, no. 10, pp. 1713–1727, 2008.
- [23] X. Pennec, P. Fillard, and N. Ayache, "A Riemannian framework for tensor computing," *International Journal of computer vision*, vol. 66, no. 1, pp. 41–66, 2006.
- [24] M. T. Harandi, M. Salzmann, and R. Hartley, "From manifold to manifold: Geometry-aware dimensionality reduction for SPD matrices," in *European conference on computer vision*, pp. 17–32, Springer, 2014.
- [25] Z. Huang, R. Wang, S. Shan, X. Li, and X. Chen, "Log-euclidean metric learning on symmetric positive definite manifold with application to image set classification," in *International conference on machine learning*, pp. 720–729, PMLR, 2015.
- [26] R. Wang, H. Guo, L. S. Davis, and Q. Dai, "Covariance discriminative learning: A natural and efficient approach to image set classification," in *2012 IEEE conference on computer vision and pattern recognition*, pp. 2496–2503, IEEE, 2012.

- [27] G. Peyré, M. Cuturi, *et al.*, “Computational optimal transport: With applications to data science,” *Foundations and Trends in Machine Learning*, vol. 11, no. 5-6, pp. 355–607, 2019.
- [28] C. Villani, *Optimal transport: old and new*, vol. 338. Berlin/Heidelberg: Springer, 2009.
- [29] F. Otto, “The geometry of dissipative evolution equations: the porous medium equation,” 2001.
- [30] Y. Chen, T. T. Georgiou, and M. Pavon, “Optimal transport in systems and control,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, 2021.
- [31] A. Galichon, *Optimal transport methods in economics*. Princeton University Press, 2018.
- [32] J.-D. Benamou and Y. Brenier, “A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem,” *Numerische Mathematik*, vol. 84, no. 3, pp. 375–393, 2000.
- [33] G. Monge, “Mémoire sur la théorie des déblais et des remblais,” *Mem. Math. Phys. Acad. Royale Sci.*, pp. 666–704, 1781.
- [34] L. V. Kantorovich, “On the translocation of masses,” in *Dokl. Akad. Nauk. USSR (NS)*, vol. 37, pp. 199–201, 1942.
- [35] C. Gini, “Sulla misura della concentrazione e della variabilità dei caratteri,” *Atti del Reale Istituto veneto di scienze, lettere ed arti*, vol. 73, pp. 1203–1248, 1914.
- [36] C. L. Mallows, “A note on asymptotic joint normality,” *The Annals of Mathematical Statistics*, pp. 508–515, 1972.
- [37] H. Tanaka, “An inequality for a functional of probability distributions and its application to Kac’s one-dimensional model of a Maxwellian gas,”

- Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, vol. 27, no. 1, pp. 47–52, 1973.
- [38] L. N. Vaserstein, “Markov processes over denumerable products of spaces, describing large systems of automata,” *Problemy Peredachi Informatsii*, vol. 5, no. 3, pp. 64–72, 1969.
- [39] S. Rachev and L. Rüschendorf, *Mass Transportation Problems: Volume I: Theory*. Mass Transportation Problems, Springer, 1998.
- [40] Y. Brenier, “Polar factorization and monotone rearrangement of vector-valued functions,” *Communications on pure and applied mathematics*, vol. 44, no. 4, pp. 375–417, 1991.
- [41] L. Ambrosio, N. Gigli, and G. Savaré, *Gradient flows in metric spaces and in the space of probability measures*. Springer, 2008.
- [42] R. Jordan, D. Kinderlehrer, and F. Otto, “The variational formulation of the Fokker–Planck equation,” *SIAM journal on mathematical analysis*, vol. 29, no. 1, pp. 1–17, 1998.
- [43] Y. Rubner, C. Tomasi, and L. J. Guibas, “The earth mover’s distance as a metric for image retrieval,” *International journal of computer vision*, vol. 40, no. 2, pp. 99–121, 2000.
- [44] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 214–223, 2017.
- [45] J. Solomon, F. De Goes, G. Peyré, M. Cuturi, A. Butscher, A. Nguyen, T. Du, and L. Guibas, “Convolutional Wasserstein distances: Efficient optimal transportation on geometric domains,” *ACM Transactions on Graphics (ToG)*, vol. 34, no. 4, pp. 1–11, 2015.

- [46] C. Frogner, C. Zhang, H. Mobahi, M. Araya, and T. A. Poggio, "Learning with a Wasserstein loss," *Advances in neural information processing systems*, vol. 28, 2015.
- [47] D. Dowson and B. Landau, "The Fréchet distance between multivariate normal distributions," *Journal of multivariate analysis*, vol. 12, no. 3, pp. 450–455, 1982.
- [48] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems (NIPS)*, p. 6629–6640, 2017.
- [49] B. Muzellec and M. Cuturi, "Generalizing point embeddings using the Wasserstein space of elliptical distributions," *arXiv preprint arXiv:1805.07594*, 2018.
- [50] S. Srivastava, C. Li, and D. B. Dunson, "Scalable Bayes via barycenter in Wasserstein space," *The Journal of Machine Learning Research*, vol. 19, no. 1, pp. 312–346, 2018.
- [51] Y.-W. Luo and C.-X. Ren, "Conditional Bures metric for domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13989–13998, 2021.
- [52] R. Flamary, M. Cuturi, N. Courty, and A. Rakotomamonjy, "Wasserstein discriminant analysis," *Machine Learning*, vol. 107, no. 12, pp. 1923–1945, 2018.
- [53] T. Kerdoncuff, R. Emonet, and M. Sebban, "Metric learning in optimal transport for domain adaptation," in *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pp. 2162–2168, 2021.

- [54] J. Xu, L. Luo, C. Deng, and H. Huang, "Multi-level metric learning via smoothed Wasserstein distance," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2919–2925, 2018.
- [55] A. Han, B. Mishra, P. K. Jawanpuria, and J. Gao, "On Riemannian optimization over positive definite matrices with the Bures-Wasserstein geometry," in *Advances in Neural Information Processing Systems* (M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, eds.), vol. 34, pp. 8940–8953, Curran Associates, Inc., 2021.
- [56] C. Sun, A. Shrivastava, S. Singh, and A. Gupta, "Revisiting unreasonable effectiveness of data in deep learning era," in *Proceedings of the IEEE international conference on computer vision*, pp. 843–852, 2017.
- [57] I. Fehervari, A. Ravichandran, and S. Appalaraju, "Unbiased evaluation of deep metric learning algorithms," *arXiv preprint arXiv:1911.12528*, 2019.
- [58] T. Liao, R. Taori, I. D. Raji, and L. Schmidt, "Are we learning yet? a meta review of evaluation failures across machine learning," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- [59] K. Musgrave, S. Belongie, and S.-N. Lim, "A metric learning reality check," in *European Conference on Computer Vision*, pp. 681–699, Springer, 2020.
- [60] K. Roth, T. Milbich, S. Sinha, P. Gupta, B. Ommer, and J. P. Cohen, "Revisiting training strategies and generalization performance in deep metric learning," in *International Conference on Machine Learning*, pp. 8242–8252, PMLR, 2020.
- [61] M. Kusner, Y. Sun, N. Kolkin, and K. Weinberger, "From word embeddings to document distances," in *International conference on machine learning*, pp. 957–966, PMLR, 2015.

- [62] M. Cuturi and D. Avis, "Ground metric learning," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 533–564, 2014.
- [63] G. Huang, C. Guo, M. J. Kusner, Y. Sun, F. Sha, and K. Q. Weinberger, "Supervised word mover's distance," *Advances in neural information processing systems*, vol. 29, 2016.
- [64] J. Mei, M. Liu, H. R. Karimi, and H. Gao, "Logdet divergence-based metric learning with triplet constraints and its applications," *IEEE Transactions on Image Processing*, vol. 23, no. 11, pp. 4920–4931, 2014.
- [65] B. Nguyen, C. Morell, and B. De Baets, "Supervised distance metric learning through maximization of the Jeffrey divergence," *Pattern Recognition*, vol. 64, pp. 215–225, 2017.
- [66] D. Bertsimas and J. N. Tsitsiklis, *Introduction to linear optimization*, vol. 6. Athena Scientific Belmont, MA, 1997.
- [67] O. Pele and M. Werman, "Fast and robust earth mover's distances," in *2009 IEEE 12th international conference on computer vision*, pp. 460–467, IEEE, 2009.
- [68] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transport," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 26, pp. 2292–2300, 2013.
- [69] L. Malagò, L. Montrucchio, and G. Pistone, "Wasserstein Riemannian geometry of Gaussian densities," *Information Geometry*, vol. 1, no. 2, pp. 137–179, 2018.
- [70] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*, vol. 4. Springer, 2006.
- [71] R. Tibshirani, "Regression shrinkage and selection via the Lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.

- [72] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the royal statistical society: series B (statistical methodology)*, vol. 67, no. 2, pp. 301–320, 2005.
- [73] D. L. Donoho, "Compressed sensing," *IEEE Transactions on information theory*, vol. 52, no. 4, pp. 1289–1306, 2006.
- [74] L. M. Bregman, "The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming," *USSR computational mathematics and mathematical physics*, vol. 7, no. 3, pp. 200–217, 1967.
- [75] I. S. Dhillon and J. A. Tropp, "Matrix nearness problems with Bregman divergences," *SIAM Journal on Matrix Analysis and Applications*, vol. 29, no. 4, pp. 1120–1146, 2008.
- [76] A. Banerjee, S. Merugu, I. S. Dhillon, J. Ghosh, and J. Lafferty, "Clustering with Bregman divergences.," *Journal of machine learning research*, vol. 6, no. 10, 2005.
- [77] H. H. Bauschke, J. M. Borwein, and P. L. Combettes, "Bregman monotone optimization algorithms," *SIAM Journal on control and optimization*, vol. 42, no. 2, pp. 596–636, 2003.
- [78] S. Sra and I. Dhillon, "Generalized nonnegative matrix approximations with Bregman divergences," *Advances in neural information processing systems*, vol. 18, 2005.
- [79] P. Ravikumar, M. J. Wainwright, G. Raskutti, and B. Yu, "High-dimensional covariance estimation by minimizing l1-penalized log-determinant divergence," *Electronic Journal of Statistics*, vol. 5, pp. 935–980, 2011.
- [80] M. Harandi, M. Salzmann, and F. Porikli, "Bregman divergences for infinite dimensional covariance matrices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1003–1010, 2014.

- [81] R. Bhatia, *Positive Definite Matrices*. Princeton University Press, 2007.
- [82] V. Arsigny, P. Fillard, X. Pennec, and N. Ayache, “Log-euclidean metrics for fast and simple calculus on diffusion tensors,” *Magnetic Resonance in Medicine: An Official Journal of the International Society for Magnetic Resonance in Medicine*, vol. 56, no. 2, pp. 411–421, 2006.
- [83] R. Bhatia, T. Jain, and Y. Lim, “On the Bures–Wasserstein distance between positive definite matrices,” *Expositiones Mathematicae*, vol. 37, no. 2, pp. 165–191, 2019.
- [84] S. Bonnabel and R. Sepulchre, “Riemannian metric and geometric mean for positive semidefinite matrices of fixed rank,” *SIAM Journal on Matrix Analysis and Applications*, vol. 31, no. 3, pp. 1055–1070, 2010.
- [85] B. Vandereycken, P.-A. Absil, and S. Vandewalle, “Embedded geometry of the set of symmetric positive semidefinite matrices of fixed rank,” in *2009 IEEE/SP 15th Workshop on Statistical Signal Processing*, pp. 389–392, IEEE, 2009.
- [86] B. Vandereycken, P.-A. Absil, and S. Vandewalle, “A Riemannian geometry with complete geodesics for the set of positive semidefinite matrices of fixed rank,” *IMA Journal of Numerical Analysis*, vol. 33, no. 2, pp. 481–514, 2013.
- [87] E. Massart and P.-A. Absil, “Quotient geometry with simple geodesics for the manifold of fixed-rank positive-semidefinite matrices,” *SIAM Journal on Matrix Analysis and Applications*, vol. 41, no. 1, pp. 171–198, 2020.
- [88] B. Mishra, G. Meyer, and R. Sepulchre, “Low-rank optimization for distance matrix completion,” in *2011 50th IEEE Conference on Decision and Control and European Control Conference*, pp. 4455–4460, IEEE, 2011.
- [89] B. Szczapa, M. Daoudi, S. Berretti, A. Del Bimbo, P. Pala, and E. Massart, “Fitting, comparison, and alignment of trajectories on positive semidefinite matrices with application to action recognition,” in *Proceedings of*

- the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 0–0, 2019.
- [90] D. Sabbagh, P. Ablin, G. Varoquaux, A. Gramfort, and D. A. Engemann, “Manifold-regression to predict from MEG/EEG brain signals without source modeling,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [91] A. Edelman, T. A. Arias, and S. T. Smith, “The geometry of algorithms with orthogonality constraints,” *SIAM journal on Matrix Analysis and Applications*, vol. 20, no. 2, pp. 303–353, 1998.
- [92] L. Eldén and H. Park, “A Procrustes problem on the Stiefel manifold,” *Numerische Mathematik*, vol. 82, no. 4, pp. 599–619, 1999.
- [93] J. P. Cunningham and Z. Ghahramani, “Linear dimensionality reduction: Survey, insights, and generalizations,” *The Journal of Machine Learning Research*, vol. 16, no. 1, pp. 2859–2900, 2015.
- [94] H. H. Bauschke, P. L. Combettes, *et al.*, *Convex analysis and monotone operator theory in Hilbert spaces*, vol. 408. Springer, 2011.
- [95] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.
- [96] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 1999.
- [97] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical programming*, vol. 45, no. 1, pp. 503–528, 1989.
- [98] D. P. Bertsekas, “On the Goldstein-Levitin-Polyak gradient projection method,” *IEEE Transactions on automatic control*, vol. 21, no. 2, pp. 174–184, 1976.

- [99] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, *et al.*, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [100] Y. Censor, S. A. Zenios, *et al.*, *Parallel optimization: Theory, algorithms, and applications*. Oxford University Press, 1997.
- [101] Y. Censor and A. Lent, “An iterative row-action method for interval convex programming,” *Journal of Optimization theory and Applications*, vol. 34, no. 3, pp. 321–353, 1981.
- [102] N. Boumal, B. Mishra, P.-A. Absil, and R. Sepulchre, “Manopt, a Matlab toolbox for optimization on manifolds,” *Journal of Machine Learning Research*, vol. 15, no. 42, pp. 1455–1459, 2014.
- [103] P.-A. Absil, R. Mahony, and R. Sepulchre, “Optimization algorithms on matrix manifolds,” in *Optimization Algorithms on Matrix Manifolds*, Princeton University Press, 2009.
- [104] N. Boumal, “An introduction to optimization on smooth manifolds,” *Available online, May*, vol. 3, 2020.
- [105] R. L. Adler, J.-P. Dedieu, J. Y. Margulies, M. Martens, and M. Shub, “Newton’s method on Riemannian manifolds and a geometric model for the human spine,” *IMA Journal of Numerical Analysis*, vol. 22, no. 3, pp. 359–390, 2002.
- [106] W. Huang, P.-A. Absil, and K. A. Gallivan, “A Riemannian BFGS method without differentiated retraction for nonconvex optimization problems,” *SIAM Journal on Optimization*, vol. 28, no. 1, pp. 470–495, 2018.
- [107] P.-A. Absil, C. G. Baker, and K. A. Gallivan, “Trust-region methods on Riemannian manifolds,” *Foundations of Computational Mathematics*, vol. 7, no. 3, pp. 303–330, 2007.

- [108] R. Sinkhorn, "Diagonal equivalence to matrices with prescribed row and column sums," *The American Mathematical Monthly*, vol. 74, no. 4, pp. 402–405, 1967.
- [109] J. Altschuler, J. Niles-Weed, and P. Rigollet, "Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration," *Advances in neural information processing systems*, vol. 30, 2017.
- [110] P. Dvurechensky, A. Gasnikov, and A. Kroshnin, "Computational optimal transport: Complexity by accelerated gradient descent is better than by Sinkhorn's algorithm," in *International conference on machine learning*, pp. 1367–1376, PMLR, 2018.
- [111] S. Guminov, P. Dvurechensky, N. Tupitsa, and A. Gasnikov, "On a combination of alternating minimization and Nesterov's momentum," in *Proceedings of the 38th International Conference on Machine Learning* (M. Meila and T. Zhang, eds.), vol. 139 of *Proceedings of Machine Learning Research*, pp. 3886–3898, PMLR, 18–24 Jul 2021.
- [112] M. Gelbrich, "On a formula for the L2 Wasserstein metric between measures on Euclidean and Hilbert spaces," *Mathematische Nachrichten*, vol. 147, no. 1, pp. 185–203, 1990.
- [113] P. C. Álvarez-Esteban, E. Del Barrio, J. Cuesta-Albertos, and C. Matrán, "A fixed-point approach to barycenters in Wasserstein space," *Journal of Mathematical Analysis and Applications*, vol. 441, no. 2, pp. 744–762, 2016.
- [114] V. M. Panaretos and Y. Zemel, "Statistical aspects of Wasserstein distances," *Annual review of statistics and its application*, vol. 6, pp. 405–431, 2019.
- [115] S. Chewi, T. Maunu, P. Rigollet, and A. J. Stromme, "Gradient descent algorithms for Bures-Wasserstein barycenters," in *Conference on Learning Theory*, pp. 1276–1304, PMLR, 2020.

- [116] M. Agueh and G. Carlier, "Barycenters in the Wasserstein space," *SIAM Journal on Mathematical Analysis*, vol. 43, no. 2, pp. 904–924, 2011.
- [117] A. Takatsu, "Wasserstein geometry of Gaussian measures," *Osaka Journal of Mathematics*, vol. 48, no. 4, pp. 1005–1026, 2011.
- [118] M. Journée, F. Bach, P.-A. Absil, and R. Sepulchre, "Low-rank optimization on the cone of positive semidefinite matrices," *SIAM Journal on Optimization*, vol. 20, no. 5, pp. 2327–2351, 2010.
- [119] G. H. Golub and C. F. Van Loan, *Matrix computations*. JHU press, 2013.
- [120] S. Wold, K. Esbensen, and P. Geladi, "Principal component analysis," *Chemometrics and intelligent laboratory systems*, vol. 2, no. 1-3, pp. 37–52, 1987.
- [121] M. Harandi, M. Salzmann, and R. Hartley, "Joint dimensionality reduction and metric learning: A geometric take," in *International Conference on Machine Learning*, pp. 1404–1413, PMLR, 2017.
- [122] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
- [123] S. T. Roweis and L. K. Saul, "Nonlinear dimensionality reduction by locally linear embedding," *science*, vol. 290, no. 5500, pp. 2323–2326, 2000.
- [124] J. B. Tenenbaum, V. d. Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *science*, vol. 290, no. 5500, pp. 2319–2323, 2000.
- [125] B. Kulis, M. A. Sustik, and I. S. Dhillon, "Low-rank kernel learning with Bregman matrix divergences," *Journal of Machine Learning Research*, vol. 10, no. 13, pp. 341–376, 2009.
- [126] G. Baudat and F. Anouar, "Generalized discriminant analysis using a kernel approach," *Neural computation*, vol. 12, no. 10, pp. 2385–2404, 2000.

- [127] S. Jayasumana, R. Hartley, M. Salzmann, H. Li, and M. Harandi, "Kernel methods on the Riemannian manifold of symmetric positive definite matrices," in *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 73–80, 2013.
- [128] M. Harandi, M. Salzmann, and R. Hartley, "Dimensionality reduction on SPD manifolds: The emergence of geometry-aware methods," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 1, pp. 48–62, 2017.
- [129] R. Sivalingam, V. Morellas, D. Boley, and N. Papanikolopoulos, "Metric learning for semi-supervised clustering of region covariance descriptors," in *2009 Third ACM/IEEE International Conference on Distributed Smart Cameras (ICDSC)*, pp. 1–8, IEEE, 2009.
- [130] R. Vemulapalli and D. W. Jacobs, "Riemannian metric learning for symmetric positive definite matrices," *arXiv preprint arXiv:1501.02393*, 2015.
- [131] S. Sra and A. Cherian, "Generalized dictionary learning for symmetric positive definite matrices with application to nearest neighbor retrieval," in *Joint European conference on machine learning and knowledge discovery in databases*, pp. 318–332, Springer, 2011.
- [132] A. Cherian, P. Stanitsas, M. Harandi, V. Morellas, and N. Papanikolopoulos, "Learning discriminative $\alpha\beta$ -divergences for positive definite matrices," in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 4280–4289, IEEE, 2017.
- [133] A. Cherian, P. Stanitsas, J. Wang, M. Harandi, V. Morellas, and N. Papanikolopoulos, "Learning log-determinant divergences for positive definite matrices," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5088–5102, 2021.

- [134] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.
- [135] A. Han, B. Mishra, P. Jawanpuria, and J. Gao, "Generalized Bures-Wasserstein geometry for positive definite matrices," *arXiv preprint arXiv:2110.10464*, 2021.
- [136] M. Talagrand, "Transportation cost for Gaussian and other product measures," *Geometric & Functional Analysis GAFA*, vol. 6, no. 3, pp. 587–600, 1996.
- [137] Y. LeCun, C. Cortes, and C. Burges, "MNIST handwritten digit database," *ATT Labs Available: <http://yann.lecun.com/exdb/mnist>*, vol. 2, 2010.
- [138] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [139] M. S. Bartlett, "An Inverse Matrix Adjustment Arising in Discriminant Analysis," *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 107 – 111, 1951.
- [140] W. Kirk and B. Panyanak, "A concept of convergence in geodesic spaces," *Nonlinear analysis: theory, methods & applications*, vol. 68, no. 12, pp. 3689–3696, 2008.
- [141] W. Kirk and N. Shahzad, *Fixed point theory in distance spaces*. Springer, 2014.
- [142] R. Bergmann, J. Persch, and G. Steidl, "A parallel Douglas–Rachford algorithm for minimizing ROF-like functionals on images with values in symmetric hadamard manifolds," *SIAM Journal on Imaging Sciences*, vol. 9, no. 3, pp. 901–937, 2016.
- [143] A. Bërdëllima and G. Steidl, "Quasi α -firmly nonexpansive mappings in Wasserstein spaces," *arXiv preprint arXiv:2203.04851*, 2022.

- [144] J. Malick, "A dual approach to semidefinite least-squares problems," *SIAM Journal on Matrix Analysis and Applications*, vol. 26, no. 1, pp. 272–284, 2004.
- [145] S. Boyd and L. Xiao, "Least-squares covariance matrix adjustment," *SIAM Journal on Matrix Analysis and Applications*, vol. 27, no. 2, pp. 532–546, 2005.
- [146] H. Qi and D. Sun, "A quadratically convergent newton method for computing the nearest correlation matrix," *SIAM journal on matrix analysis and applications*, vol. 28, no. 2, pp. 360–385, 2006.
- [147] R. A. Horn and C. R. Johnson, *Matrix analysis*. Cambridge university press, 2012.
- [148] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Transactions on mathematical software (TOMS)*, vol. 23, no. 4, pp. 550–560, 1997.
- [149] J. Fiala, M. Kočvara, and M. Stingl, "Penlab: A MATLAB solver for nonlinear semidefinite optimization," *arXiv preprint arXiv:1311.5240*, 2013.
- [150] D. Dua and C. Graff, "UCI machine learning repository," 2017.
- [151] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [152] G. Meyer, S. Bonnabel, and R. Sepulchre, "Regression on fixed-rank positive semidefinite matrices: a riemannian approach," *The Journal of Machine Learning Research*, vol. 12, pp. 593–625, 2011.
- [153] S. Burer and R. D. Monteiro, "A nonlinear programming algorithm for solving semidefinite programs via low-rank factorization," *Mathematical Programming*, vol. 95, no. 2, pp. 329–357, 2003.
- [154] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE.," *Journal of machine learning research*, vol. 9, no. 11, 2008.

- [155] Z. Lin, "Riemannian geometry of symmetric positive definite matrices via Cholesky decomposition," *SIAM Journal on Matrix Analysis and Applications*, vol. 40, no. 4, pp. 1353–1370, 2019.
- [156] S. Sra, "A new metric on the manifold of kernel matrices with application to matrix geometric means," *Advances in neural information processing systems*, vol. 25, 2012.
- [157] B. Leibe and B. Schiele, "Analyzing appearance and contour based methods for object categorization," in *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, vol. 2, pp. II–409, IEEE, 2003.
- [158] H. Jhuang, J. Gall, S. Zuffi, C. Schmid, and M. J. Black, "Towards understanding action recognition," in *Proceedings of the IEEE international conference on computer vision*, pp. 3192–3199, 2013.
- [159] R. H. Tütüncü, K.-C. Toh, and M. J. Todd, "Solving semidefinite-quadratic-linear programs using SDPT3," *Mathematical programming*, vol. 95, no. 2, pp. 189–217, 2003.
- [160] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using gabor filters," *Pattern recognition*, vol. 24, no. 12, pp. 1167–1186, 1991.
- [161] Z. Huang and L. Van Gool, "A Riemannian network for SPD matrix learning," in *Thirty-first AAAI conference on artificial intelligence*, 2017.